



智能计算系统

第二章 神经网络基础

北京邮电大学人工智能学院，何召锋

北京邮电大学人工智能学院，徐振博

北京邮电大学人工智能学院，项刘宇

中国科学院计算技术研究所，李威

上一章重点

- ▶ 为什么要开这门课
- ▶ 人工智能（发展现状、流派、趋势等）
- ▶ 智能计算系统（概念和构成）
- ▶ 驱动范例

人工智能发展途径一

- ▶ 推进“大数据+大算力+强算法”的信息技术方法，从而得到信息模型
- ▶ 收集尽可能多的数据，采用深度学习、注意力模型等算法，将大数据中蕴藏的规律转换为人工神经网络的参数，这实际上是凝练了大数据精华的“隐式知识库”，可以为各类文本、图像等信息处理应用提供共性智能模型。

——黄铁军，北京智源人工智能研究院院长，北大教授，2021

人工智能发展途径二

- ▶ 推进“结构仿脑、功能类脑、性能超脑”的类脑途径，从而得到生命模型。
- ▶ 把大自然亿万年进化训练出的生物神经网络作为新一代人工神经网络的蓝本，构造逼近生物神经网络的神经形态芯片和系统，站在人类智能肩膀上发展机器智能。
——黄铁军，北京智源人工智能研究院院长，北大教授，2021

人工智能发展途径三

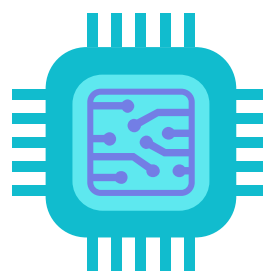
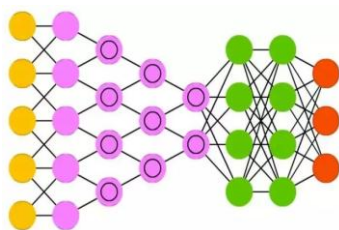
- ▶ 推进“强化学习+物理模型+算力”的自主学习途径，从而得到自主智能模型
 - ▶ 比如，构造地球物理模型，训练出的人工智能系统能够适应地球环境，与人类共处共融；
 - ▶ 比如，构造高精度物理模型（例如基于量子力学模型构造出粒子、原子、分子和材料模型），可以训练出能够从事物理学和材料学研究的人工智能；
 - ▶ 比如，构造出宇宙及其他星球的物理模型，可以训练出的人工智能则有望走出地球，适应宇宙中更复杂的环境。

——黄铁军，北京智源人工智能研究院院长，北大教授，2021

第三代智能计算系统展望

- ▶ 第三代智能计算系统将是通用人工智能（强人工智能）发育的沙盒虚拟世界
 - ▶ 大量独立的智能主体，足够丰富的虚拟世界，以及背后的无限计算能力
 - ▶ 智能主体将在其中成长，通过和环境的交互，形成感知、认知和逻辑能力，甚至理解虚拟世界、改造虚拟世界，从而具备通用智能

Driving Example



输入

建模

实现

运行

输出



神经网络基础



深度学习

第二章将学习到搭建一个神经网络需要的基本知识，为深入理解深度学习打下基础

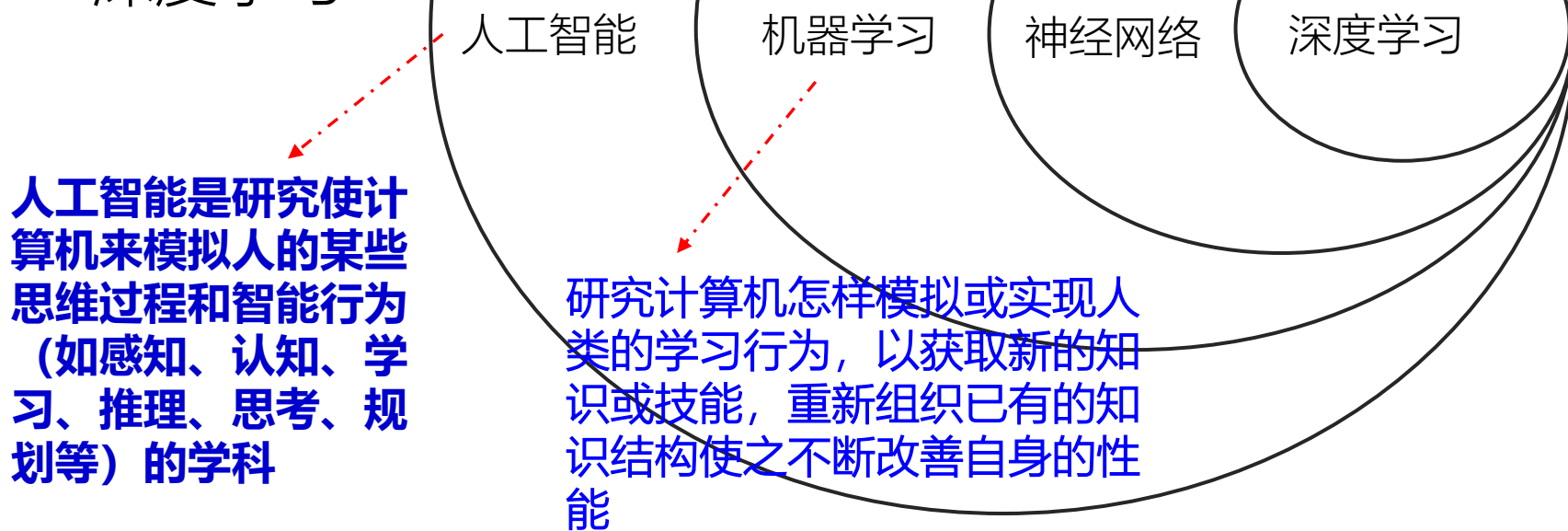
提纲

- ▶ 从机器学习到神经网络
- ▶ 神经网络训练
 - ▶ 正向传播（推理）
 - ▶ 反向传播（训练）
- ▶ 神经网络设计原则
 - ▶ 网络的拓扑结构
 - ▶ 激活函数
 - ▶ 损失函数
- ▶ 过拟合与正则化
- ▶ 交叉验证
- ▶ 小结

包含关系

- ▶ 人工智能
- ▶ 机器学习
- ▶ 神经网络
- ▶ 深度学习

讲深度学习之前先讲神经网络，在讲神经网络之前我们先要讲机器学习。

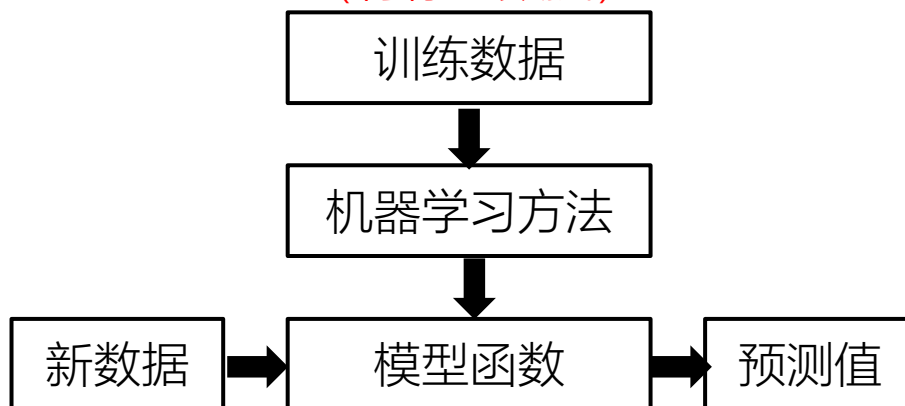


机器学习相关概念

研究计算机怎样模拟或实现人类的学习行为，以获取新的知识或技能，重新组织已有的知识结构使之不断改善自身的性能

- 机器学习是对能通过经验自动改进的计算机算法的研究 (Mitchell)
- 机器学习是用数据或以往的经验，以此提升计算机程序的能力 (Alpaydin)
- 机器学习是研究如何通过计算的手段、利用经验来改善系统自身性能的一门学科 (周志华)

典型（监督）机器学习过程
(有标签数据)



机器学习是机器通过训练逐渐提升自己的智能处理能力的方法

如何学习？ 如何理解？ 如何学会？

从最简单的线性回归模型开始，直至搭建出一个完整的神经网络架构

符号说明

输入数据： x

真实值（实际值）： y

计算值（模型输出值）： $\hat{y}$

模型函数： $H(x)$

激活函数： $G(x)$

损失函数： $L(x)$

标量：斜体小写字母 a 、 b 、 c

向量：黑斜体小写字母 a 、 b 、 c

矩阵：黑斜体大写字母 A 、 B 、 C

希腊字母表

序号	大写	小写	英文注音	国际音标注音	对应英文字母	中文读音	意义	对应数值
1	A	α	alpha	a:lfa	a	阿尔法	角度（小写）；系数	1
2	B	β	beta	beta	b, v	贝塔	磁通系数；磁感应强度（大写）；角度（小写）；系数	2
3	Γ	γ	gamma	ga:m	g, gh, y	伽马	电导系数（小写）	3
4	Δ	δ	delta	delta	d, dh, th	戴尔塔	变化量（大写）；屈光度	4
5	E	ϵ	epsilon	ep`silon	e	艾普西龙	对数之基数；电势能（小写）	5
6	Z	ζ	zeta	zeta	z	截塔	系数；方位角；阻抗；相对粘度；原子序数	7
7	H	η	eta	eta	e, i	艾塔	磁滞系数；效率（小写）	8
8	Θ	θ	theta	theta	th	西塔	温度；相位角	9
9	I	ι	iot	jot	i	约塔	微小，一点儿	10
10	K	κ	kappa	kap	k	卡帕	介质常数	20
11	Λ	λ	lambda	lamda	l	兰姆达	波长（小写）；体积	30
12	M	μ	mu	mju	m	缪	磁导系数微（千分之一）放大因数（小写）；动摩擦因数（小写）	40
13	N	ν	nu	nju	n	纽	磁阻系数；光频率（小写），中微子（小写）	50
14	Ξ	ξ	xi	ksi	x, ks	克西	随机变量	60
15	O	$\omicron$	omicron	omik`ron	o	奥密克戎	无穷小量： $o(x)$	70
16	Π	π	pi	pai	p	派	圆周率=圆周÷直径=3.14159 2653589793	80
17	P	ρ	rho	rou	r	柔	电阻系数（小写）密度(小写)	100
18	Σ	σ	sigma	`sigma	s	西格马	总和（大写），表面密度；跨导（小写）	200
19	T	τ	tau	to:	t	套	时间常数；周期（大写）	300
20	Y	υ	upsilon	jup`silon	u, y, v, f	宇普西龙	位移	400
21	Φ	ϕ	phi	fai	ph, f	法爱	微通（大写）；电势（小写）；黄金分割符号；工程中表示直径（大写与斜体）	500
22	X	χ	chi	khi	ch, kh	奇	卡方分布；电感	600
23	Ψ	ψ	psi	psai	ps	帕赛	角速；介质电通量（静电力线）；角	700
24	Ω	ω	omega	o`miga	o	欧米伽	欧姆（大写）；角速度（小写）；角	800

如何学习？ 如何理解？ 如何学会？

从最简单的线性回归模型开始，直至搭建出一个完整的神经网络架构

线性回归

什么是回归 (regression) 和线性回归?

- 一种用统计方法来分析一个因变量和一组自变量关系的一个方法

为什么从线性回归开始?

- 研究得最充分的、一个最基础的机器学习模型

问题引入:

- 假设房屋销售中心有这样一组关于房屋面积和房屋位置与销售价格的数据, 用 x_1 表示房屋面积, x_2 表示房屋楼层, y 表示售价 (万元)。

x_1	50	47	60	55	...	65
x_2	2	1	4	3	...	10
...					...	
y	50	42	80	52	...	?

设计一个回归
程序进行预测

进一步简化，只考虑面积

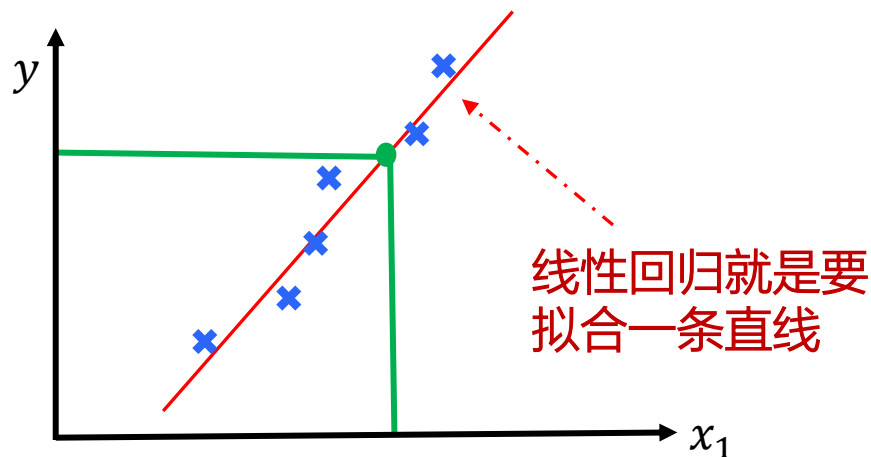
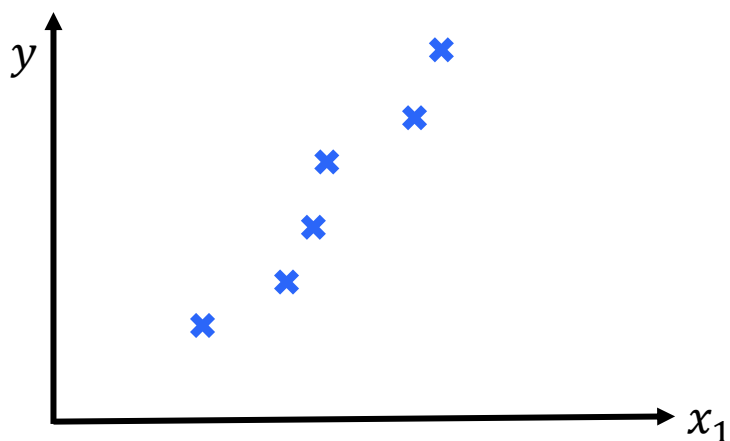
房屋面积 x_1 ，与售价 y 的关系

x_1	50	47	60	55	...	65
y	50	42	80	52	...	?

如果给出一个新的房屋面积，在销售记录中没有的，如何确定在该面积下的房屋售价 y 的值呢？

如预测 $x_1 = 65$ 时的售价？

解决办法->寻找 x_1 和 y 的之间的关系，使用已有记录数据进行训练



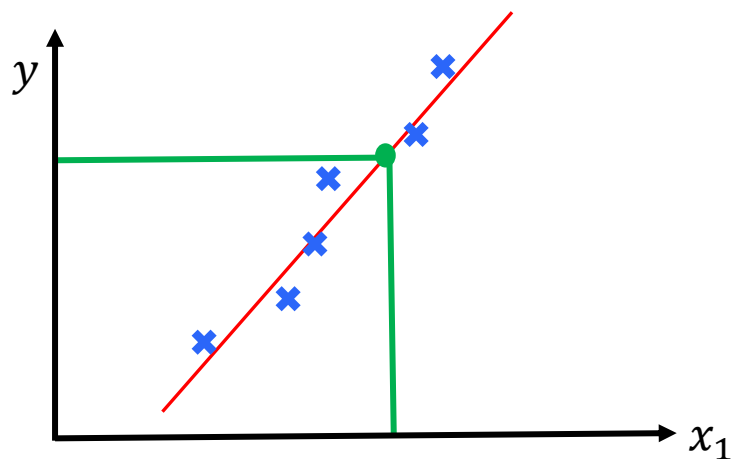
单变量线性回归模型（一元回归模型）

- 线性回归可以找到一些点的集合背后的规律：一个点集可以用一条直线来拟合，这条拟合出来的直线的参数特征，就是线性回归找到的点集背后的规律。

单变量线性模型

$$H_w(x) = w_0 + wx$$

x : Feature; $H(x)$: hypothesis



多变量线性回归模型

假设影响售价 y 的特征不仅仅只有房屋面积 x_1 ，还有楼层 x_2 ，房屋朝向

x_3 , x_4 , ... , x_n

多变量线性模型

单变量线性模型

$$H_w(x) = w_0 + wx$$



$$H_w(x) = w_0 + w_1x_1 + w_2x_2 \quad \text{2个特征}$$



$$H_w(x) = \sum_{i=0}^n w_i x_i = \hat{\mathbf{w}}^T \mathbf{x}, \quad \text{n个特征}$$

$$\hat{\mathbf{w}} = [w_0; w_1; \dots w_n]^T,$$

$$\mathbf{x} = [x_0; x_1; \dots x_n]^T, \quad x_0 = 1$$

多变量线性回归的方法其实应用方法广泛的，比如说经济学里好多的货币需求模型，预测消费支出，固定资产投资等，流行病学史上的一些分析，还有包括生物学科里，植物的生长模型等等都是可以用这个去分析的。

线性函数拟合好不好？

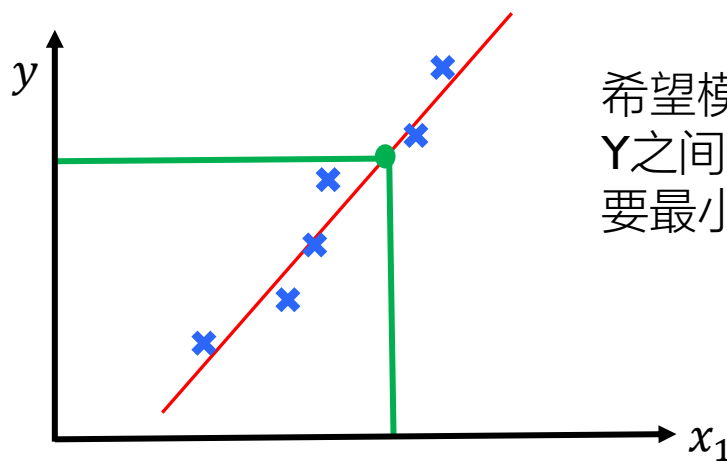
模型预测值 $\hat{y}$ 与真实值 y 之间存在误差 $\varepsilon = y - \hat{y} = y - \hat{\mathbf{w}}^T \mathbf{x}$



最小二乘法

损失
函数

$$L(\hat{\mathbf{w}}) = \frac{1}{2} \sum_{j=1}^m (H_{\mathbf{w}}(\mathbf{x}_j) - \mathbf{y}_j)^2 = \frac{1}{2} \sum_{j=1}^m (\hat{\mathbf{w}}^T \mathbf{x} - \mathbf{y}_j)^2$$



希望模型计算结果跟真实值
Y之间误差的平方累计起来
要最小

目标: 求出参数 $\hat{\mathbf{w}}$, 使得损失函数 $L(\hat{\mathbf{w}})$ 取值最小

使用均方误差的解释

先提出两个假设：

最小二乘法实际上是假设误差满足高斯分布且独立同分布情况下，使似然性最大化。

- 假设一：每一个样例 $(x^{(i)}, y^{(i)})$ ， x 和目标值 y 的关系： $y^{(i)} = \theta^T x^{(i)} + \varepsilon^{(i)}$ ，其中 $\varepsilon^{(i)}$ 表示 $\theta^T x^{(i)}$ 与目标值的误差。
- 假设二： $\varepsilon^{(i)}$ 服从正态分布： $\varepsilon \sim N(0, \sigma^2)$ 。

解释：根据中心极限定理——许多独立随机变量的和趋向于正态分布，因为影响误差的因素有很多，而这些因素都是独立且随机分布的，所得根据此可得假设二。

由此可得： $P(\varepsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(\varepsilon^{(i)})^2}{2\sigma^2}\right)$ ，从而也表示，当给定参数 θ 和 x

时，目标值 y 也服从正态分布，所以有：

$$P(y^{(i)} | x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(\theta^T x^{(i)} - y^{(i)})^2}{2\sigma^2}\right)。$$

使用均方误差的解释

- 假设三：对于误差 $\epsilon^{(i)}$ ，是IID（独立同分布）的随机变量。

根据这些假设，利用极大似然估计，来求解：

最大似然法：找到一组参数，使得似然函数最大。

似然函数：

$$l(\theta) = P(Y|x; \theta) = \prod_{i=1}^m P(y^{(i)}|x^{(i)}; \theta) = \prod_{i=1}^m \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(\theta^T x^{(i)} - y^{(i)})^2}{2\sigma^2}\right)$$

对似然函数取对数得：

$$L(\theta) = \log l(\theta) = \log \prod_{i=1}^m \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(\theta^T x^{(i)} - y^{(i)})^2}{2\sigma^2}\right) = \sum_{i=1}^m \log \left[\exp\left(-\frac{(\theta^T x^{(i)} - y^{(i)})^2}{2\sigma^2}\right) \right] = m \log \frac{1}{\sqrt{2\pi}\sigma} + \sum_{i=1}^m -\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}$$

所以，最大化 $L(\theta)$ 等价于最小化 $\sum_{i=1}^m \frac{(y^{(i)} - \theta^T x^{(i)})^2}{2} = J(\theta)$ ，即证得最小二乘法实际

上是在假设误差项满足高斯分布且独立同分布情况下，使似然性最大化。

寻找参数 $\hat{w}$ ->使得 $L(\hat{w})$ 最小

迭代法（**梯度下降法**）寻找参数

- 初始先给定一个 $\hat{w}$ ，如 $\mathbf{0}$ 向量或随机向量
- 沿着梯度下降的方向进行迭代，使更新后的 $L(\hat{w})$ 不断变小



$$L(\hat{w}) = \frac{1}{2} \sum_{j=1}^m (\hat{w}^T x - y_j)^2$$

$$\hat{w} = \hat{w} - \alpha \frac{\partial L(\hat{w})}{\partial \hat{w}}$$

α 称为学习率或步长

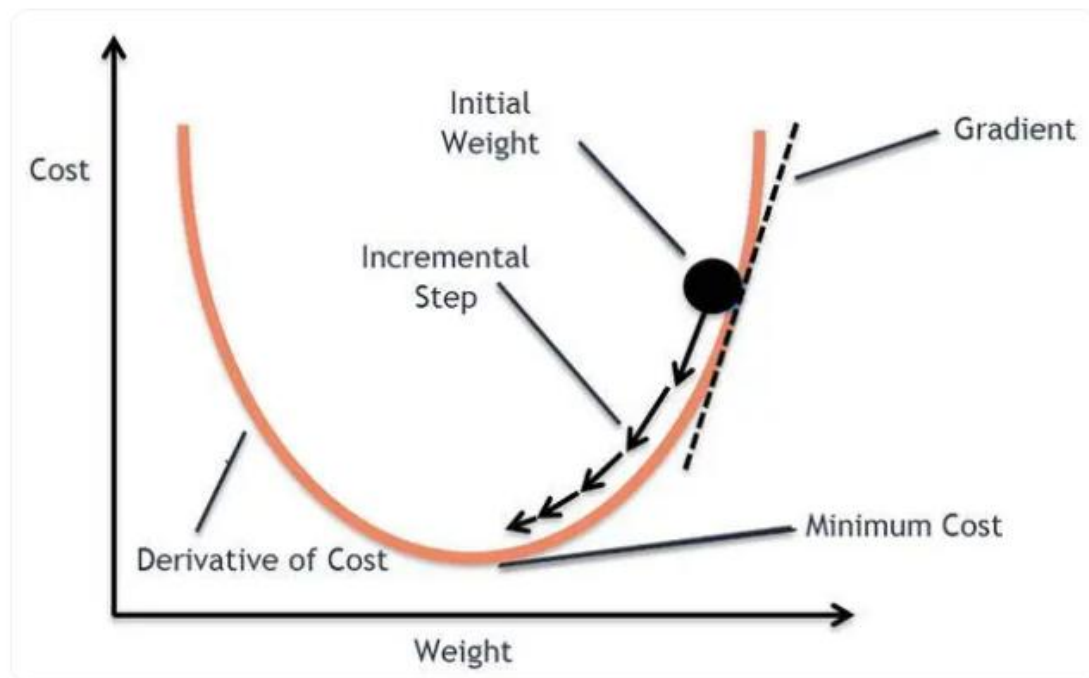


迭代至找到使得 $L(\hat{w})$ 最小的 $\hat{w}$ 值停止，从而得到回归模型参数

使损失函数最快下降到 $\mathbf{0}$ 的方向去调整 w ，调整后再看新的梯度是什么我们再调整

梯度下降法

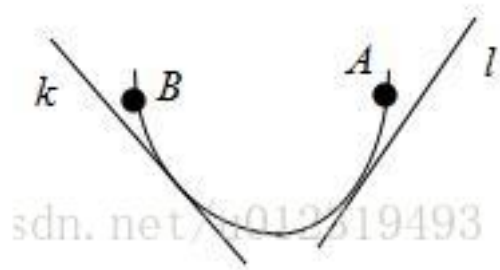
梯度下降最初是由柯西在1847年提出的。它也被称为最速下降。



梯度下降算法的目标是最小化给定函数（比如成本函数）。为了实现这一目标，它迭代地执行两个步骤：

1. 计算梯度（斜率），函数在该点的一阶导数
2. 在与梯度相反的方向上做一步（移动）

梯度下降法（为何要减梯度）



目标：对于一个代价函数，寻找其最小值，即最低点。

方法：x减去当前梯度，即斜率，然后再得到y。

解释：如果当前点是A点，则斜率是正数，减去后x变小，y变小。

如果当前点是B点，则斜率是负数，减去后x变大，y变小。

改进：为了避免下降过快，减去“学习率*梯度”，用学习率控制下降的快慢。

参考

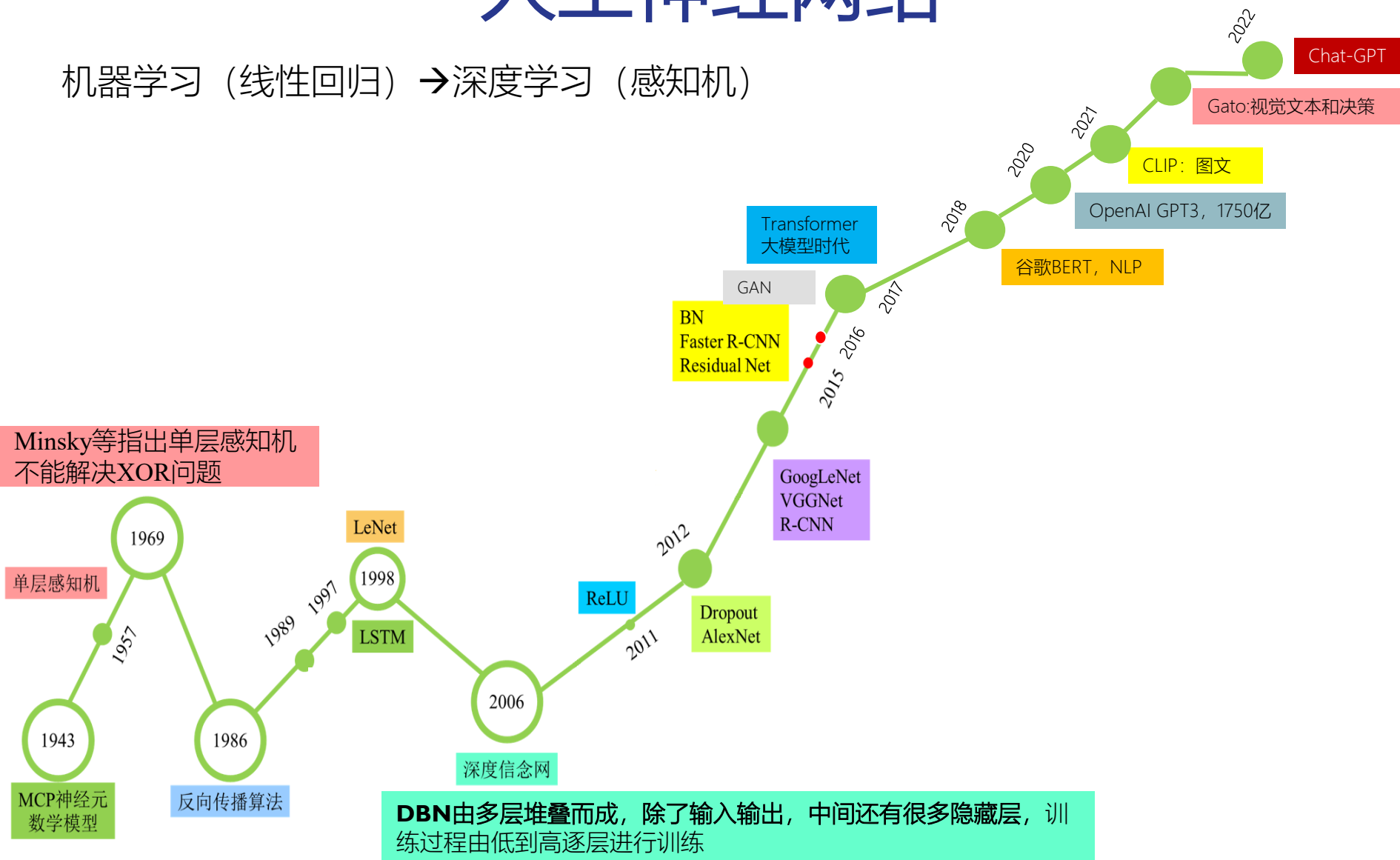
Michael Nielsen. 神经网络Q与深度学习[M]

线性回归→神经网络

从最简单的线性回归模型开始，直至搭建出一个完整的神经网络架构

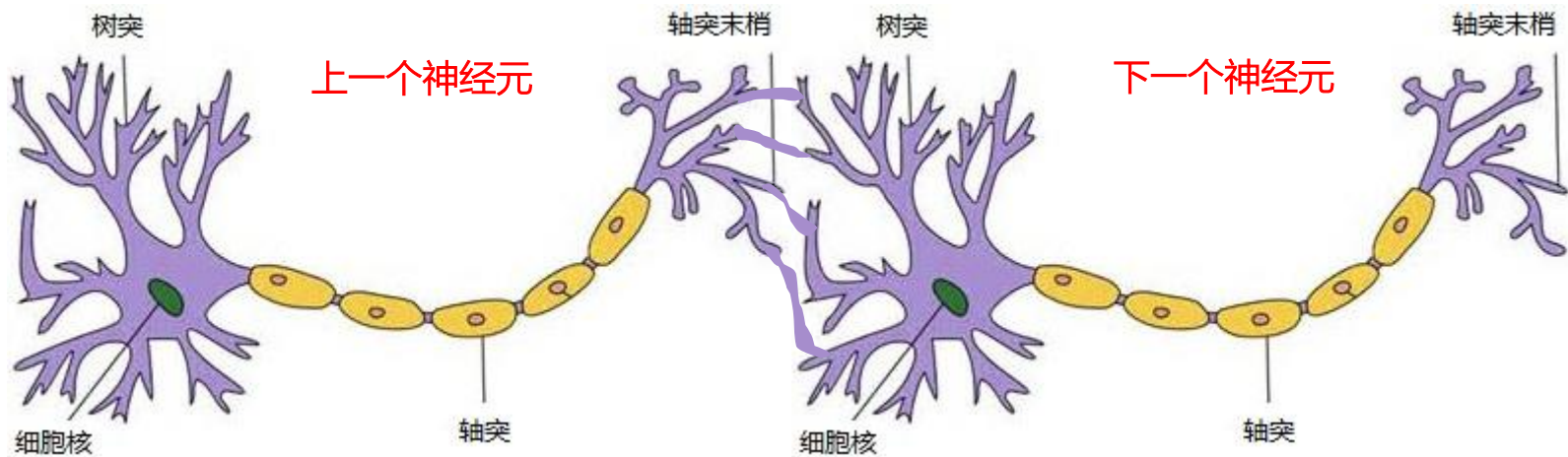
人工神经网络

机器学习（线性回归）→深度学习（感知机）



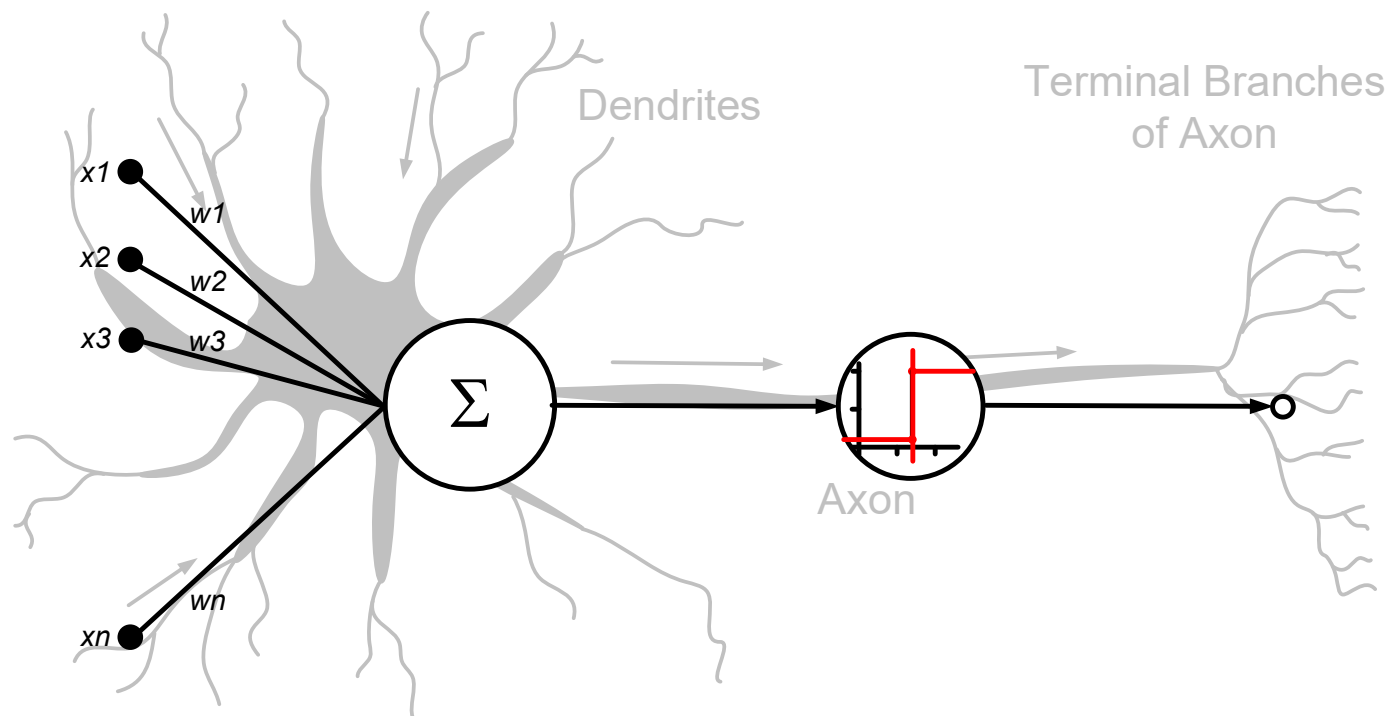
生物神经元

生物学领域，一个生物神经元有多个树突（接受传入信息）；有一条轴突，轴突尾端有许多轴突末梢（给其他多个神经元传递信息）。轴突末梢跟其它生物神经元的树突产生连接的位置叫做“突触”。



人工神经元

机器学习领域，**人工神经元是一个包含输入，输出与计算功能的模型**。不严格地说，其输入可类比为生物神经元的树突，其输出可类比为神经元的轴突，其计算可类比为细胞体。

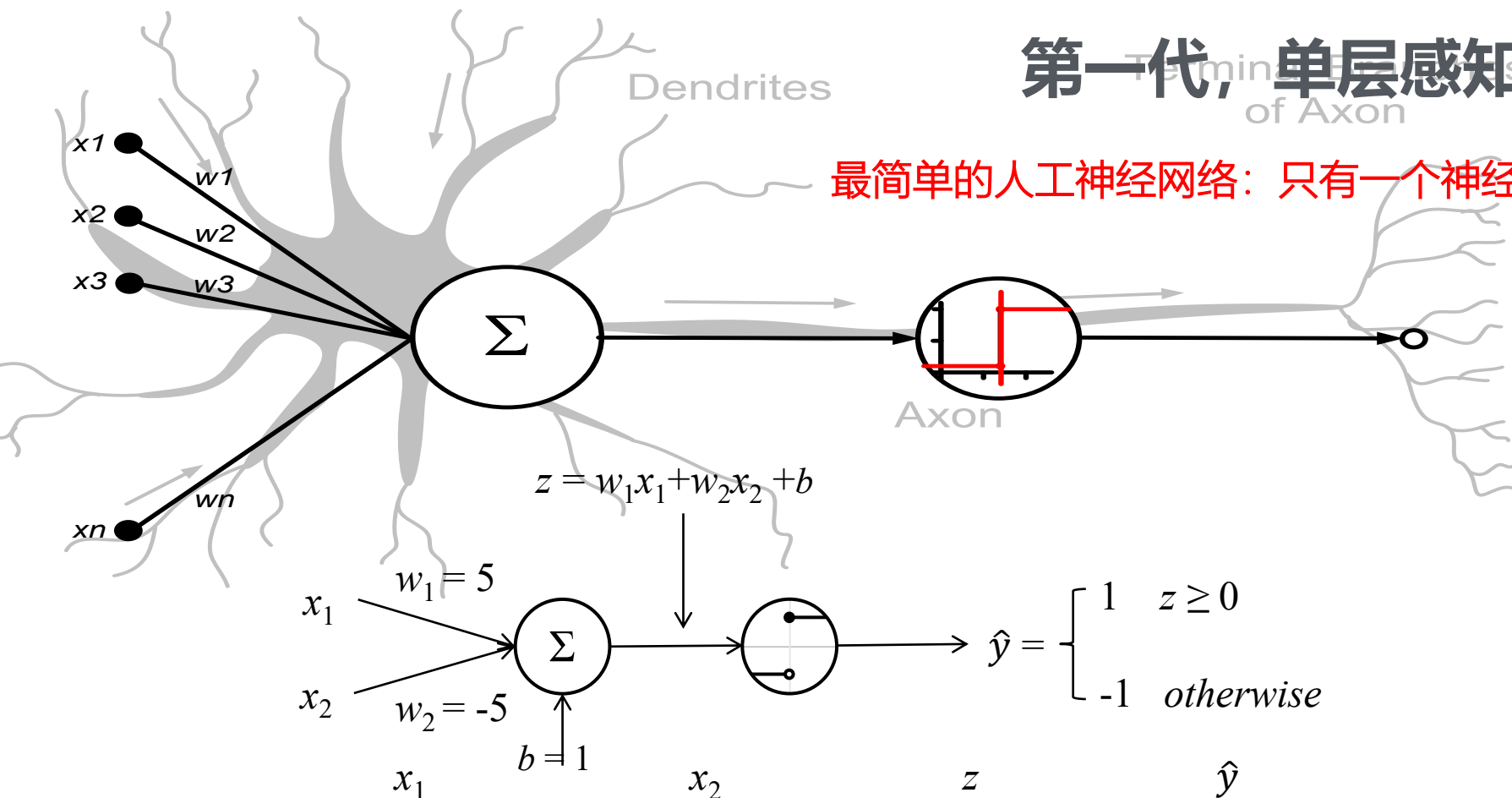


生物神经元：人工神经元=老鼠：米老鼠

一个神经元的单层感知机

第一代，单层感知机

最简单的人工神经网络：只有一个神经元



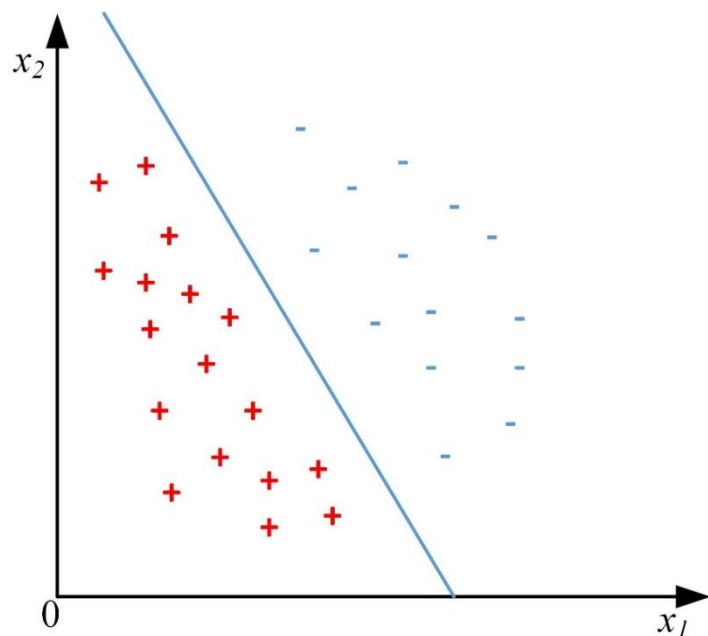
	1		-1	11	1
	-1		1	-9	-1

感知机 (Perceptron) 模型

感知机可以完成简单的线性分类任务。

n 维欧氏空间中余维度等于一的线性子空间

感知机模型 $H(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$ 对应一个超平面 $\mathbf{w}^T \mathbf{x} + b = 0$ ，模型参数是 $(\mathbf{w}, b)$ 。感知机模型训练的目标是找到一个 $(\mathbf{w}, b)$ ，将线性可分的数据集 T 中的所有样本点正确地分为两类。



$$H(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$$

$$\text{sign}(x) = \begin{cases} +1 & x \geq 0 \\ -1 & x < 0 \end{cases}$$



寻找损失函数，并
将损失函数最小化

点 $P_0(x_0, y_0)$ 到直线
 $l: Ax+By+C=0$ 的距离

$$d = \frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$$

➤ 寻找损失函数

考虑一个训练数据集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$ ，其中， $\mathbf{x}_j \in \mathbb{R}^n, y_j \in \{+1, -1\}$ 。如果存在某个超平面 $S: (\mathbf{w}^T \mathbf{x} + b = 0)$ ，能将正负样本分到 S 两侧，则说明数据集可分，那么，如何求出这个超平面 S 的表达式？

为何不是全部点？

策略：假设误分类的点为数据集 M ，使用**误分类点**到超平面的总距离来寻找损失函数（**直观来看，总距离越小越好**）

注：距离需满足的三个特性，非负、自反、三角不等式

样本点 $\mathbf{x}_j$ 到超平面 S 的距离：

$$d = \frac{1}{\|\mathbf{w}\|} |\mathbf{w}^T \mathbf{x}_j + b|$$

$\|\mathbf{w}\|$ 是 $\mathbf{w}$ 的 L^2 范数

➤ 寻找损失函数

数据集中误分类点满足条件: $-y_j(\mathbf{w}^T \mathbf{x}_j + b) > 0$

去掉点 $\mathbf{x}_j$ 到超平面S的距离的绝对值符号:

$$d = \frac{1}{\|\mathbf{w}\|} |\mathbf{w}^T \mathbf{x}_j + b| \quad \longrightarrow \quad d = -\frac{1}{\|\mathbf{w}\|} y_j(\mathbf{w}^T \mathbf{x}_j + b)$$

所有误分类点到超平面S的总距离为

$$d = -\frac{1}{\|\mathbf{w}\|} \sum_{\mathbf{x}_j \in M} y_j(\mathbf{w}^T \mathbf{x}_j + b)$$

由此寻找到感知机的损失函数 $L(\mathbf{w}, b) = - \sum_{\mathbf{x}_j \in M} y_j(\mathbf{w}^T \mathbf{x}_j + b)$

感知机算法

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$$

$$\text{sign}(x) = \begin{cases} +1 & x \geq 0 \\ -1 & x < 0 \end{cases}$$



损失函数

$$L(\mathbf{w}, b) = - \sum_{\mathbf{x}_j \in M} y_j (\mathbf{w}^T \mathbf{x}_j + b)$$

问题转化为寻找 $(\mathbf{w}, b)$ 使得损失函数极小化的最优化问题

损失函数极小化的最优化问题可使用：随机梯度下降法

$$L(\mathbf{w}, b) = - \sum_{x_j \in M} y_j (\mathbf{w}^T \mathbf{x}_j + b)$$

$$\nabla_{\mathbf{w}} L(\mathbf{w}, b) = - \sum_{x_j \in M} y_j \mathbf{x}_j \quad \Rightarrow \quad \mathbf{w} \leftarrow \mathbf{w} + \alpha y_j \mathbf{x}_j$$

$$\nabla_b L(\mathbf{w}, b) = - \sum_{x_j \in M} y_j \quad \Rightarrow \quad b \leftarrow b + \alpha y_j$$

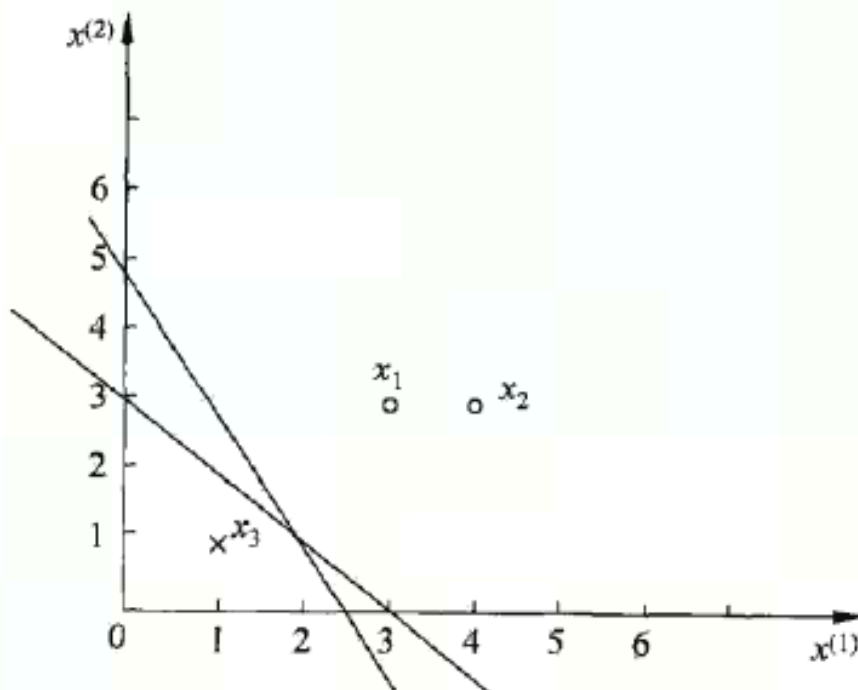
随机选取误分类点 (x_j, y_j)

对 $\mathbf{w}, b$ 以 α 为步长进行更新，通过迭代可以使得损失函数 $L(\mathbf{w}, b)$ 不断减小，直到为0

$$L(\mathbf{w}, b) \rightarrow 0$$

感知机算法实例

例2.1 如图2.2所示的训练数据集，其正实例点是 $x_1 = (3, 3)^T$, $x_2 = (4, 3)^T$, 负实例点是 $x_3 = (1, 1)^T$, 试用感知机学习算法的原始形式求感知机模型 $f(x) = \text{sign}(w \bullet x + b)$. 这里, $w = (w^{(1)}, w^{(2)})^T$, $x = (x^{(1)}, x^{(2)})^T$.



感知机算法实例

$$L(\mathbf{w}, b) = - \sum_{x_j \in M} y_j (\mathbf{w}^T \mathbf{x}_j + b) \quad \mathbf{w} \leftarrow \mathbf{w} + \alpha y_j \mathbf{x}_j \quad b \leftarrow b + \alpha y_j$$

1) 设 $\alpha=1$, 取初值 $\mathbf{w}_0 = 0$, $b_0=0$

2) 对 $\mathbf{x}_1 = (3, 3)^T$, 有 $y_1 ((\mathbf{w}_0 \mathbf{x}_1) + b_0) = 0$, 说明未能正确分类, 更新 $\mathbf{w}, b$:

$$\mathbf{w}_1 = \mathbf{w}_0 + y_1 \mathbf{x}_1 = (3, 3)^T, \quad b_1 = b_0 + y_1 = 1$$

3) 对于 $\mathbf{x}_2 = (4, 3)^T$, 有 $y_2 ((\mathbf{w}_1 \mathbf{x}_2) + b_1) > 0$, 说明被正确分类, 不更新 $\mathbf{w}, b$

感知机算法实例

4) 对于 $x_3 = (1, 1)^T$, 有 $y_3 ((w_1 x_3) + b_1) < 0$, 说明未能正确分类, 更新 w, b :

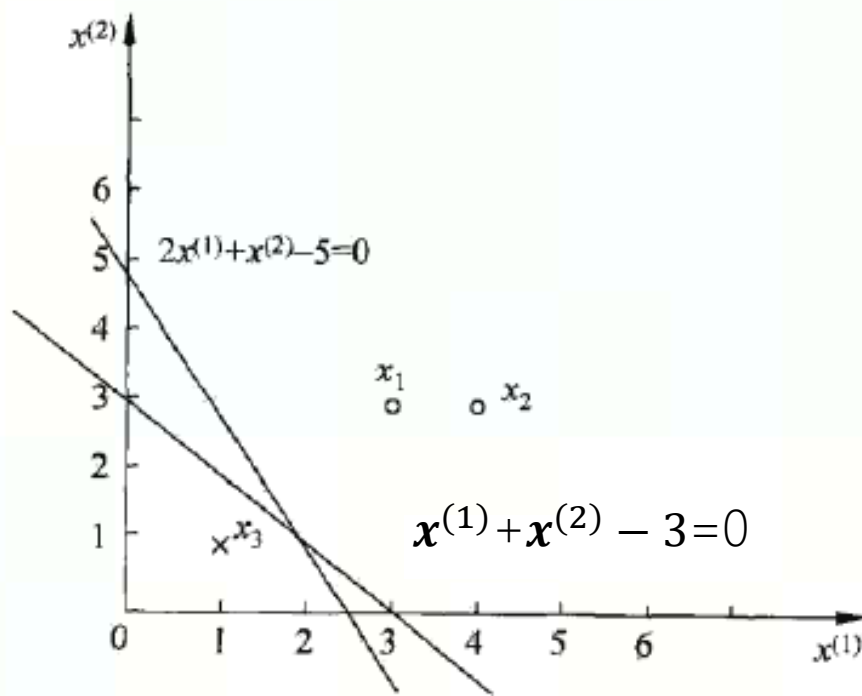
$$w_2 = w_1 + y_3 x_3 = (2, 2)^T, \quad b_2 = b_1 + y_3 = 0$$

迭代次数	误分类点	w	b	$w \cdot x + b$
0		0	0	0
1	x_1	$(3, 3)^T$	1	$3x^{(1)} + 3x^{(2)} + 1$
2	x_3	$(2, 2)^T$	0	$2x^{(1)} + 2x^{(2)}$
3	x_3	$(1, 1)^T$	-1	$x^{(1)} + x^{(2)} - 1$
4	x_3	$(0, 0)^T$	-2	-2
5	x_1	$(3, 3)^T$	-1	$3x^{(1)} + 3x^{(2)} - 1$
6	x_3	$(2, 2)^T$	-2	$2x^{(1)} + 2x^{(2)} - 2$
7	x_3	$(1, 1)^T$	-3	$x^{(1)} + x^{(2)} - 3$
8	0	$(1, 1)^T$	-3	$x^{(1)} + x^{(2)} - 3$

感知机算法实例

在计算中误分类点依次取 $x_1, x_3, x_3, x_3, x_1, x_3, x_3$, 得到分离超平面: $\mathbf{x}^{(1)} + \mathbf{x}^{(2)} - 3 = 0$

如果在计算误分类点时依次取 $x_1, x_3, x_3, x_3, x_2, x_3, x_3, x_3, x_1, x_3, x_3$, 则得到分离超平面: $2\mathbf{x}^{(1)} + \mathbf{x}^{(2)} - 5 = 0$

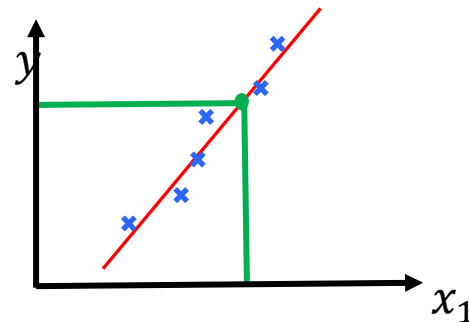


线性回归 VS 感知机

▶ 线性回归

$$L(\hat{\mathbf{w}}) = \frac{1}{2} \sum_{j=1}^m (H_{\mathbf{w}}(\mathbf{x}_j) - \mathbf{y}_j)^2 = \frac{1}{2} \sum_{j=1}^m (\hat{\mathbf{w}}^T \mathbf{x} - \mathbf{y}_j)^2$$

$$\hat{\mathbf{w}} = \hat{\mathbf{w}} - \alpha \frac{\partial L(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}}$$



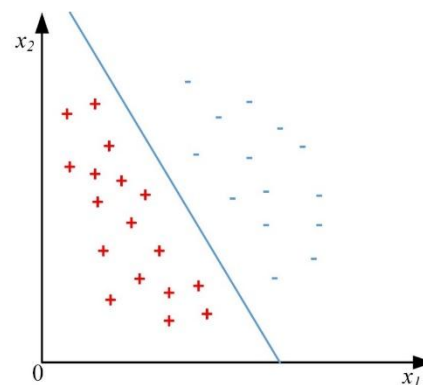
▶ 感知机

$$L(\mathbf{w}, b) = - \sum_{\mathbf{x}_j \in M} y_j (\mathbf{w}^T \mathbf{x}_j + b)$$

$$\nabla_{\mathbf{w}} L(\mathbf{w}, b) = - \sum_{\mathbf{x}_j \in M} y_j \mathbf{x}_j \Rightarrow \mathbf{w} \leftarrow \mathbf{w} + \alpha y_j \mathbf{x}_j$$

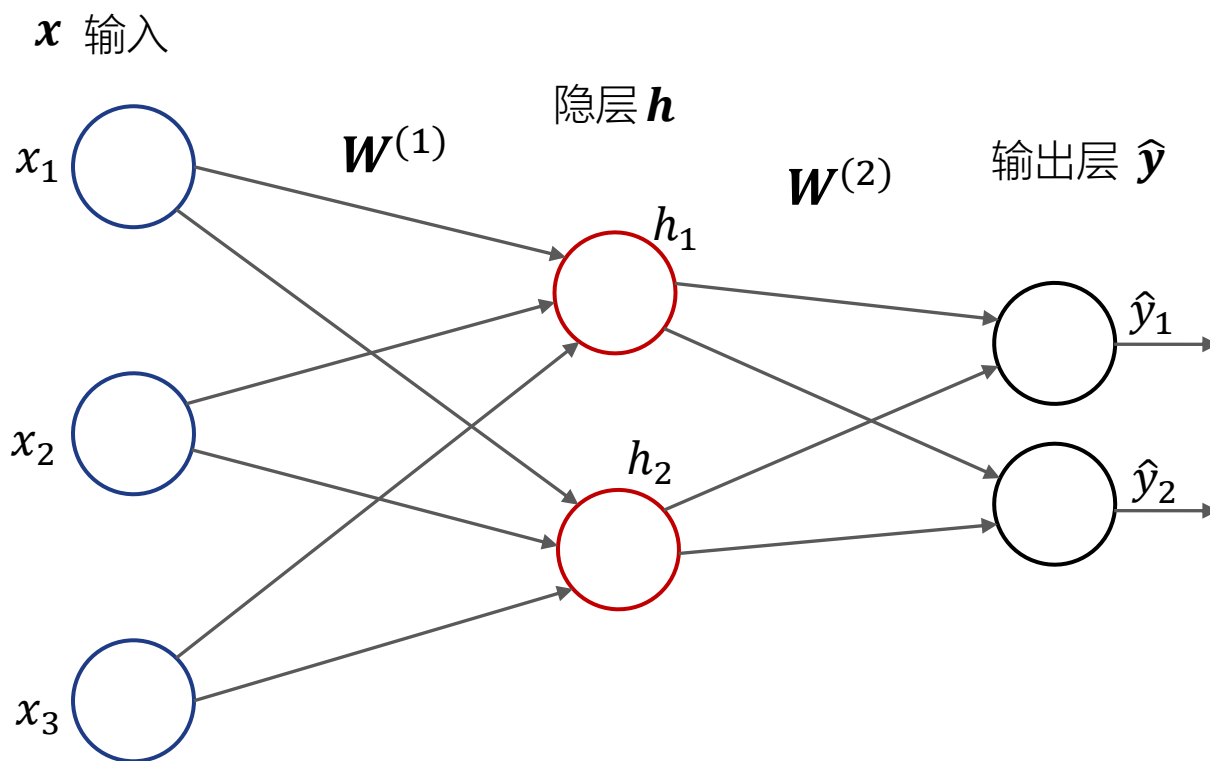
$$\nabla_b L(\mathbf{w}, b) = - \sum_{\mathbf{x}_j \in M} y_j \Rightarrow b \leftarrow b + \alpha y_j$$

感知机算法



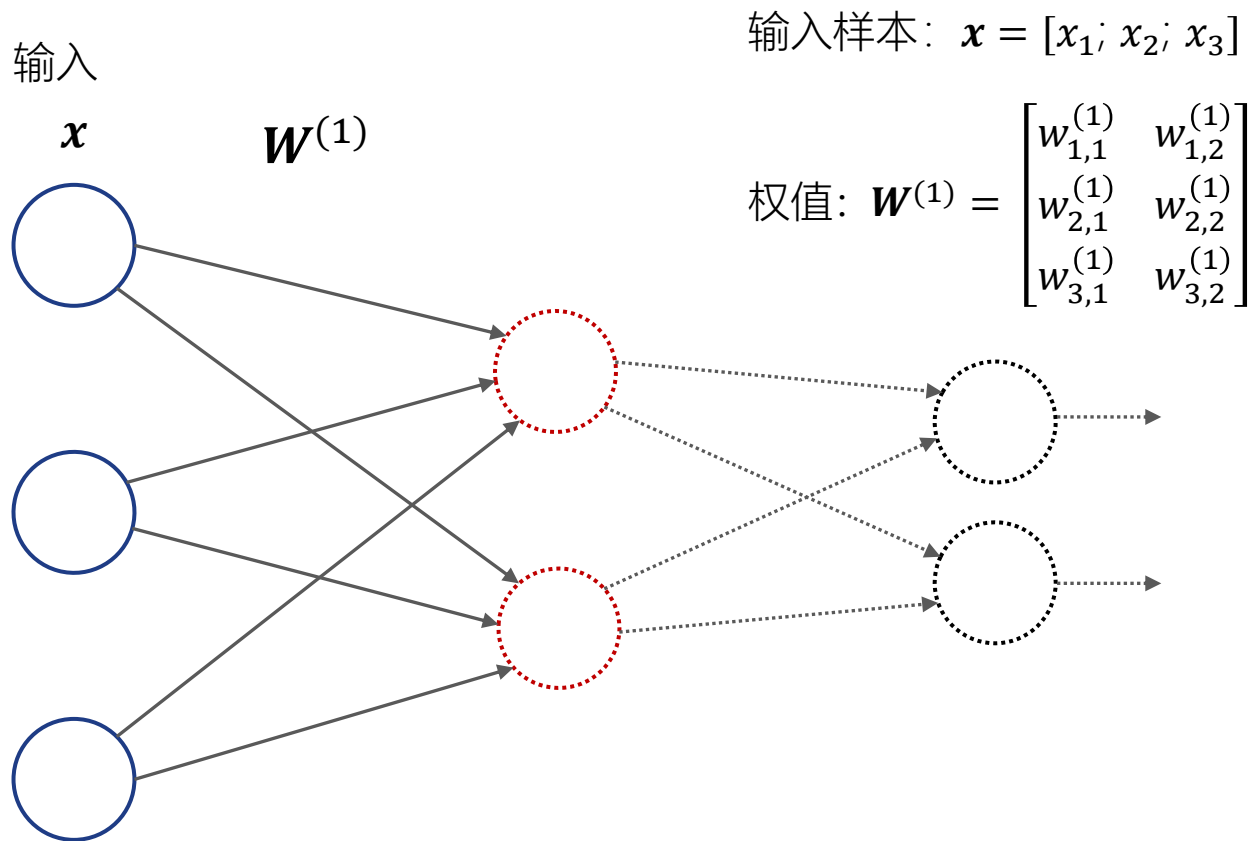
两层神经网络-多层感知机

- 将大量的神经元模型进行组合，用不同的方法进行连接并作用在不同的激活函数上，就构成了人工神经网络模型。
- 多层感知机一般指全连接的两层神经网络模型



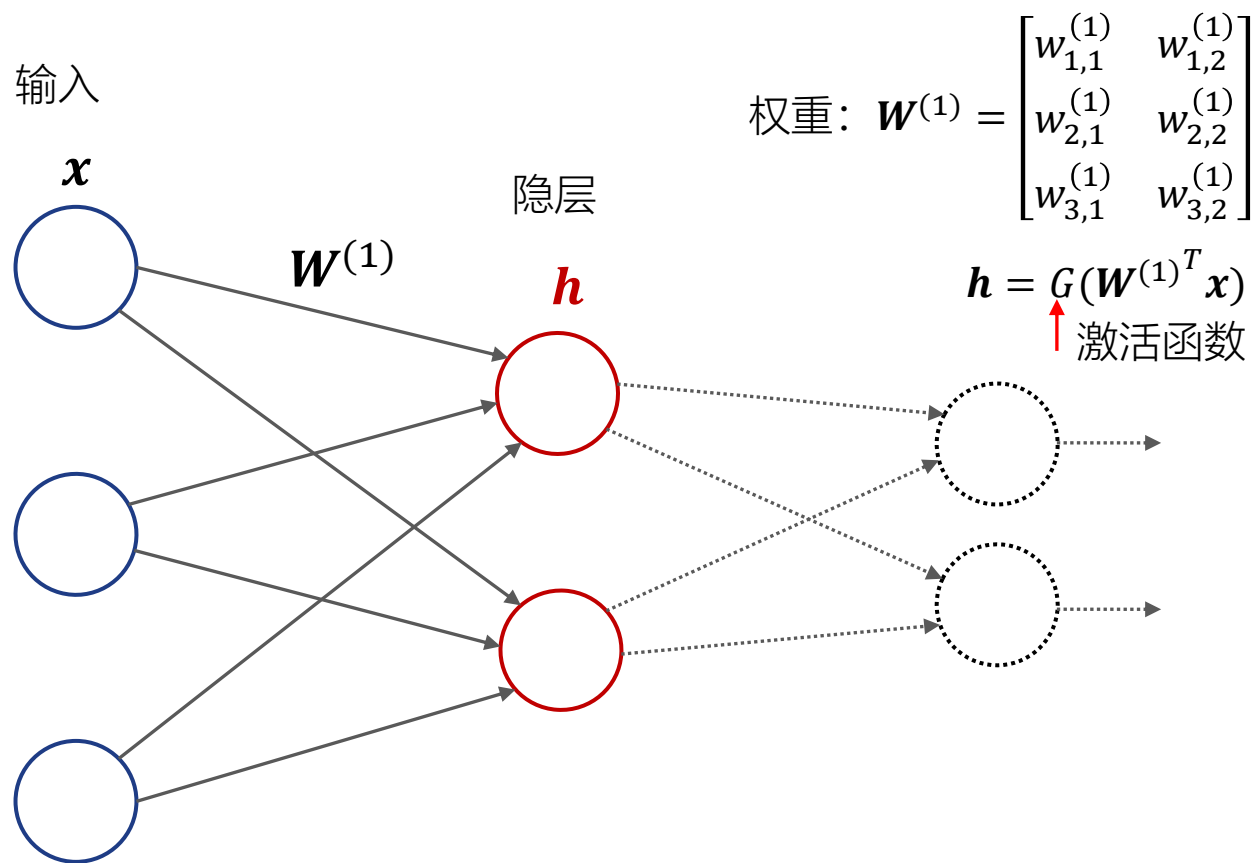
首先输入是 $\mathbf{x}$ 的一组向量, x_1 、 x_2 、 x_3 , 然后这组向量跟这个 $\mathbf{W}^{(1)}$ 权重矩阵相乘

w 两个下标, 第一个对应输入序号, 第二个对应输出序号



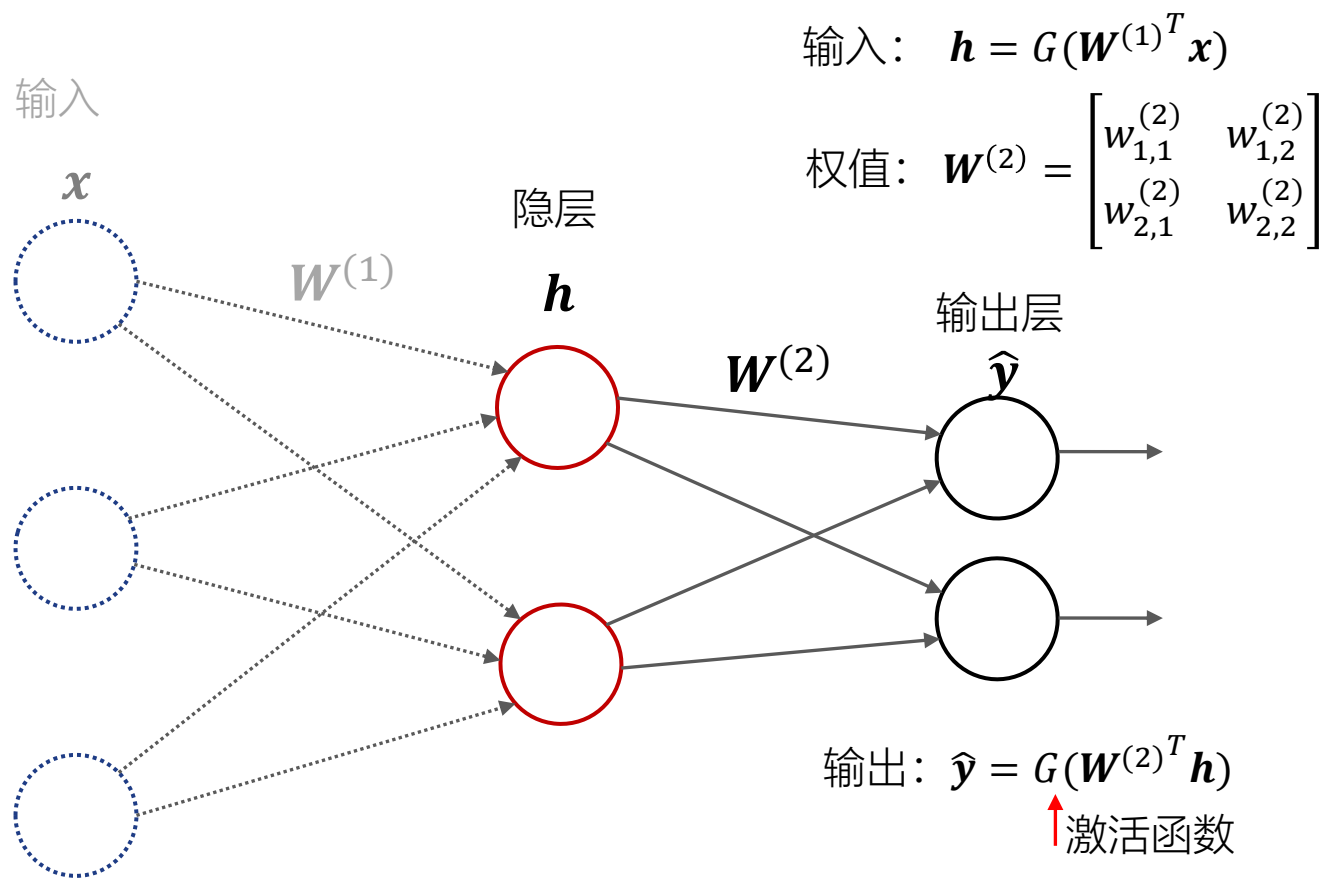
做完矩阵乘法之后通过一个激活函数

输入： $\mathbf{x} = [x_1; x_2; x_3]$



激活函数是用来加入非线性因素的，因为线性模型的表达能力不够。加上非线性激活函数，神经网络的表达能力更加强大。

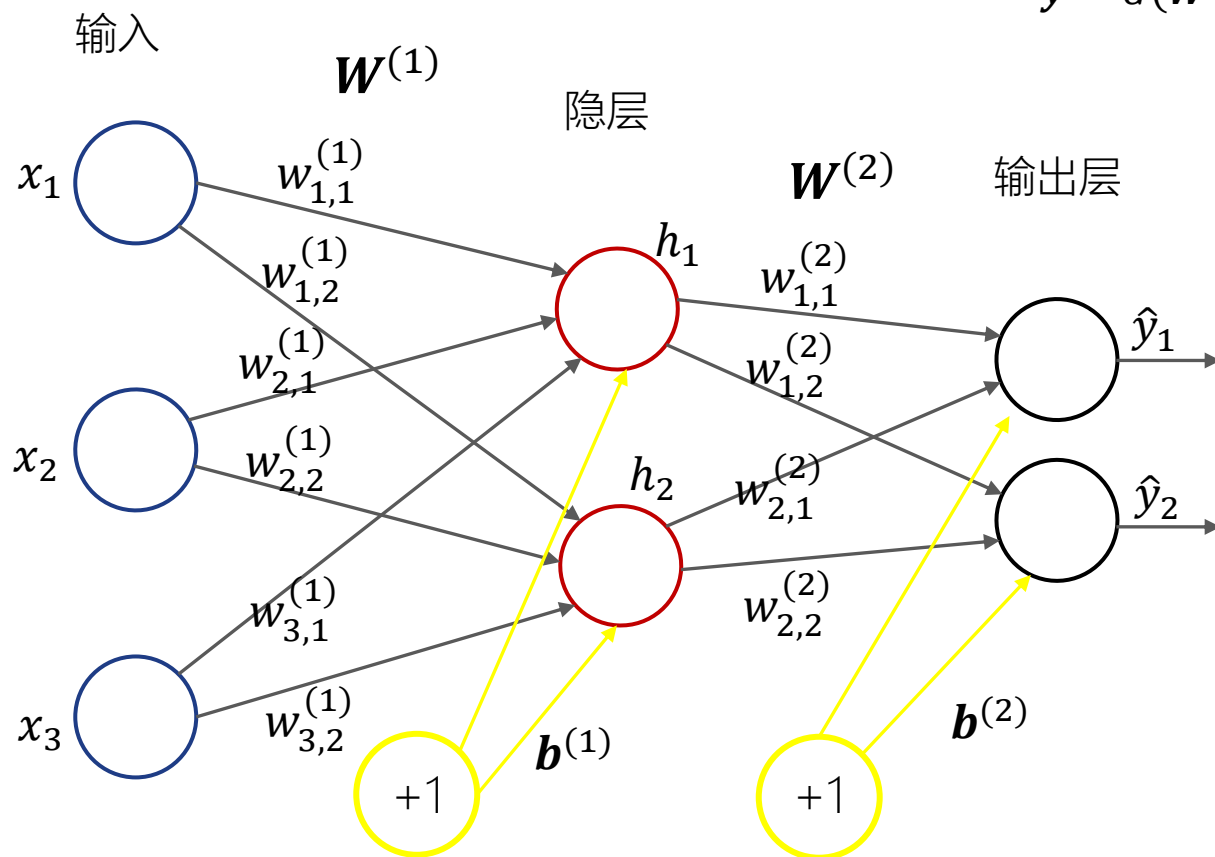
类似的， h 继续跟 $W^{(2)}$ 矩阵去做乘法，做完之后再通过一个激活函数，就可以得到整个神经网络的输出 $\hat{y}$



偏置节点

$$\mathbf{h} = G(\mathbf{W}^{(1)T} \mathbf{x} + \mathbf{b}^{(1)})$$

$$\hat{\mathbf{y}} = G(\mathbf{W}^{(2)T} \mathbf{h} + \mathbf{b}^{(2)})$$



在神经网络中，除了输出层以外，都会有一个偏置单元 $\mathbf{b}$ ，与后一层的所有节点相连接。

$\mathbf{W}$ 称之为权重， $\mathbf{b}$ 为偏置， $(\mathbf{W}, \mathbf{b})$ 合称为神经网络的参数。本例子，一共14个参数。6+2+4+2=14

浅层神经网络特点

需要数据量小、训练快，其局限性在于对复杂函数的表示能力有限，针对复杂分类问题其泛化能力受到制约

- ▶ 例如，用多层感知机识别上千类物体，不现实

Why Not Go Deeper?

- ▶ 主观上：Kurt Hornik证明了理论上两层神经网络足以拟合任意函数，
- ▶ 客观上：过去也没有足够的数据和计算能力

多层神经网络（深度学习）

- ▶ 2006年，Hinton在Science发表了论文（Reducing the dimensionality of data with neural networks. Science, Vol. 313. no. 5786），给多层神经网络相关的学习方法赋予了一个新名词--“深度学习”
- ▶ Hinton、LeCun以及Bengio三人被称为深度学习三位开创者，同获2018年图灵奖。把整个神经网络重新推向了研究的高潮。
- ▶ Hinton获2024年诺贝尔物理学奖



多层神经网络（深度学习）

2006年，Hinton在Science发表了论文（Reducing the dimensionality of data with neural networks. Science, Vol. 313. no. 5786），给多层神经网络相关的学习方法赋予了一个新名词--“深度学习”。他和LeCun以及Bengio三人被称为深度学习三位开创者，他们三个人一起获得了**2018**年的图灵奖。把整个神经网络重新推向了研究的高潮。



Geoffery Hinton

两个观点：

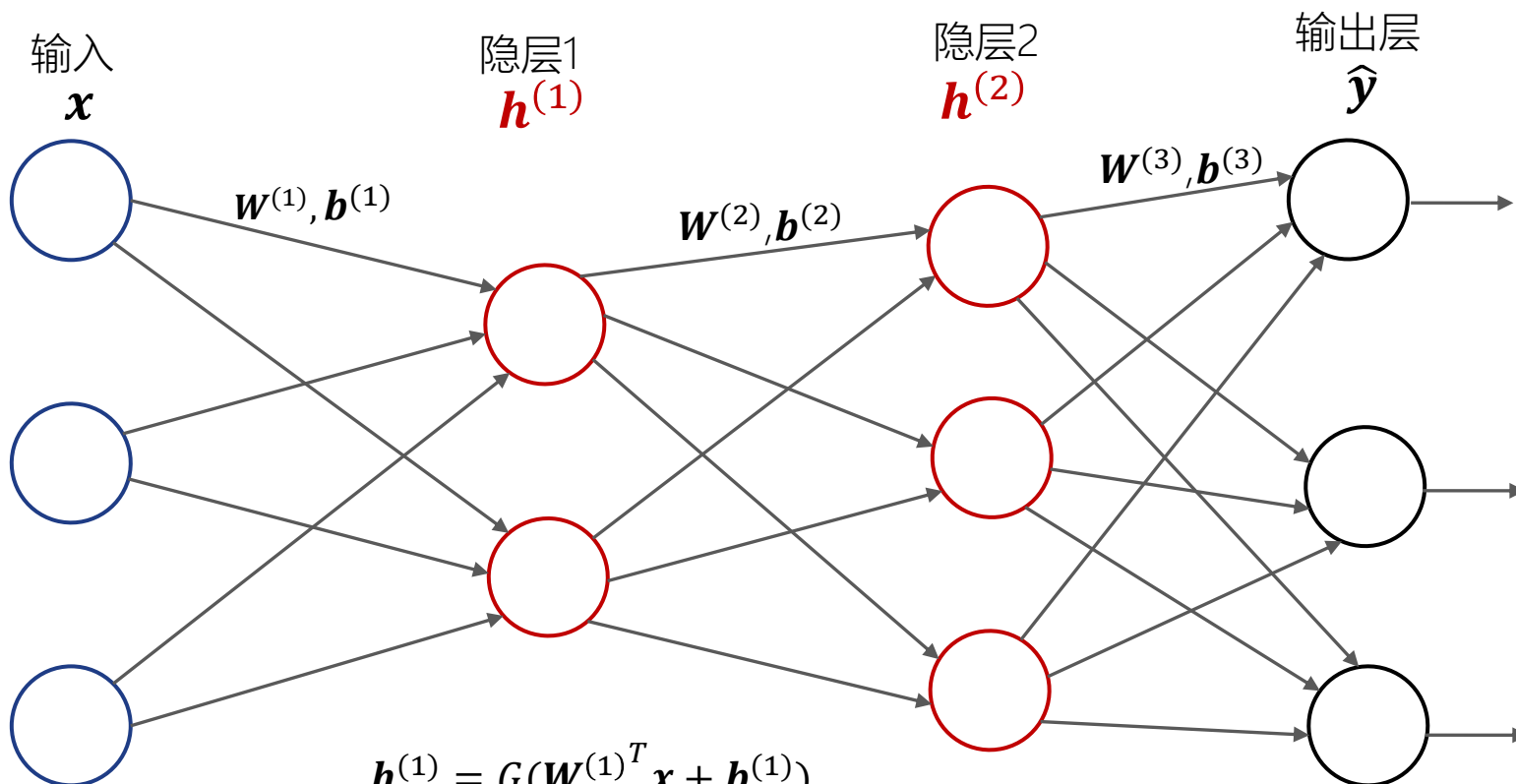
- （1）多层人工神经网络模型有很强的特征学习能力，深度学习模型学习得到的特征数据对原始数据有更本质的代表性，这将大大便于分类和可视化问题；
- （2）对于过去深度神经网络很难训练达到最优的问题，可以采用逐层训练方法解决。将上层训练好的结果作为下层训练过程中的初始化参数。

深度神经网络的成功：A+B+C

深度神经网络不断发展不仅依赖于自身的结构优势，也依赖于如下一些外在因素

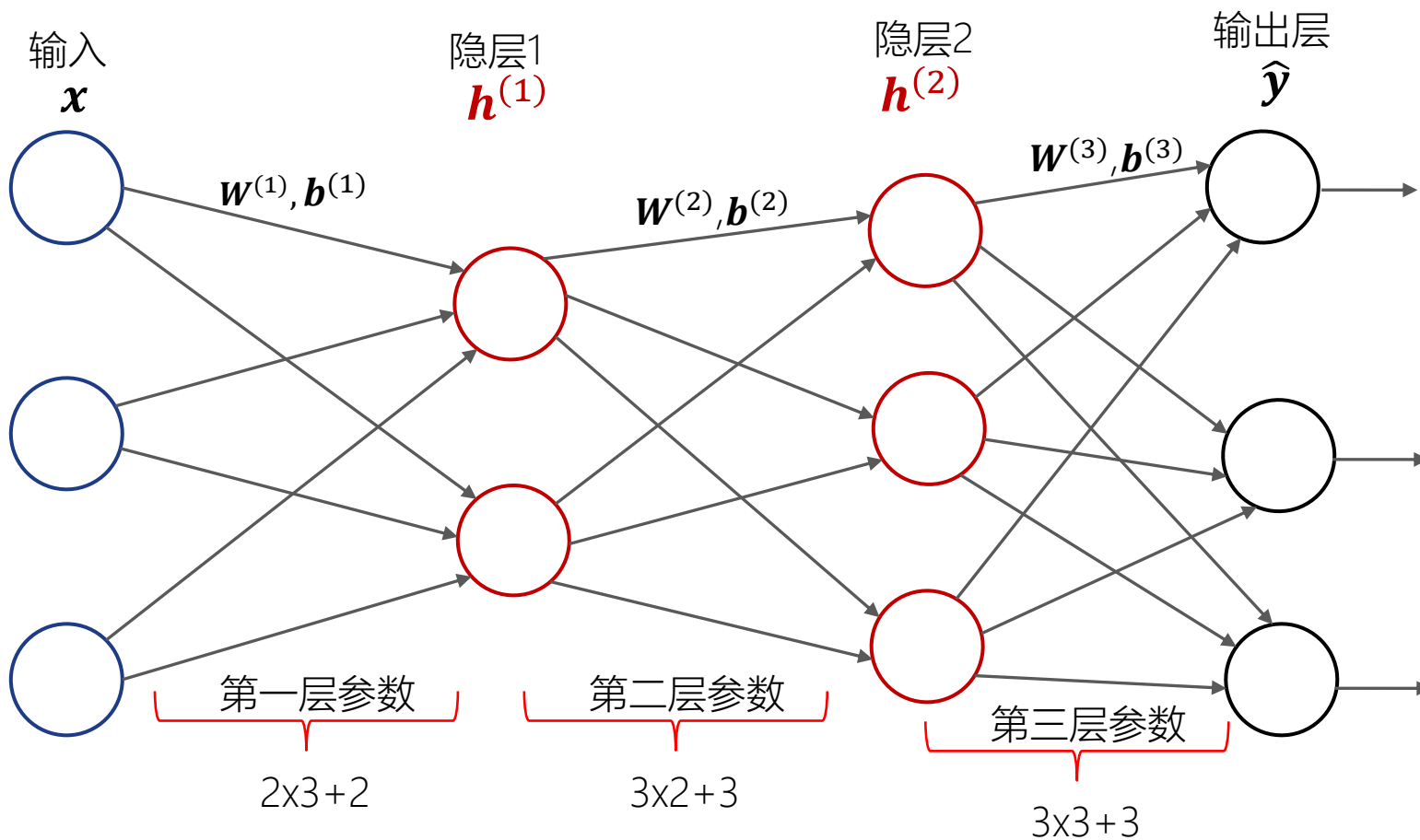
- Algorithm: 算法日新月异，优化算法层出不穷（学习算法->BP 算法-> Pre-training, Dropout等方法)
- Big data: 数据量不断增大 (10-> 10 k ->100M)
- Computing: 处理器计算能力的不断提升（晶体管->CPU -> 集群/GPU -> 智能处理器)

多层神经网络（深度学习）- 层数不断增加

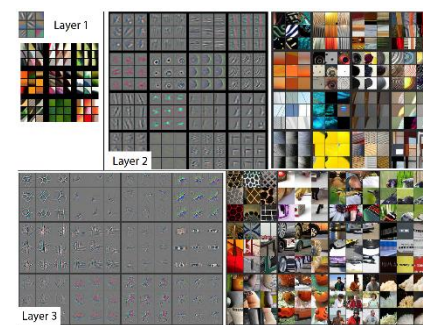


推导公式

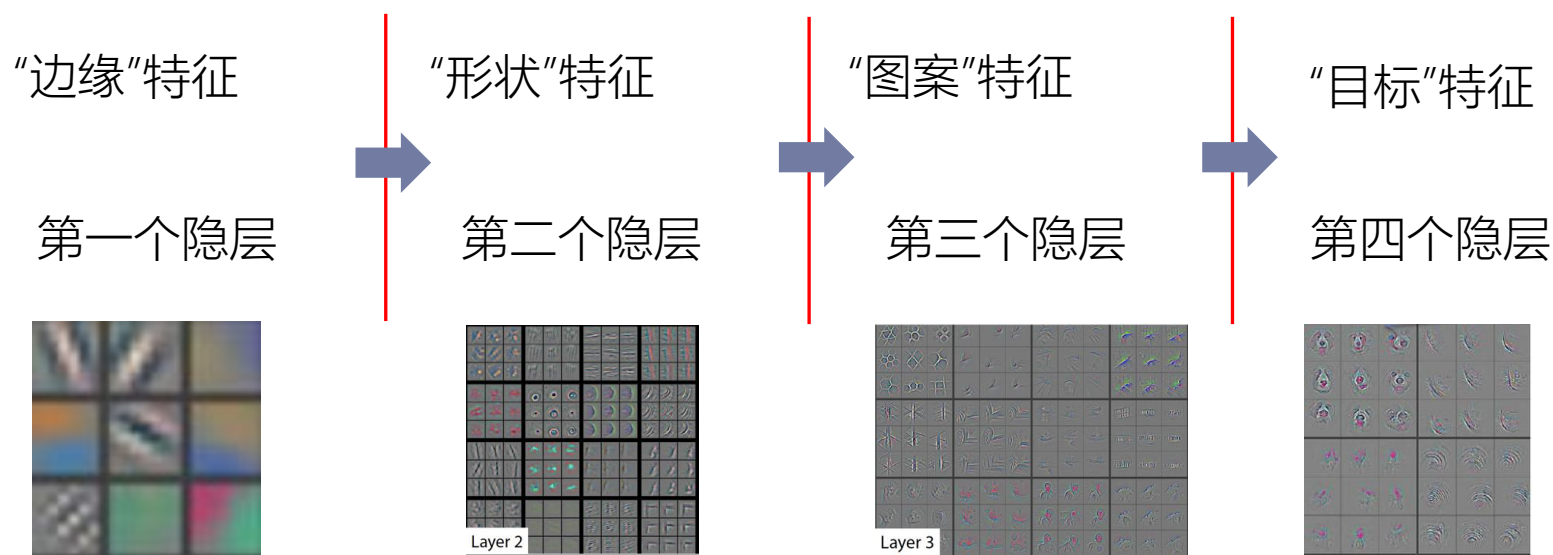
$$h^{(1)} = G(W^{(1)T}x + b^{(1)})$$
$$h^{(2)} = G(W^{(2)T}h^{(1)} + b^{(2)})$$
$$\hat{y} = G(W^{(3)T}h^{(2)} + b^{(3)})$$



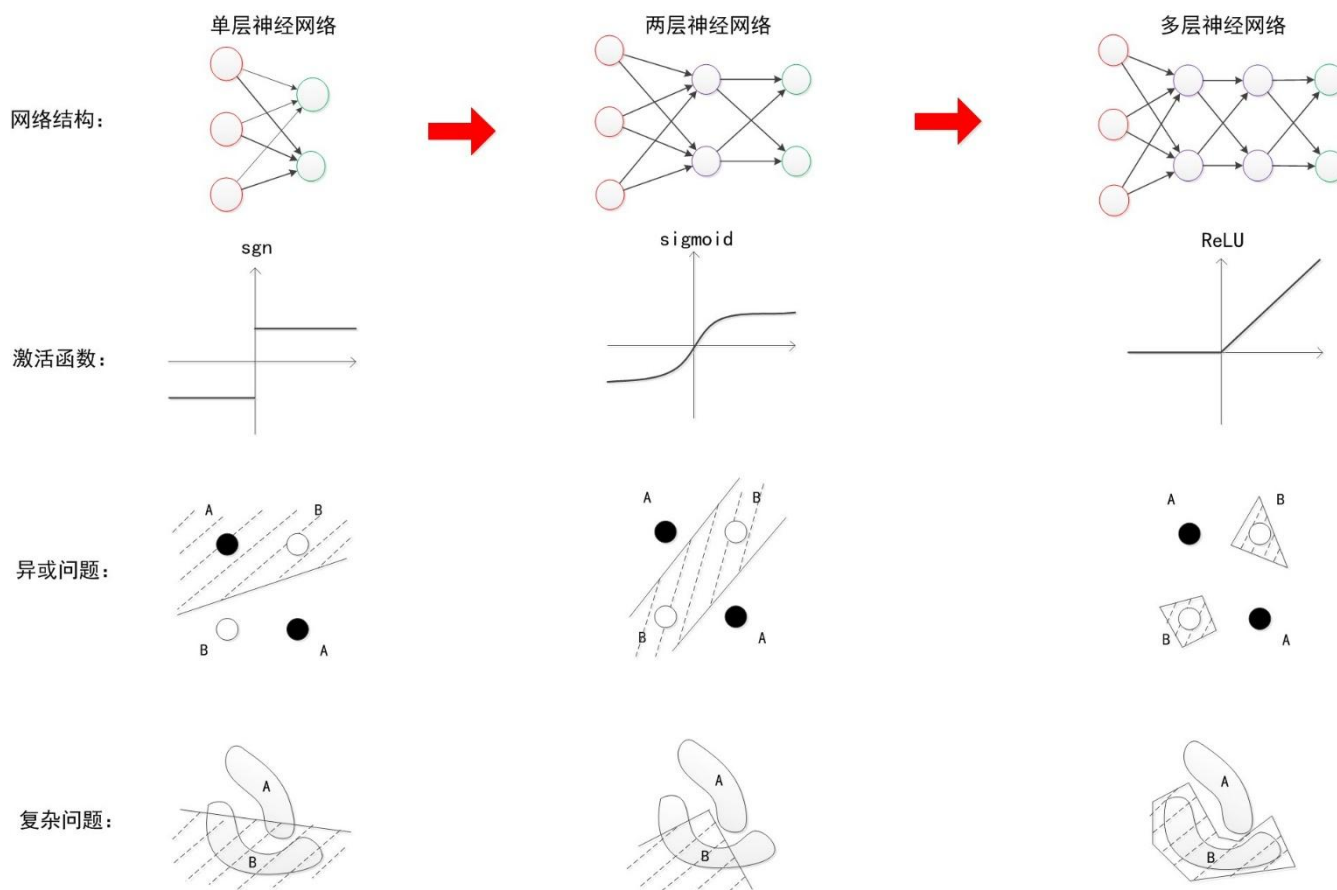
需 $6+6+9+8=29$ 个参数



- 随着网络的层数增加，每一层对于前一层次的抽象表示更深入，每一层神经元学习到的是前一层神经元更抽象的表示
- 通过抽取更抽象的特征来对事物进行区分，从而获得更好的区分与分类能力

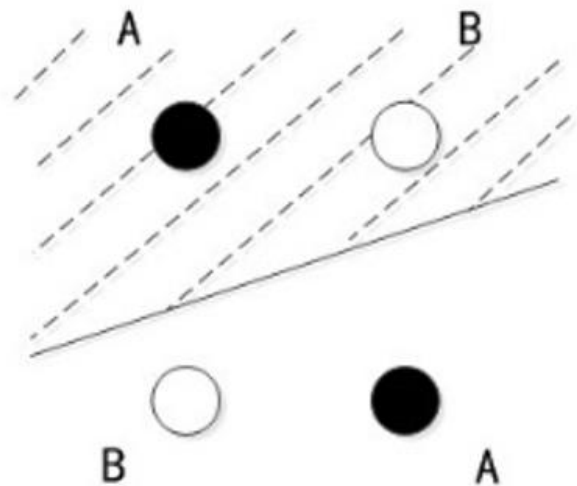


从单层神经网络，到两层神经网络，再到多层神经网络，随着网络层数的增加，以及激活函数的调整，神经网络拟合非线性问题能力不断增强。



为什么感知机无法解决异或问题

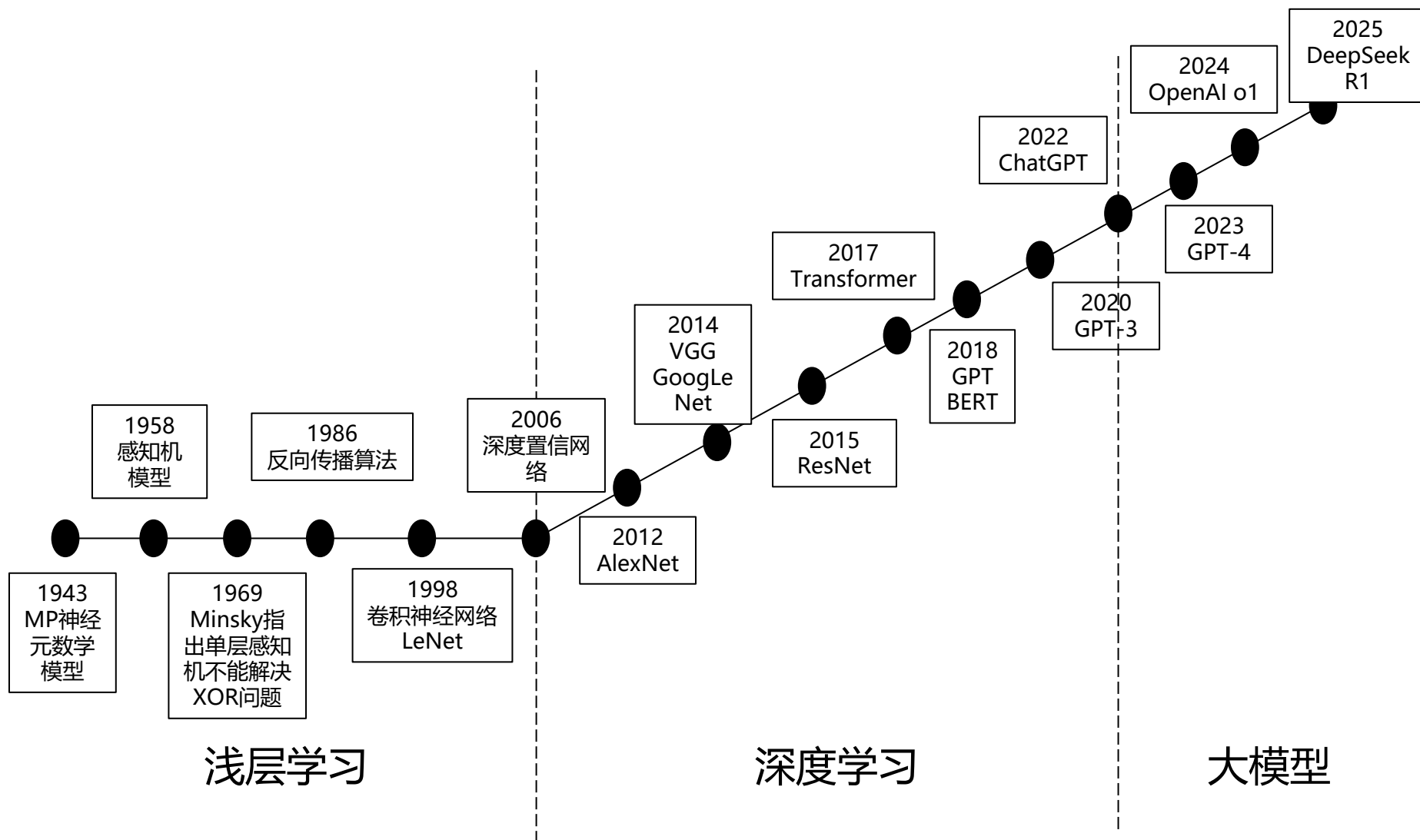
- 核心原因是 异或问题的样本在输入空间中是「线性不可分」的，而单层感知机的决策边界是线性的（只能用“直线 / 平面”划分样本）。



单层感知机的决策边界是“线性的”单层感知机的模型为： $y = \text{sign}(w_1x_1 + w_2x_2 + b)$ 。

这个模型的本质是：用一条直线 $w_1x_1 + w_2x_2 + b = 0$ 作为“决策边界”，把平面分成两部分，分别对应输出 1 和 0。

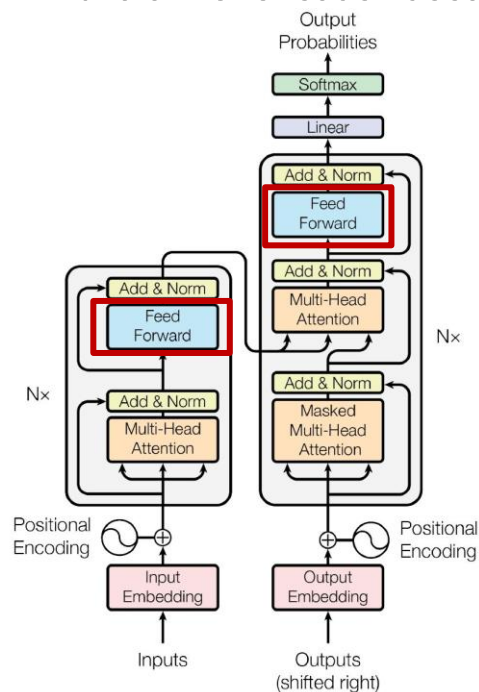
深度学习发展历程



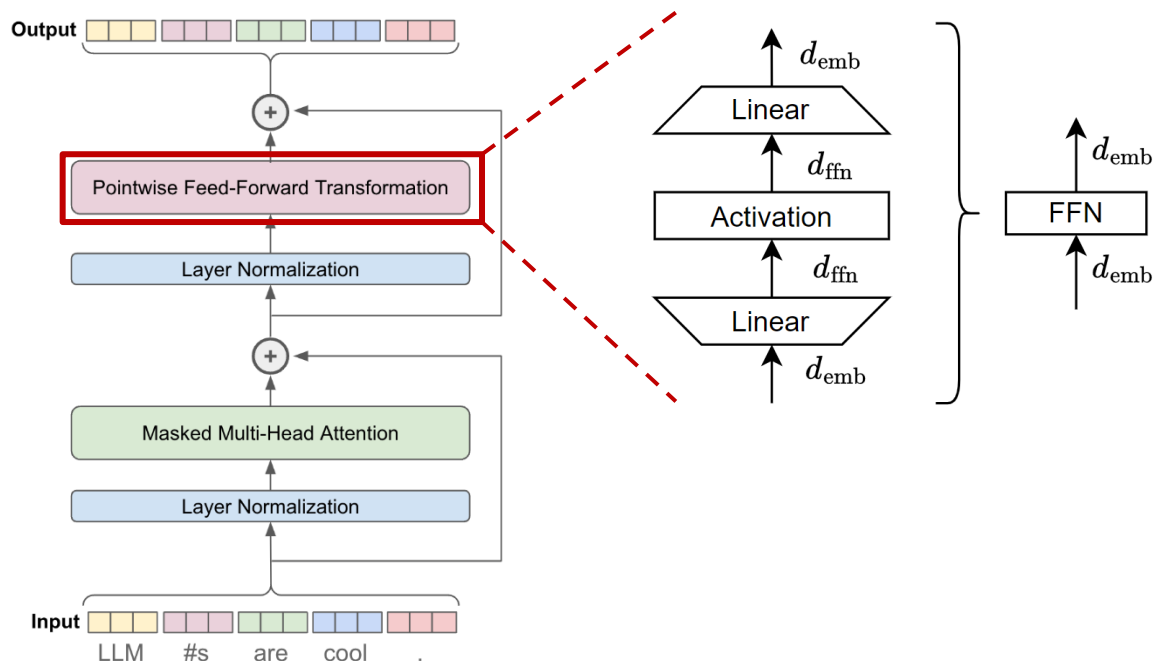
Transformer中的FFN

- Transformer中FFN为两层全连接

Transformer encoder-decoder



Transformer decoder-only



深而大的深度神经网络

时间	机构	模型名称	模型规模	数据规模	计算时间
2018.6	OpenAI	GPT	110M	4GB	3天
2018.10	Google	BERT	330M	16GB	50天
2019.2	OpenAI	GPT-2	1.5B	40GB	200天
2019.7	Facebook	RoBERTa	330M	160GB	3年
2019.10	Google	T5	11B	800GB	66年
2020.6	OpenAI	GPT-3	175B	560GB	90年

注：M-百万，B-十亿，最后一列计算时间为使用单块 NVIDIA V100 GPU训练的估计时间

深而大的深度神经网络

时间	机构	模型名称	模型规模	数据规模	训练 / 计算情况 (核心特点)
2023.03	OpenAI	GPT-4	超千亿 (未完全公开)	多模态数据 (文本 + 图像等)	依托 Azure 大规模 GPU 集群, 训练周期 数周 (分布式并行加速)
2023.03	百度	文心一言 (ERNIE Bot)	千亿级	中文网页、知识图谱等	基于百度算力集群 (昆仑芯 + GPU), 训练周期 约数月
2023.04	阿里云	通义千问	千亿级	中文为主的多模态数据	阿里云 “飞天智算” 平台 (大规模 GPU 集群), 训练周期 数周
2023.12	Google	Gemini Ultra	超千亿	多模态数据 (文本 / 图像 / 视频)	谷歌 TPU v4/v5 专用集群, 训练周期 约数月
2024.03	Anthropic	Claude 3 (Opus)	超百亿 - 千亿级	大规模文本 + 多模态数据	Anthropic 专用算力集群, 训练周期 数周 (效率优化)
2024.04	Meta	Llama 3 (70B)	700 亿	开源多语言数据	Meta GPU 集群, 训练周期 数月 (依托高效并行框架加速)
2024.10	字节跳动	豆包 (Doubao)	千亿级	中文社交、网页等数据	字节跳动算力集群, 训练周期 数周
2025.03	OpenAI	GPT-4o (多模态迭代版)	超千亿	大规模多模态数据 (文本 / 图像 / 音频)	Azure 新一代算力集群, 训练周期 2-3 周 (硬件与算法协同优化, 效率进一步提升)

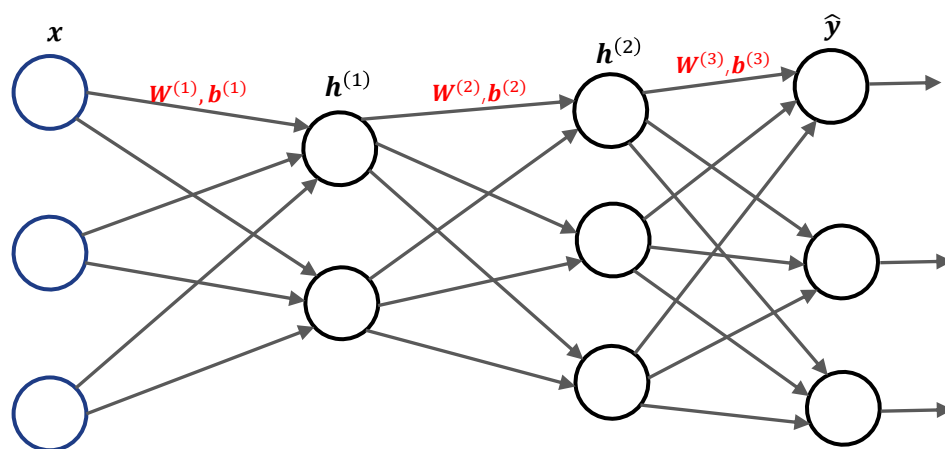
Claude 3 benchmarks

	Claude 3 Opus	Claude 3 Sonnet	Claude 3 Haiku	GPT-4	GPT-3.5	Gemini 1.0 Ultra	Gemini 1.0 Pro
Undergraduate level knowledge <i>MMLU</i>	86.8% 5 shot	79.0% 5-shot	75.2% 5-shot	86.4% 5-shot	70.0% 5-shot	83.7% 5-shot	71.8% 5-shot
Graduate level reasoning <i>GPQA, Diamond</i>	50.4% 0-shot CoT	40.4% 0-shot CoT	33.3% 0-shot CoT	35.7% 0-shot CoT	28.1% 0-shot CoT	—	—
Grade school math <i>GSM8K</i>	95.0% 0-shot CoT	92.3% 0-shot CoT	88.9% 0-shot CoT	92.0% 5-shot CoT	57.1% 5-shot	94.4% Maj1@32	86.5% Maj1@32
Math problem-solving <i>MATH</i>	60.1% 0-shot CoT	43.1% 0-shot CoT	38.9% 0-shot CoT	52.9% 4-shot	34.1% 4-shot	53.2% 4-shot	32.6% 4-shot
Multilingual math <i>MGSM</i>	90.7% 0-shot	83.5% 0-shot	75.1% 0-shot	74.5% 8-shot	—	79.0% 8-shot	63.5% 8-shot
Code <i>HumanEval</i>	84.9% 0-shot	73.0% 0-shot	75.9% 0-shot	67.0% 0-shot	48.1% 0-shot	74.4% 0-shot	67.7% 0-shot
Reasoning over text <i>DROP, F1 score</i>	83.1 3-shot	78.9 3-shot	78.4 3-shot	80.9 3-shot	64.1 3-shot	82.4 Variable shots	74.1 Variable shots
Mixed evaluations <i>BIG-Bench-Hard</i>	86.8% 3-shot CoT	82.9% 3-shot CoT	73.7% 3-shot CoT	83.1% 3-shot CoT	66.6% 3-shot CoT	83.6% 3-shot CoT	75.0% 3-shot CoT

提纲

- ▶ 从机器学习到神经网络
- ▶ 神经网络训练
- ▶ 神经网络设计原则
 - ▶ 网络的拓扑结构
 - ▶ 激活函数
 - ▶ 损失函数
- ▶ 过拟合与正则化
- ▶ 交叉验证
- ▶ 小结

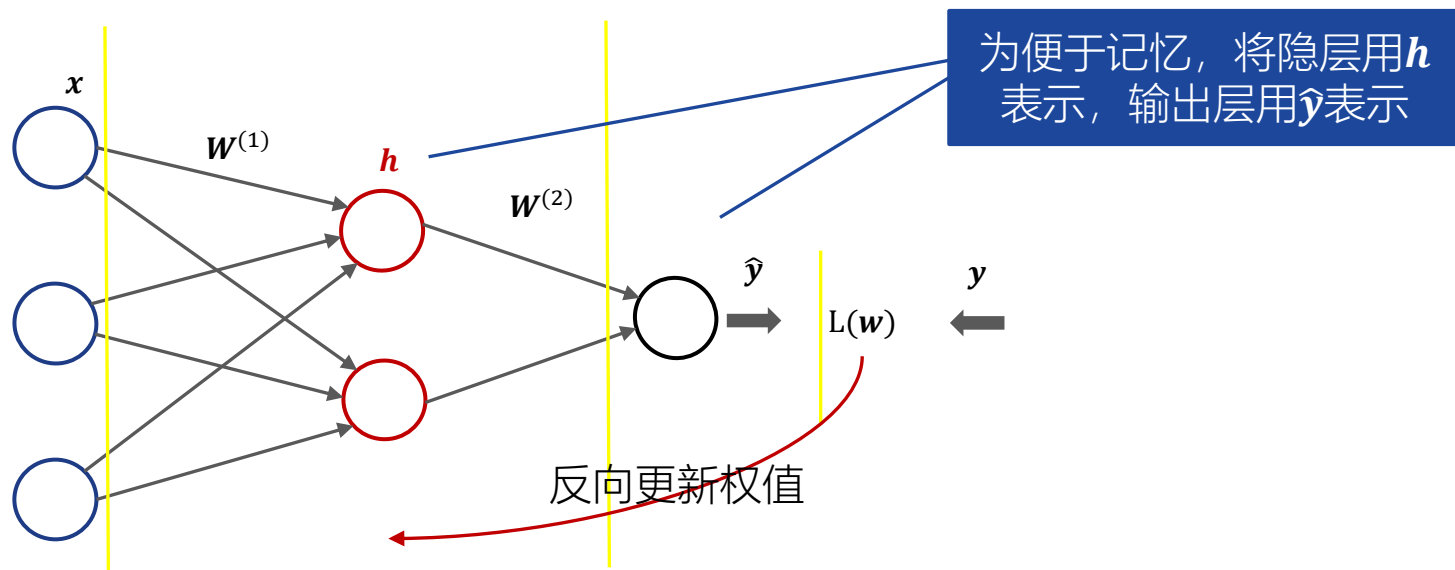
神经网络的模型训练



- 模型训练的目的，就是调整参数使得模型计算值 $\hat{y}$ 尽可能的与真实值 y 逼近

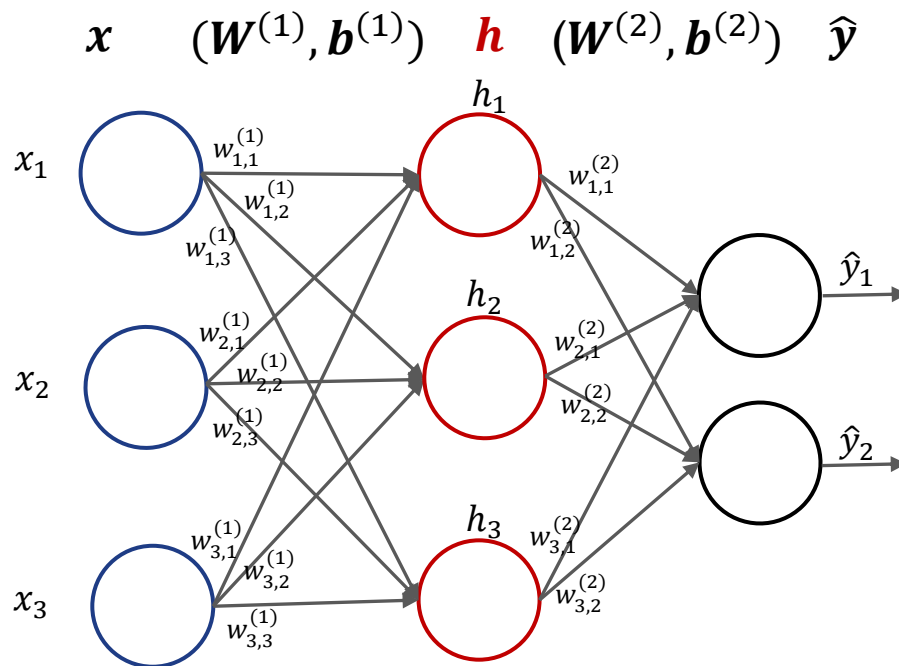
神经网络训练

- **正向传播（推断）** 是根据输入，经过权重、激活函数计算出隐层，将输入的特征向量从低级特征逐步提取为抽象特征，直到得到最终输出结果的过程。



- **反向传播（训练）** 是根据正向传播的输出结果和期望值计算出损失函数，再通过链式求导，最终从网络后端逐步修改权重使输出和期望值的差距变到最小的过程。

正向传播（网络结构介绍）



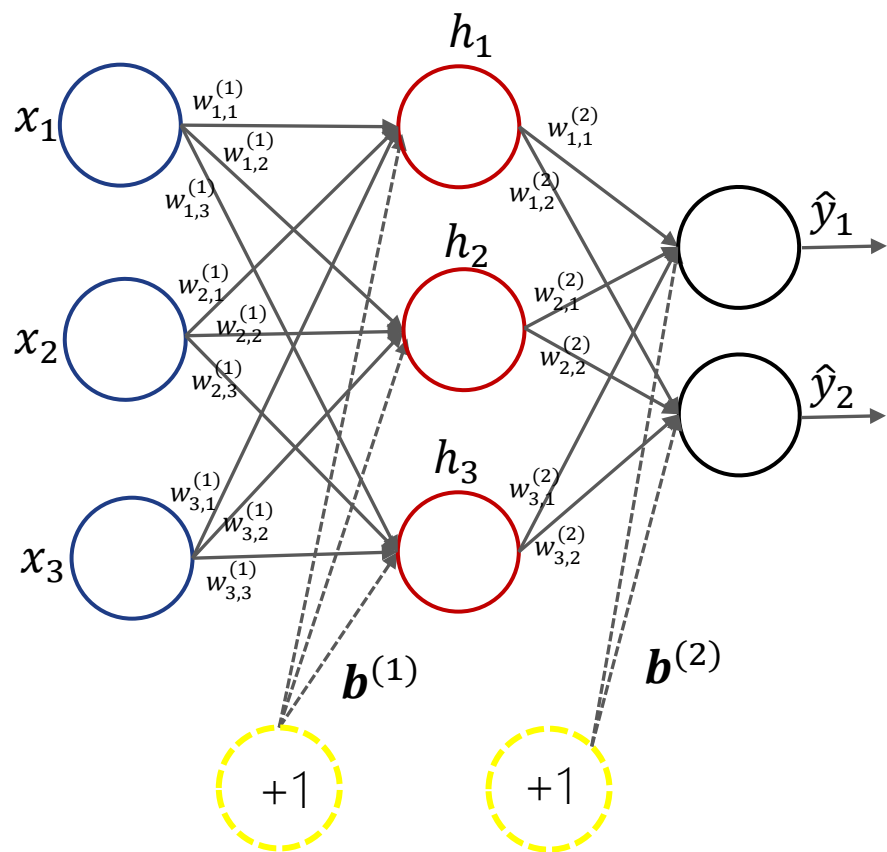
输入 $\mathbf{x} = [x_1; x_2; x_3]$

输入 $\mathbf{h} = [h_1; h_2; h_3]$

$$\text{权重 } \mathbf{W}^{(1)} = \begin{bmatrix} w_{1,1}^{(1)} & w_{1,2}^{(1)} & w_{1,3}^{(1)} \\ w_{2,1}^{(1)} & w_{2,2}^{(1)} & w_{2,3}^{(1)} \\ w_{3,1}^{(1)} & w_{3,2}^{(1)} & w_{3,3}^{(1)} \end{bmatrix}$$

$$\text{权重 } \mathbf{W}^{(2)} = \begin{bmatrix} w_{1,1}^{(2)} & w_{1,2}^{(2)} \\ w_{2,1}^{(2)} & w_{2,2}^{(2)} \\ w_{3,1}^{(2)} & w_{3,2}^{(2)} \end{bmatrix}$$

正向传播 (2.计算函数)



➤ 使用 sigmoid 函数作为激活函数

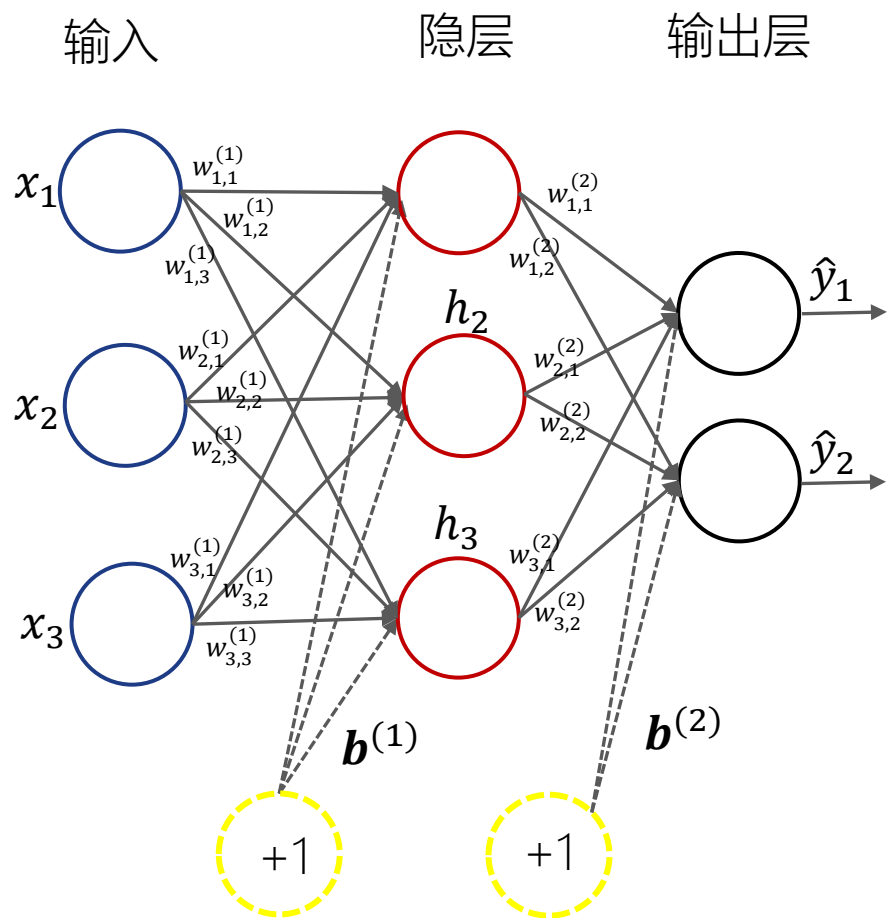
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

前向传输：输入到隐层

$$\begin{aligned} v &= \mathbf{W}^{(1)T} \mathbf{x} + \mathbf{b}^{(1)} \\ &= \begin{bmatrix} w_{1,1}^{(1)} & w_{2,1}^{(1)} & w_{3,1}^{(1)} \\ w_{1,2}^{(1)} & w_{2,2}^{(1)} & w_{3,2}^{(1)} \\ w_{1,3}^{(1)} & w_{2,3}^{(1)} & w_{3,3}^{(1)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \mathbf{b}^{(1)} \end{aligned}$$

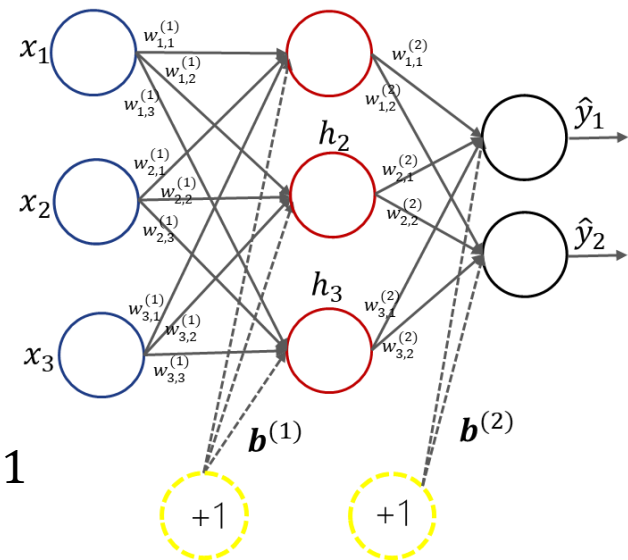
$$h = \frac{1}{1 + e^{-v}}$$

正向传播 (3.计算过程)



- 输入：包含神经元 x_1, x_2, x_3
- 隐层：包含 h_1, h_2, h_3
- 输出层：包含 $\hat{y}_1, \hat{y}_2$
- 输入和隐层之间
偏置： $b^{(1)}$
权重： $W^{(1)}$
- 隐层和输出层之间
偏置： $b^{(2)}$
权重： $W^{(2)}$

示例



假定输入数据 $x_1 = 0.02$ 、 $x_2 = 0.04$ 、 $x_3 = 0.01$

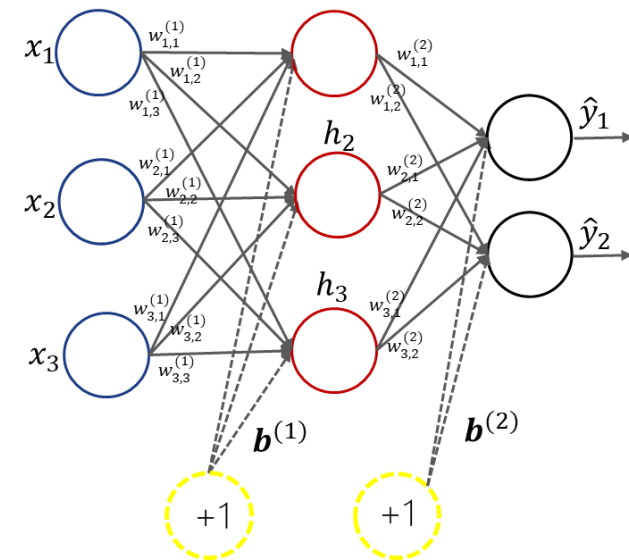
固定偏置 $\mathbf{b}^{(1)} = [0.4; 0.4; 0.4]$ 、 $\mathbf{b}^{(2)} = [0.7; 0.7]$

期望输出 $y_1 = 0.9$ 、 $y_2 = 0.5$

未知权重

$$\mathbf{W}^{(1)} = \begin{bmatrix} w_{1,1}^{(1)} & w_{1,2}^{(1)} & w_{1,3}^{(1)} \\ w_{2,1}^{(1)} & w_{2,2}^{(1)} & w_{2,3}^{(1)} \\ w_{3,1}^{(1)} & w_{3,2}^{(1)} & w_{3,3}^{(1)} \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} w_{1,1}^{(2)} & w_{1,2}^{(2)} \\ w_{2,1}^{(2)} & w_{2,2}^{(2)} \\ w_{3,1}^{(2)} & w_{3,2}^{(2)} \end{bmatrix}$$

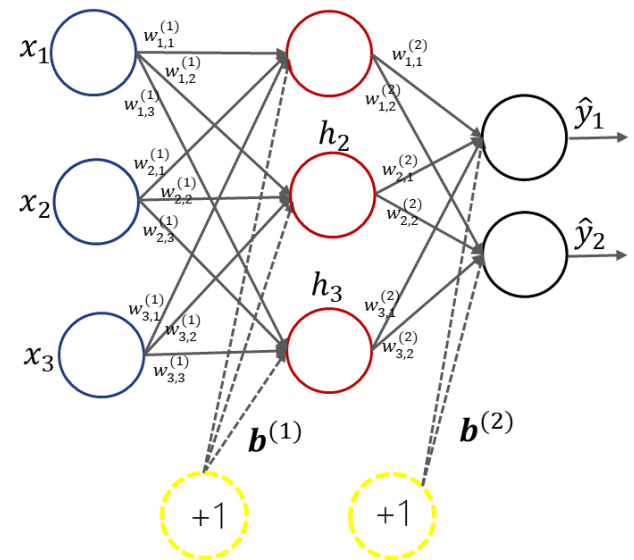
目的是为了能得到 $y_1 = 0.9$ 、 $y_2 = 0.5$ 的期望的值，需计算出合适的 $\mathbf{W}^{(1)}$ 、 $\mathbf{W}^{(2)}$ 的权重值



➤ 初始化权重值

$$\mathbf{W}^{(1)} = \begin{bmatrix} w_{1,1}^{(1)} & w_{1,2}^{(1)} & w_{1,3}^{(1)} \\ w_{2,1}^{(1)} & w_{2,2}^{(1)} & w_{2,3}^{(1)} \\ w_{3,1}^{(1)} & w_{3,2}^{(1)} & w_{3,3}^{(1)} \end{bmatrix} = \begin{bmatrix} 0.25 & 0.15 & 0.30 \\ 0.25 & 0.20 & 0.35 \\ 0.10 & 0.25 & 0.15 \end{bmatrix}$$

$$\mathbf{W}^{(2)} = \begin{bmatrix} w_{1,1}^{(2)} & w_{1,2}^{(2)} \\ w_{2,1}^{(2)} & w_{2,2}^{(2)} \\ w_{3,1}^{(2)} & w_{3,2}^{(2)} \end{bmatrix} = \begin{bmatrix} 0.40 & 0.25 \\ 0.35 & 0.30 \\ 0.01 & 0.35 \end{bmatrix}$$



➤ 输入到隐层计算

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \mathbf{W}^{(1)T} \mathbf{x} + \mathbf{b}^{(1)} = \begin{bmatrix} 0.25 & 0.25 & 0.10 \\ 0.15 & 0.20 & 0.25 \\ 0.30 & 0.35 & 0.15 \end{bmatrix} \begin{bmatrix} 0.02 \\ 0.04 \\ 0.01 \end{bmatrix} + \begin{bmatrix} 0.4 \\ 0.4 \\ 0.4 \end{bmatrix} = \begin{bmatrix} 0.416 \\ 0.4135 \\ 0.4215 \end{bmatrix}$$

$$\mathbf{h} = \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = \frac{1}{1 + e^{-v}} = \begin{bmatrix} \frac{1}{1 + e^{-0.416}} \\ \frac{1}{1 + e^{-0.4135}} \\ \frac{1}{1 + e^{-0.4215}} \end{bmatrix} = \begin{bmatrix} 0.6025 \\ 0.6019 \\ 0.6038 \end{bmatrix}$$

➤ 隐层到输出层计算

$$\mathbf{z} = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \mathbf{W}^{(2)T} \mathbf{h} + \mathbf{b}^{(2)} = \begin{bmatrix} 0.40 & 0.35 & 0.01 \\ 0.25 & 0.30 & 0.35 \end{bmatrix} \begin{bmatrix} 0.6025 \\ 0.6019 \\ 0.6038 \end{bmatrix} + \begin{bmatrix} 0.7 \\ 0.7 \end{bmatrix} = \begin{bmatrix} 1.1577 \\ 1.2425 \end{bmatrix}$$

$$\hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \end{bmatrix} = \frac{1}{1 + e^{-z}} = \begin{bmatrix} \frac{1}{1 + e^{-1.1577}} \\ \frac{1}{1 + e^{-1.2425}} \end{bmatrix} = \begin{bmatrix} 0.7609 \\ 0.7760 \end{bmatrix}$$

↑
距离期望输出 $y_1 = 0.9$ 、
 $y_2 = 0.5$ 还有差距，通过反向传播修改权重

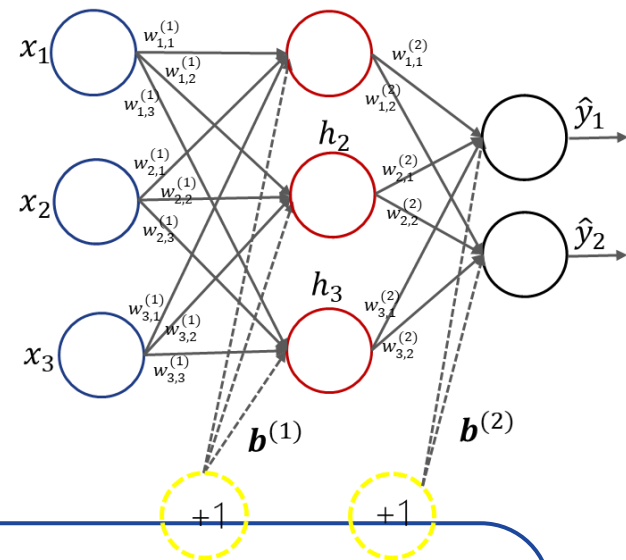
模型计算输出

$$\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \end{bmatrix} = \begin{bmatrix} 0.7609 \\ 0.7760 \end{bmatrix}$$

期望输出

$$y_1 = 0.9$$

$$y_2 = 0.5$$



➤ 计算误差

$$\begin{aligned} L(W) &= L_1 + L_2 = \frac{1}{2} (y_1 - \hat{y}_1)^2 + \frac{1}{2} (y_2 - \hat{y}_2)^2 \\ &= \frac{1}{2} (0.7609 - 0.9)^2 + \frac{1}{2} (0.7760 - 0.5)^2 = 0.0478 \end{aligned}$$

计算值与真实值之间还有很大的差距，如何缩小计算值与真实值之间的误差？

通过反向传播进行反馈，调节权重值

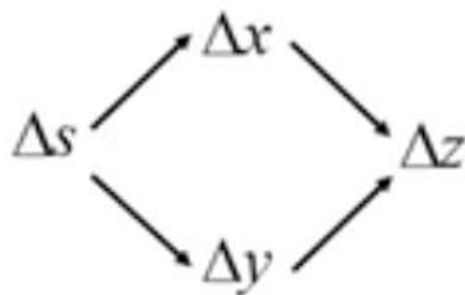
链式法则

Case 1 $y = g(x) \quad z = h(y)$

$$\Delta x \rightarrow \Delta y \rightarrow \Delta z \qquad \frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$

Case 2

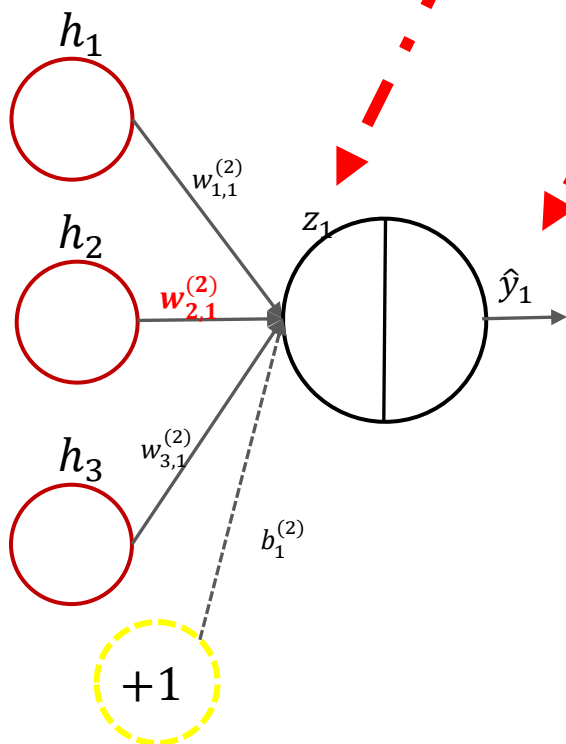
$$x = g(s) \quad y = h(s) \quad z = k(x, y)$$



$$\frac{dz}{ds} = \frac{\partial z}{\partial x} \frac{dx}{ds} + \frac{\partial z}{\partial y} \frac{dy}{ds}$$

反向传播

- 隐层到输出层的权值 $\mathbf{W}^{(2)}$ 的更新



$$L(\mathbf{W}) = L_1 + L_2 = \frac{1}{2} (y_1 - \hat{y}_1)^2 + \frac{1}{2} (y_2 - \hat{y}_2)^2$$

- 以 $w_{2,1}^{(2)}$ (记为 ω) 参数为例子, 计算 ω 对整体误差的影响有多大, 可以使用整体误差对 ω 参数求偏导

➤ 根据偏导数的链式法则推导

$$\frac{\partial L(\mathbf{W})}{\partial \omega} = \frac{\partial L(\mathbf{W})}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial z_1} \frac{\partial z_1}{\partial \omega}$$

$$L(\mathbf{W}) = \frac{1}{2} (y_1 - \hat{y}_1)^2 + \frac{1}{2} (y_2 - \hat{y}_2)^2$$

$$\frac{\partial L(\mathbf{W})}{\partial \hat{y}_1} = -(y_1 - \hat{y}_1) = -(0.9 - 0.7609) = -0.1391$$

$$\hat{y}_1 = \frac{1}{1 + e^{-z_1}}$$

$$\frac{\partial \hat{y}_1}{\partial z_1} = \hat{y}_1(1 - \hat{y}_1) = 0.7609 * (1 - 0.7609) = 0.1819$$

➤ 根据偏导数的链式法则推导

$$\frac{\partial L(W)}{\partial \omega} = \frac{\partial L(W)}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial z_1} \frac{\partial z_1}{\partial \omega}$$

$$z_1 = w_{1,1}^{(2)} \times h_1 + \omega \times h_2 + w_{3,1}^{(2)} \times h_3 + b_1^{(2)}$$

$$\frac{\partial z_1}{\partial \omega} = h_2 = 0.6019$$



$$\frac{\partial L(W)}{\partial \omega} = \frac{\partial L(W)}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial z_1} \frac{\partial z_1}{\partial \omega}$$

$$= -(y_1 - \hat{y}_1) \times \hat{y}_1 (1 - \hat{y}_1) \times h_2$$

$$= -0.1391 \times 0.1819 \times 0.6019 = -0.0152$$

➤ 更新 $w_{2,1}^{(2)}$ (记为 ω) 的权重

$$\mathbf{W}^{(2)} = \begin{bmatrix} w_{1,1}^{(2)} & w_{1,2}^{(2)} \\ \mathbf{w_{2,1}^{(2)}} & w_{2,2}^{(2)} \\ w_{3,1}^{(2)} & w_{3,2}^{(2)} \end{bmatrix} = \begin{bmatrix} 0.40 & 0.25 \\ \mathbf{0.35} & 0.30 \\ 0.01 & 0.35 \end{bmatrix}$$

初始值

$$\frac{\partial L(\mathbf{W})}{\partial \omega} = -0.0152$$

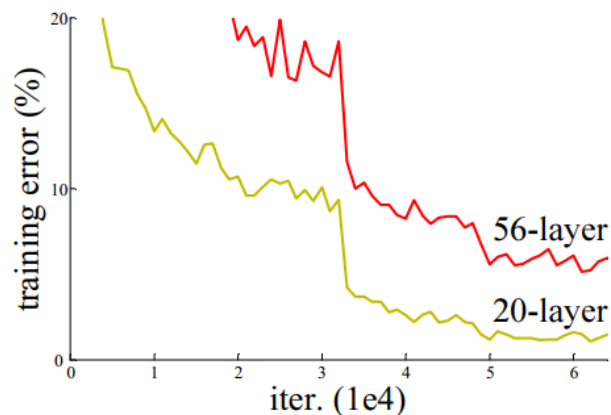
$$\omega = \omega - \alpha \times \frac{\partial L(\mathbf{W})}{\partial \omega} = 0.35 - (-0.0152) = 0.3652$$

➤ 同理，可以计算新的 $\mathbf{W}^{(2)}$ 的其他元素的权重值

- 反向传播的作用是将神经网络的输出误差反向传播到神经网络的输入端，并以此来更新神经网络中各个连接的权重。
- 当第一次反向传播法完成后，网络的模型参数得到更新，网络进行下一轮的正向传播过程，如此反复的迭代进行训练，从而不断缩小计算值与真实值之间的误差。

训练相关概念

- ▶ Batch (批)
 - ▶ 批梯度下降算法中，用于一次参数更新的训练数据量
- ▶ Iteration (迭代)
 - ▶ 使用一个或一个batch的数据对模型进行一次参数更新的过程
- ▶ Epoch (轮次)
 - ▶ 整个训练数据集 (N个样本) 完整地通过一次训练
 - ▶ 每个epoch更新 $N/\text{Batch_size}$ 次参数



提纲

- ▶ 从机器学习到神经网络
- ▶ 神经网络训练
- ▶ 神经网络设计原则
 - ▶ 网络的拓扑结构
 - ▶ 激活函数
 - ▶ 损失函数
- ▶ 过拟合与正则化
- ▶ 交叉验证
- ▶ 小结

神经网络的模型训练

训练完了结果就是不准，怎么办？

- 调整网络拓扑结构
- 选择合适的激活函数
- 选择合适的损失函数

神经网络的拓扑调节

神经网络的结构一般为：输入×隐层×输出层

输入：神经元个数=特征维度

输出层：神经元个数=分类类别数

隐层：

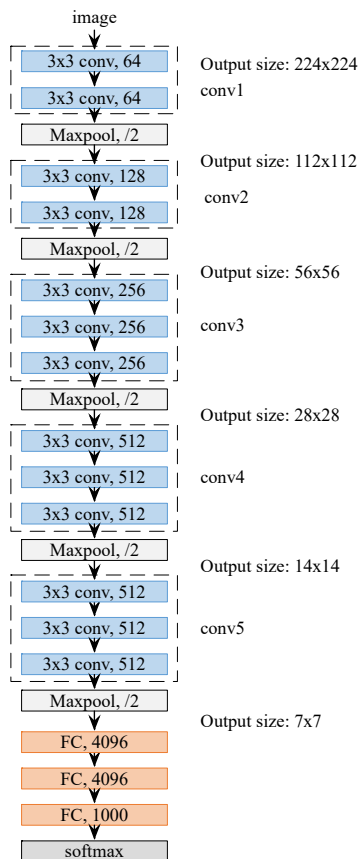
- 隐层的数量？
- 隐层神经元的个数？

给定训练样本后，输入和输出层节点数便已确定

神经网络的拓扑调节

- 隐层的设计：
 - 隐层节点的作用是提取输入特征中的隐藏规律，每个节点都赋予一定权重
 - **隐层节点数太少**，则网络从样本中获取信息的能力就越差，无法反映数据集的规律；**隐层节点数太多**，则网络的拟合能力过强，可能拟合数据集中的噪声部分，导致模型泛化能力变差。

神经网络的拓扑调节-VGG

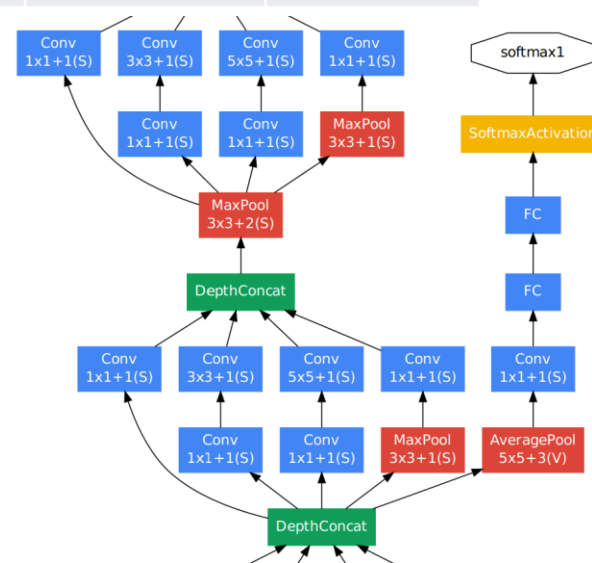
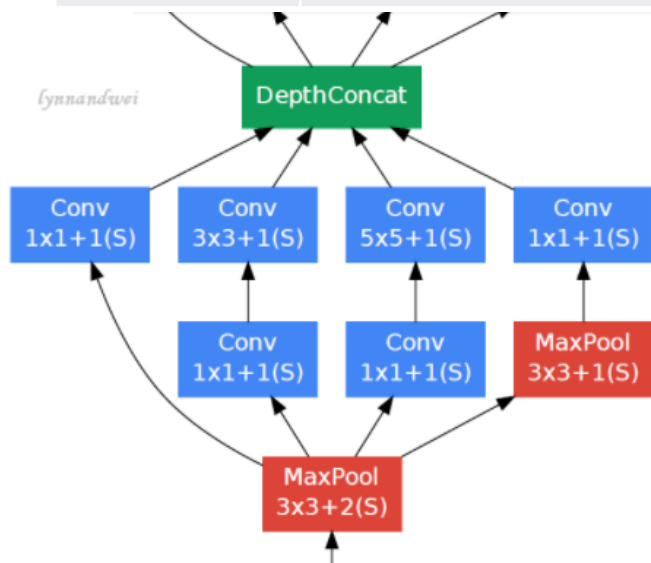


VGG16

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input				$224 \times 224 \times 3$	
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool				$112 \times 112 \times 64$	
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool				$56 \times 56 \times 128$	
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool				$28 \times 28 \times 256$	
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool				$14 \times 14 \times 512$	
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool				$7 \times 7 \times 512$	

神经网络的拓扑调节-Inception

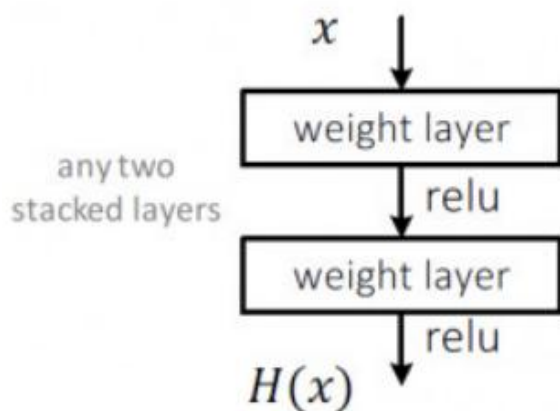
网络	主要创新	Top5错误率	网络层数
GoogLeNet	提出inception结构	6.67%	22
BN-Inception	提出 Batch Normalization, 用3x3代替5x5	4.82%	—
Inception-v3	将一个二维卷积拆成两个一维卷积, 辅助分类器的全连接层做BN	3.5%	42
Inception-v4	inception模块化, 结合ResNet的跳转结构	3.08%	—



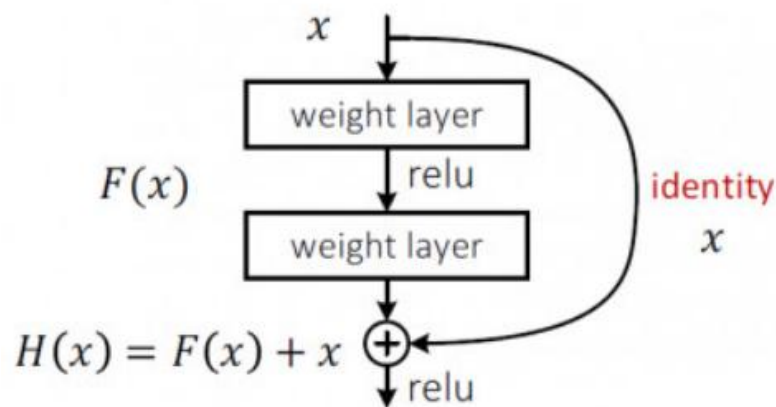
神经网络的拓扑调节-ResNet

- 残差块

- Plain net



- Residual net

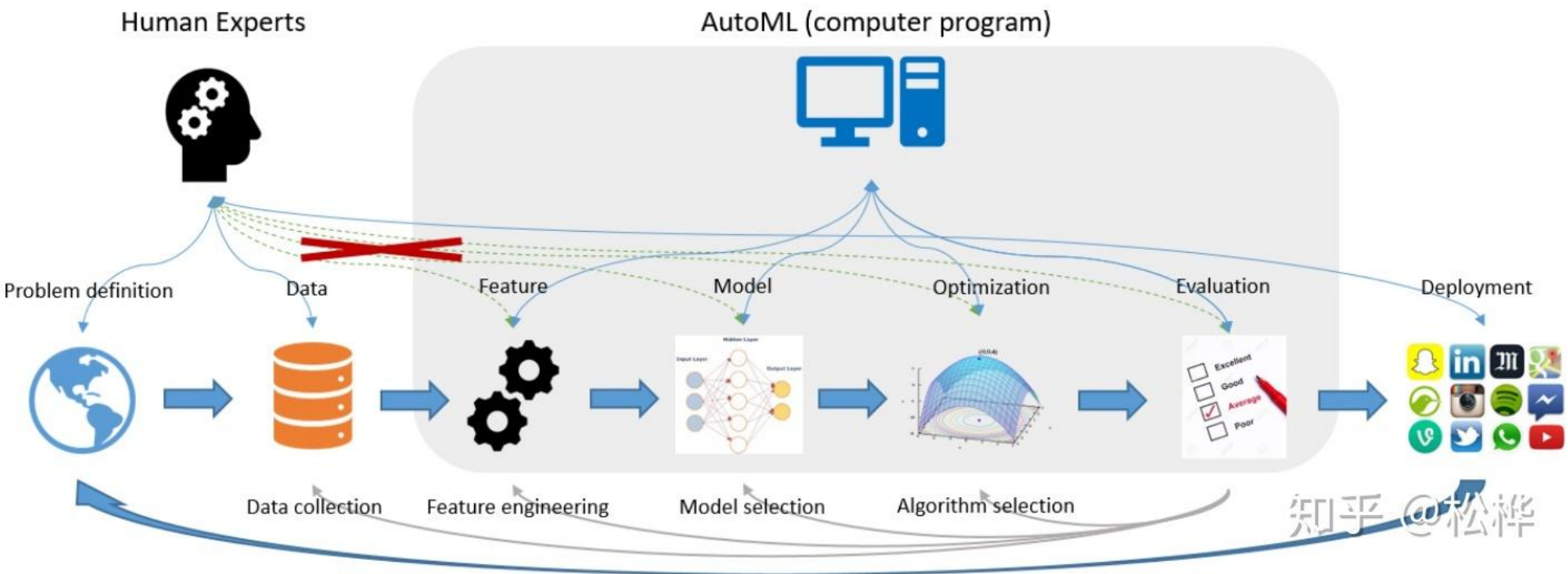


- $H(x) = F(x) + x$

- 残差网络在训练时更容易收敛

H(x) 偏导至少有一个1。表明残差网络可以无损的传播梯度，不会导致梯度消失，所以残差网络的训练会更容易。

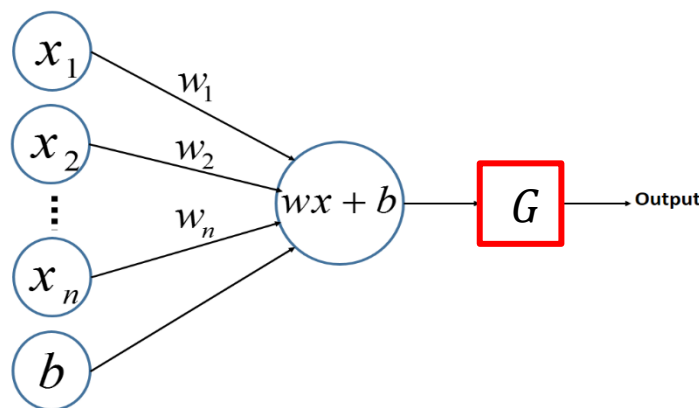
AutoML (网络结构搜索NAS)



► <https://zhuanlan.zhihu.com/p/93109455>

► 基于强化学习的方法、进化算法、贝叶斯优化、梯度方法等等

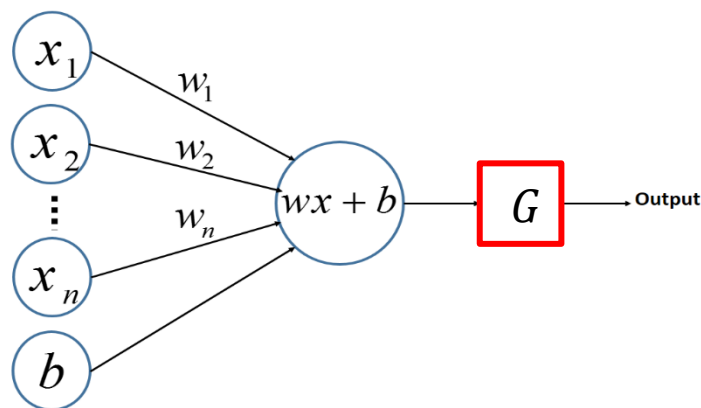
选择合适的激活函数



神经网络中激活函数的主要作用是提供网络的非线性建模能力

- 在神经元中，输入的数据通过加权求和后，还被作用了一个函数 G ，这个函数 G 就是激活函数（Activation Function）。
- 神经网络中激活函数的主要作用是提供网络的非线性建模能力，如不特别说明，激活函数一般而言是非线性函数。
 - 假设一个示例神经网络中仅包含线性卷积和全连接运算，那么该网络仅能够表达线性映射，即便增加网络的深度也依旧还是线性映射，难以有效建模实际环境中非线性分布的数据。
 - 加入（非线性）激活函数之后，深度神经网络才具备了分层的非线性映射学习能力。
 - 因此，激活函数是深度神经网络中不可或缺的部分。

选择合适的激活函数



➤ 激活函数需具备的性质

- **可微性**： 神经网络的训练优化是建立在梯度下降基础上的，通过链式法则去求导，所以激活函数必须可微。
- **输出值的范围**： 激活函数的输出决定了下一层神经网络的输入，所以激活函数输出的范围非常重要。当激活函数输出值是有限的时候，对于一些比较大的输入也会比较稳定，基于梯度的优化方法会更加稳定；当激活函数的输出是无限的时候，模型的训练会更加高效，不过在这种情况下，一般需要更小的学习率。

sigmoid函数

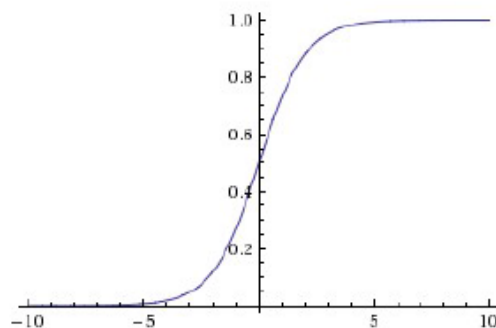
数学表达式

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\sigma'(z) = (1 - \sigma(z)) \cdot \sigma(z)$$

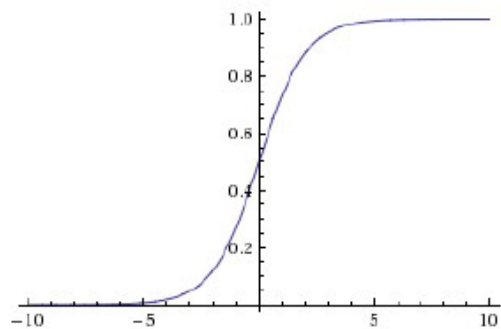
- Sigmoid 是最常见的非线性激活函数
- 能够把输入的连续实值变换为0和1之间的输出；如果是非常大的负数，那么输出就变为0；如果是非常大的正数，输出就变为1。
- 在中间斜率比较大的地方是神经元的敏感区，在两边斜率很平缓的地方是神经元的抑制区。

几何图像

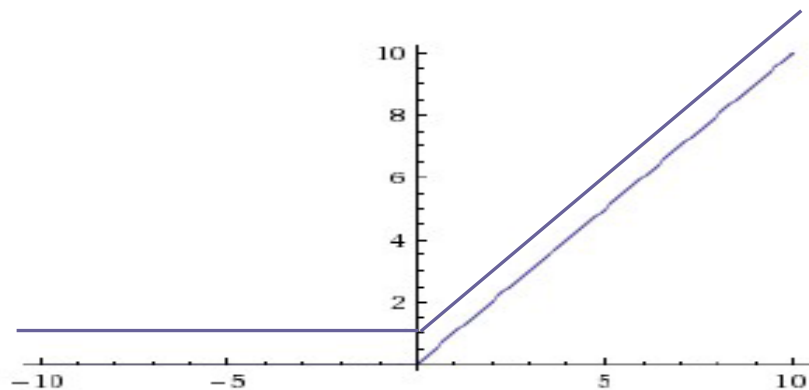


- 输出均值不为零，导致下一层均值产生偏置，改变数据分布，影响网络收敛性。
- 计算机进行指数运算速度慢
- 饱和性问题或梯度消失现象
 - 当我们输入的值x是比较大时候，它的梯度非常接近于0，就会导致我们整个参数的更新会非常慢

软饱和和硬饱和



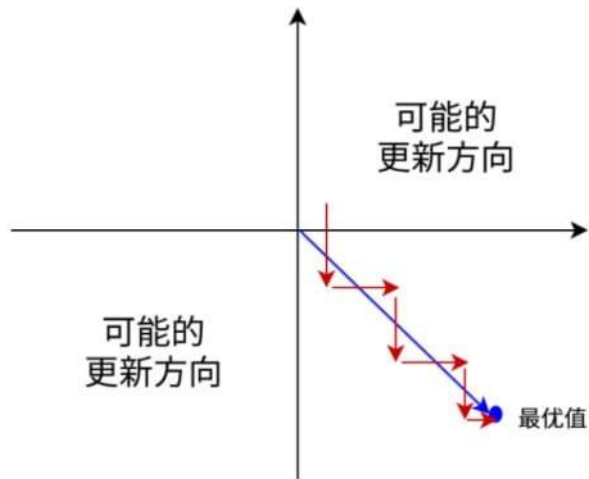
sigmoid函数



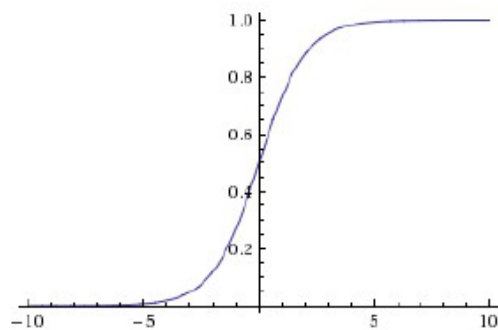
ReLU

- ▶ sigmoid 在定义域内处处可导，且两侧导数逐渐趋近于0，称为软饱和
- ▶ 与软饱和相对的是硬饱和激活函数，即： $f'(x)=0$ ，当 $|x| > c$ ，其中 c 为常数。
- ▶ Sigmoid 的软饱和性，使得深度神经网络在二三十年里一直难以有效的训练，是阻碍神经网络发展的重要原因。具体来说，由于在后向传递过程中，sigmoid向下传导的梯度包含了一个 $f'(x)$ 因子（sigmoid关于输入的导数），因此一旦输入落入饱和区， $f'(x)$ 就会变得接近于0，导致了向底层传递的梯度也变得非常小。此时，网络参数很难得到有效训练。**这种现象被称为梯度消失。**一般来说，sigmoid 网络在 5 层之内就会产生梯度消失现象。
- ▶ 梯度消失问题至今仍然存在，但被新的优化方法有效缓解了，例如Batch Normalization的逐层归一化，MSRA权重初始化等代表性技术。

偏移现象



几何图像



Sigmoid函数的输出值均大于0，使得输出不是0的均值，这会导致后一层的神经元将得到上一层非0均值的信号作为输入，**改变数据分布**，这会对梯度产生影响，**影响网络收敛性**。

这一情况将影响梯度下降的运作，因为如果输入神经元的数据总是正数(比如在 $f=wx+b$ 中每个元素都 $x>0$)，那么关于 w 的梯度在反向传播的过程中，**将会要么全部是正数，要么全部是负数（具体依整个表达式 f 而定）**。这将会导致梯度下降权重更新时出现z字型的下降（如上图所示）。

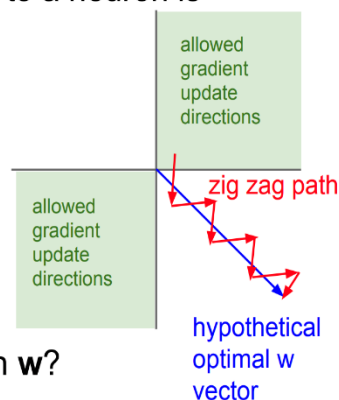
然而，可以看到整个批量的数据的梯度被加起来后，对于权重的最终更新将会有不同的正负，这样就从一定程度上减轻了这个问题的。因此，该问题相对于上面的神经元饱和问题来说只是个小麻烦，没有那么严重。

偏移现象

- 这一情况将影响梯度下降的运作，因为如果输入神经元的数据总是正数（比如在 $f=wx+b$ 中每个元素都 $x>0$ ），那么关于 w 的梯度在反向传播的过程中，~~将会要么全部是正数，要么全部是负数（具体依整个表达式 f 而定）~~。这将会导致梯度下降权重更新时出现z字型的下降（如上图所示）。

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



What can we say about the gradients on w ?

Always all positive or all negative :(
(this is also why you want zero-mean data!)

$$f = \sum w_i x_i + b$$

$$\frac{df}{dw_i} = x_i$$

$$\frac{dL}{dw_i} = \frac{dL}{df} \frac{df}{dw_i} = \frac{dL}{df} x_i$$

because $x_i > 0$, the gradient $\frac{dL}{dw_i}$ always has the same sign as $\frac{dL}{df}$ (all positive or all negative).

<https://stats.stackexchange.com/questions/237169/why-are-non-zero-centered-activation-functions-a-problem-in-backpropagation>

tanh函数（双曲正切函数）

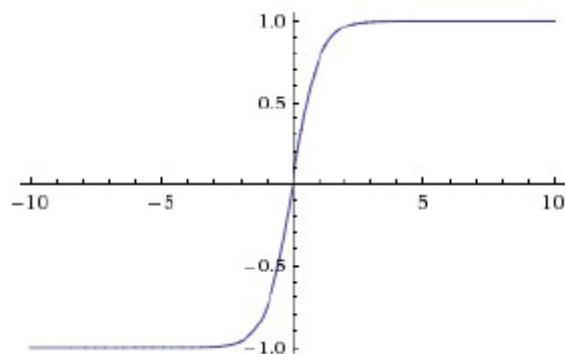
Sigmoid函数
存在神经元会
产生非0均值
的输出的问题

寻找解
决办法



$$\tanh(x) = \frac{\sinh(x)}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

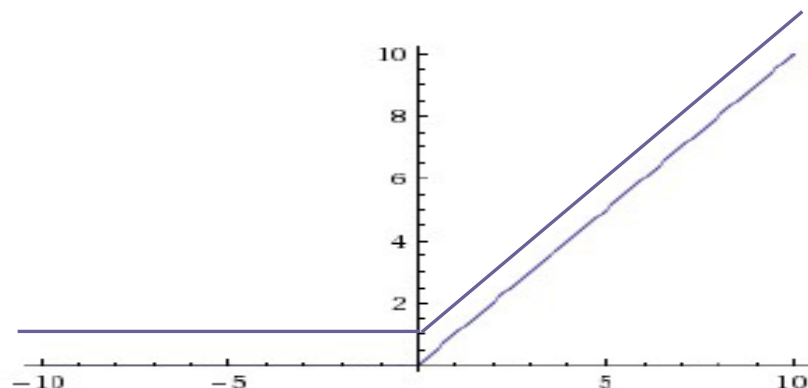
$$\tanh(x) = 2\textcolor{red}{sigmoid}(2x) - 1$$



- 与sigmoid相比，tanh是0均值的
- 在输入很大或是很小的时候，输出几乎平滑，梯度很小，不利于权重更新

ReLU函数

$$f(x) = \max(0, x)$$



寻找解决办法

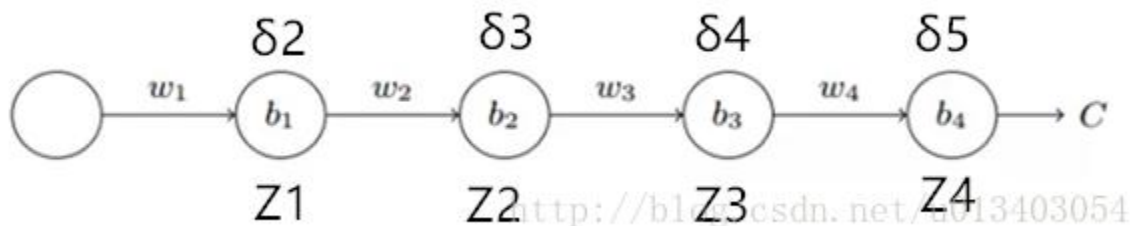


- ReLU 能够在 $x > 0$ 时保持梯度不衰减，从而缓解梯度消失问题
- Relu会使一部分神经元的输出为0，使得网络具有稀疏性，减少了参数的相互依存关系，缓解了过拟合问题的发生
- ReLU函数的输出不是零均值的，收敛受影响
- ReLU死掉
 - 如果学习率很大，反向传播后的参数可能为负数，导致下一轮正向传播的输入为负数。当输入是负数的时候，ReLU是完全不被激活的，这就表明一旦输入到了负数，ReLU就会死掉
- 输出范围是无限的（神经网络输出幅度不断增加）
- 梯度爆炸

tanh函数虽然解决了sigmoid函数存在非0均值输出的问题，但仍然没改变梯度消失问题

梯度爆炸

- ▶ 深度神经网络训练的时候，采用的是反向传播方式，该方式使用链式求导，计算每层梯度的时候会涉及一些连乘操作，因此如果网络过深。
- ▶ 如果连乘的因子大部分小于1，最后乘积的结果可能趋于0，也就是梯度消失，后面的网络层的参数不发生变化。
- ▶ 如果连乘的因子大部分大于1，最后乘积可能趋于无穷，这就是梯度爆炸



PReLU/Leaky ReLU 函数

ReLU 在 $x < 0$ 时，
ReLU完全不被激活

改进

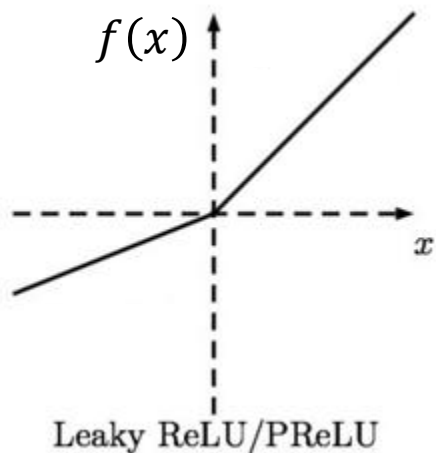


出现了ReLU的改进版本：

Leaky ReLU

$$f(x) = \max(\alpha x, x), \quad \alpha \in (0, 1)$$

- 负数区域内，Leaky ReLU有一个很小的斜率，可以避免ReLU死掉的问题



Leaky ReLU/PRelu

PReLU定义类似

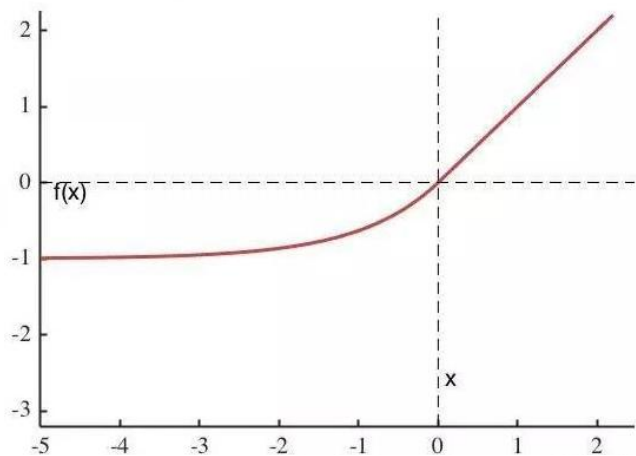
- α 为可调参数，每个通道有一个 α ，反向传播训练得到

ELU函数(Exponential Linear Unit)

融合sigmoid和ReLU



$$\text{ELU: } f(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$



- α 是可调参数，控制着ELU在负值区间的饱和位置
- ELU的输出均值接近于零，所以收敛速度更快
- 右侧线性部分使得ELU能够缓解梯度消失，而左侧软饱能够让ELU对输入变化或噪声更鲁棒，避免神经元死掉

GELU函数(Gaussian Error Linear Unit)

$$\text{GELU}(x) = 0.5x \cdot [1 + \text{erf}(\frac{x}{\sqrt{2}})]$$

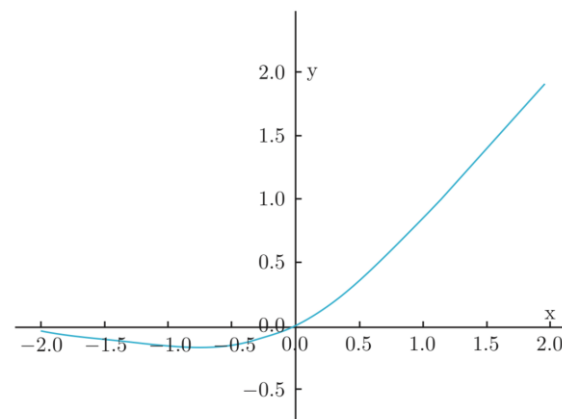
其中, $\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_1^x e^{-t^2} dx$ 表示标准正态分布的累积分布函数

近似计算:

$$\text{GELU} = 0.5x(1 + \tanh[\sqrt{2/\pi} (x + 0.044715x^3)])$$

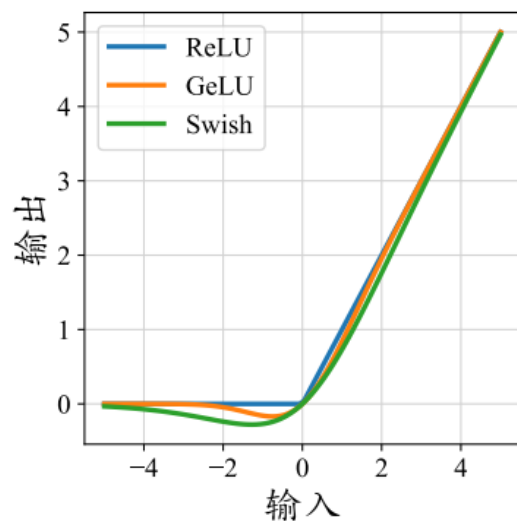
或

$$\text{GELU}(x) = x \cdot \text{sigmoid}(1.702x)$$



- 输入接近零时保持近似线性, 远离零时表现出非线性
- 计算复杂, 通常采用近似计算
- 平滑的激活曲线, 提供更好的梯度, 有益于模型收敛
- 用于BERT、GPT系列、T5等大模型中

SiLU函数(Sigmoid Linear Unit)



$$\text{SiLU}(x) = x \cdot \text{sigmoid}(x)$$

- 也称为Swish函数
- 更加平滑，可以更好地优化和更快地收敛，减少过拟合提高泛化能力
- 变体：SwishGLU
$$\text{SwiGLU}(x, W, V, b, c) = \text{SiLU}(xW + b) \cdot (xV + c)$$
- 用于DeepSeek、Llama、Palm、GLM等大模型中

Transformer中的FFN

- ▶ 原始Transformer采用ReLU激活函数

▶ 实验中

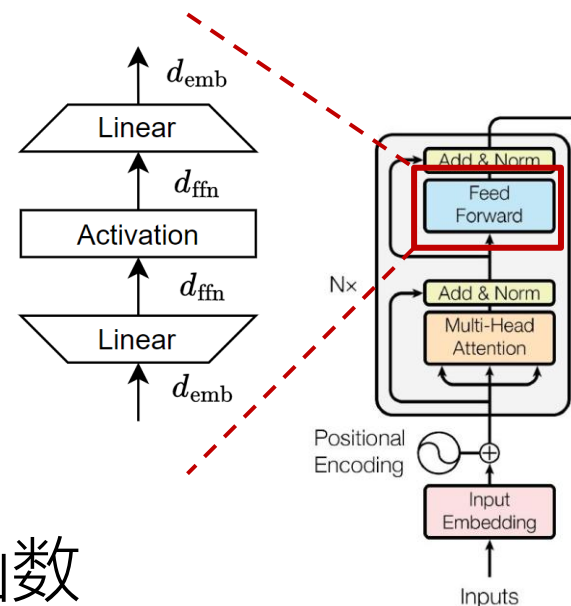
$$\text{FFN}(x, W_1, W_2, b_1, b_2) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$
$$\text{FFN}_{\text{ReLU}}(x, W_1, W_2) = \text{ReLU}(xW_1)W_2$$

- ▶ BERT、GPT等采用GELU激活函数

$$\text{FFN}_{\text{GELU}}(x, W_1, W_2) = \text{GELU}(xW_1)W_2$$

- ▶ Llama、DeepSeek等采用Swish激活函数

$$\text{FFN}_{\text{SwishGLU}}(x, W_1, W_2, W_3) = (\text{SiLU}(xW_1) \cdot (xW_3))W_2$$



选择恰当的损失函数

损失函数 $L = f(\hat{y}, y)$, $\hat{y}$ 是模型预测值, 是神经网络模型参数 W 的函数, 记作 $\hat{y} = H_w(x)$

从 $\mathbf{w}$ 角度看, 损失函数可以记为 $L(\mathbf{w}) = f(H_w(x), y)$

常用损失函数

➤ 均方差损失函数

- 均方差损失函数是神经网络优化常用的损失函数

$$L = \frac{1}{2}(y - \hat{y})^2 \quad \leftarrow \text{以一个神经元的均方差损失函数为例}$$

- 假设使用sigmoid函数作为激活函数, 则 $\hat{y} = \sigma(z)$, 其中 $z = wx + b$,

$$\frac{\partial L}{\partial w} = (\hat{y} - y)\sigma'(z)x \qquad \sigma'(z) = (1 - \sigma(z)) \cdot \sigma(z)$$

$$\frac{\partial L}{\partial b} = (\hat{y} - y)\sigma'(z)$$

所求的与 $\frac{\partial L}{\partial w}$ 和 $\frac{\partial L}{\partial b}$ 梯度中都含有 $\sigma'(z)$, 当神经元输出接近1时, 梯度将趋于0, 出现梯度消失, 导致神经网络反向传播时参数更新缓慢, 学习效率下降。

均方差损失函数+Sigmoid激活函数出现问题—>如何解决？

➤引入交叉熵损失函数

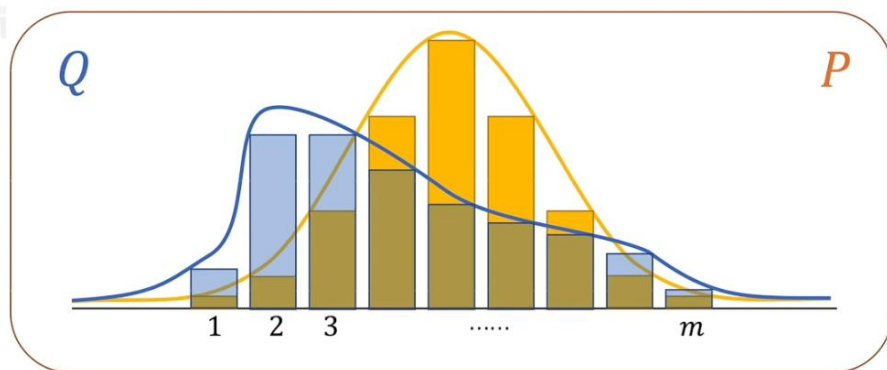
交叉熵损失+Sigmoid激活函数可以解决输出层神经元学习率缓慢的问题。

交叉熵

- 交叉熵（Cross Entropy）是Shannon信息论中一个重要概念，主要用于度量两个概率分布间的差异性信息。

$$H(p, q) = - \sum_{i=1}^n p(x_i) \log(q(x_i))$$

其中p为真实概率分布，而q为预测的概率分布。



交叉熵就是用来计算与现实世界最接近的概率模型

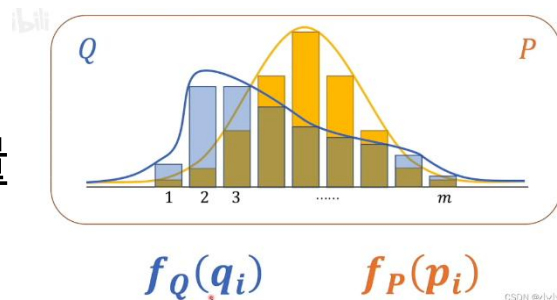
$f_Q(q_i)$

$f_P(p_i)$

CSDN @小小小方

交叉熵损失函数的意义和作用

- ▶ 交叉熵损失函数(Cross Entropy Loss)是机器学习中常用的损失函数，它能够用来衡量两个分布之间的距离。



- ▶ 交叉熵损失函数的意义是，它能够帮助机器学习模型从训练样本中估计出最接近目标分布的预测概率密度函数。
- ▶ 交叉熵损失函数常用于深度学习应用，如图像分类，自然语言处理，机器翻译等任务中。在这些任务中，将实际情况进行模型化，交叉熵损失函数能使模型更逼近真实结果；它也被经常被用作分类问题的代价函数，也就是它能够快速准确地判断两个分布之间的距离。
- ▶ 综上所述，交叉熵损失函数的意义在于，它可以用来衡量两个分布之间的距离，帮助机器学习模型估计出最接近真实分布的预测概率密度函数以及用于多类分类方面的模型判断等

交叉熵

$$H(p, q) = - \sum_{i=1}^n p(x_i) \log(q(x_i))$$

(1) 二分类

在二分的情况下，模型最后需要预测的结果只有两种情况，对于每个类别我们的预测得到的概率为 p 和 $1 - p$ ，此时表达式为（ $\log$ 的底数是 e ）：

$$L = \frac{1}{N} \sum_i L_i = \frac{1}{N} \sum_i -[y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i)]$$

其中：

- y_i —— 表示样本 i 的label，正类为 1，负类为 0
- p_i —— 表示样本 i 预测为正类的概率

对于一个具体样本，其为正样本的概率要么为1，要么为0。
都可以根据其最后的预测概率，计算其交叉熵

(2) 多分类

多分类的情况实际上就是对二分类的扩展：

$$L = \frac{1}{N} \sum_i L_i = - \frac{1}{N} \sum_i \sum_{c=1}^M y_{ic} \log(p_{ic})$$

其中：

- M —— 类别的数量
- y_{ic} —— 符号函数（0 或 1），如果样本 i 的真实类别等于 c 取 1，否则取 0
- p_{ic} —— 观测样本 i 属于类别 c 的预测概率

交叉熵

► Cross Entropy损失函数常用于分类问题中

- 我们希望根据图片动物的轮廓、颜色等特征，来预测动物的类别，有三种可预测类别：猫、狗、猪。假设我们当前有两个模型（参数不同），这两个模型都是通过sigmoid/softmax的方式得到对于每个预测结果的概率值：

模型1

预测	真实	是否正确
0.3 0.3 0.4	0 0 1 (猪)	正确
0.3 0.4 0.3	0 1 0 (狗)	正确
0.1 0.2 0.7	1 0 0 (猫)	错误

模型1对于样本1和样本2以非常微弱的优势判断正确，对于样本3的判断则彻底错误。

模型2

预测	真实	是否正确
0.1 0.2 0.7	0 0 1 (猪)	正确
0.1 0.7 0.2	0 1 0 (狗)	正确
0.3 0.4 0.3	1 0 0 (猫)	错误

模型2对于样本1和样本2判断非常准确，对于样本3判断错误，但是相对来说没有错得太离谱。

现在我们利用这个表达式计算上面例子中的损失函数值：

模型1：

$$\text{sample 1 loss} = -(0 \times \log 0.3 + 0 \times \log 0.3 + 1 \times \log 0.4) = 0.91$$

$$\text{sample 2 loss} = -(0 \times \log 0.3 + 1 \times \log 0.4 + 0 \times \log 0.3) = 0.91$$

$$\text{sample 3 loss} = -(1 \times \log 0.1 + 0 \times \log 0.2 + 0 \times \log 0.7) = 2.30$$

对所有样本的loss求平均：

$$L = \frac{0.91 + 0.91 + 2.3}{3} = 1.37$$

模型2：

$$\text{sample 1 loss} = -(0 \times \log 0.1 + 0 \times \log 0.2 + 1 \times \log 0.7) = 0.35$$

$$\text{sample 2 loss} = -(0 \times \log 0.1 + 1 \times \log 0.7 + 0 \times \log 0.2) = 0.35$$

$$\text{sample 3 loss} = -(1 \times \log 0.3 + 0 \times \log 0.4 + 0 \times \log 0.4) = 1.20$$

对所有样本的loss求平均：

$$L = \frac{0.35 + 0.35 + 1.2}{3} = 0.63$$

交叉熵损失函数特点

- (1) 当预测结果正确时，该正确类别的概率越大 ($q(x)$ 越接近1)，交叉熵损失函数越小。
- (2) 当预测结果正确，但正确类别的概率不够大时 ($q(x)$ 较小)，交叉熵损失函数较大。
- (3) 当预测结果错误时，交叉熵损失函数也很大。

交叉熵如何避免梯度消失

➤ 以二分类为例，交叉熵损失函数为：

$$L = -\frac{1}{m} \sum_{x \in D} (y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y}))$$

➤ 使用Sigmoid激活函数时的交叉熵损失函数为：

$$L = -\frac{1}{m} \sum_{x \in D} (y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y}))$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (\ln x)' = \frac{1}{x} \quad z = \mathbf{w}x + b$$

$$\frac{\partial L}{\partial \mathbf{w}} = -\frac{1}{m} \sum_{x \in D} \left[\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right] \cdot \frac{\partial \sigma(z)}{\partial \mathbf{w}} = -\frac{1}{m} \sum_{x \in D} \left[\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right] \cdot \sigma'(z) \cdot \mathbf{x} = \frac{1}{m} \sum_{x \in D} \frac{\sigma'(z) \cdot \mathbf{x}}{\sigma(z)(1-\sigma(z))} \cdot (\sigma(z) - y)$$



$$\sigma'(z) = (1 - \sigma(z)) \cdot \sigma(z)$$

$$\frac{\partial L}{\partial \mathbf{w}} = \frac{1}{m} \sum_{x \in D} (\sigma(z) - y) \cdot \mathbf{x}$$

同理得：

$$\frac{\partial L}{\partial b} = \frac{1}{m} \sum_{x \in D} (\sigma(z) - y)$$

sigmoid的导数被约掉，这样最后一层的梯度中就没有 $\sigma'(z)$ ，从而避免了梯度消失

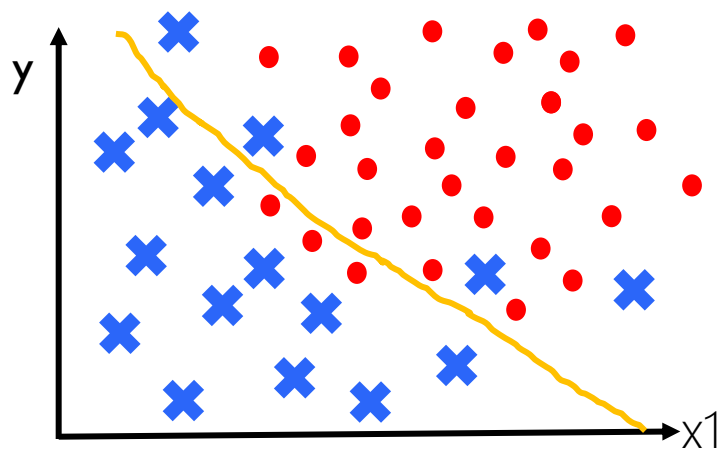
神经网络中损失函数的特性

- 同一个算法的损失函数不是唯一的
- 损失函数是参数(w, b)的函数
- 损失函数可以评价网络模型的好坏，损失函数越小说明模型和参数越符合训练样本(x, y)
- 损失函数是一个标量
- 选择损失函数时，挑选对参数(w, b)可微的函数（全微分存在，偏导数一定存在）
- 损失函数又称为代价函数、目标函数

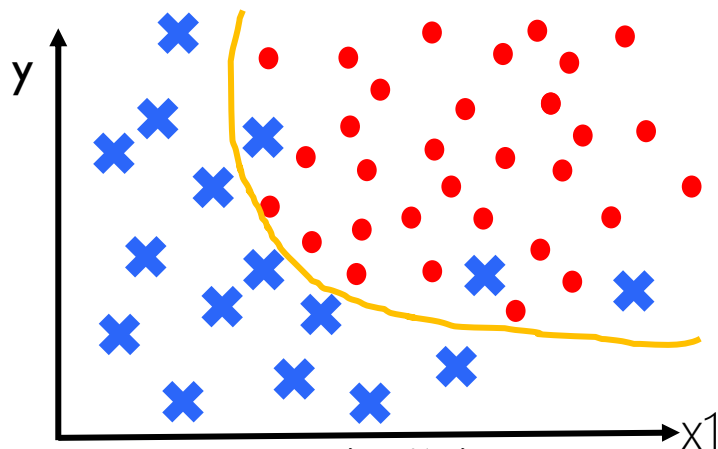
提纲

- ▶ 从机器学习到神经网络
- ▶ 神经网络训练
- ▶ 神经网络设计原则
 - ▶ 网络的拓扑结构
 - ▶ 激活函数
 - ▶ 损失函数
- ▶ 过拟合与正则化
- ▶ 交叉验证
- ▶ 小结

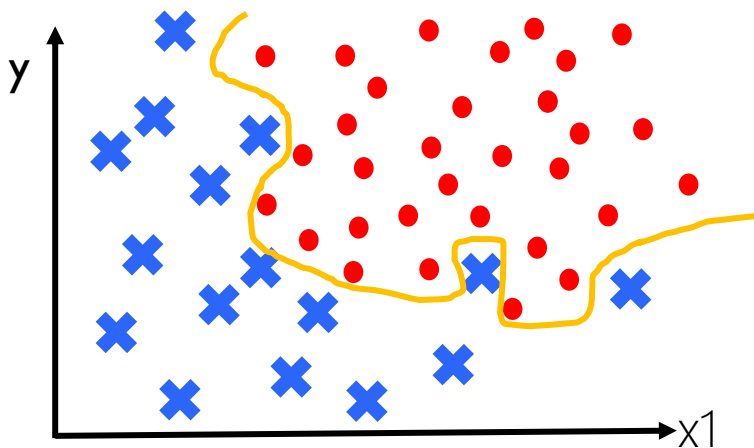
欠拟合和过拟合



欠拟合



合适拟合

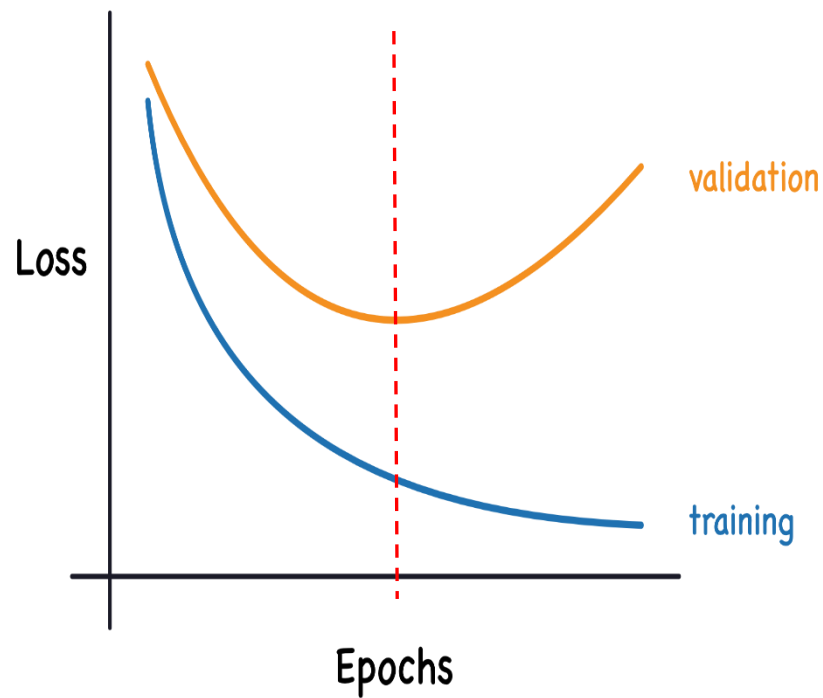
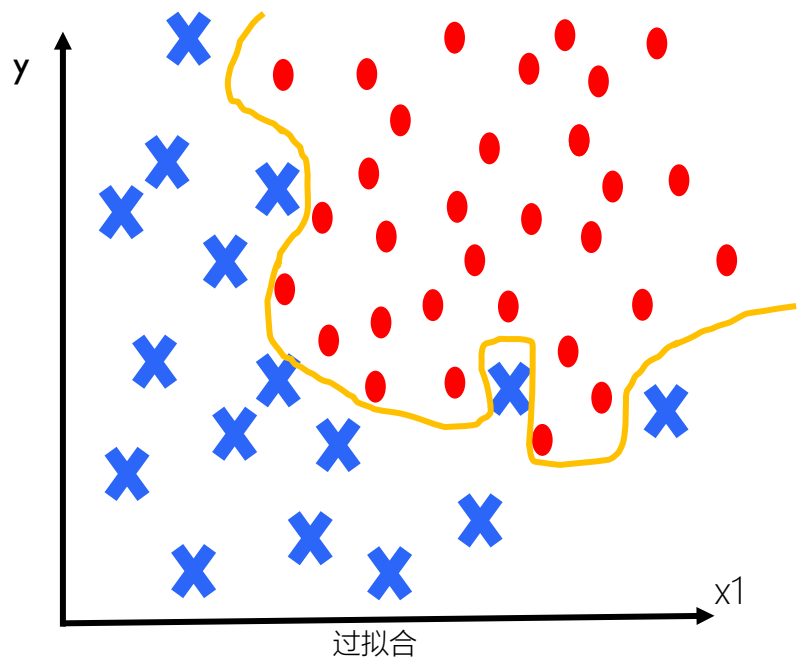


过拟合

欠拟合：训练考虑的维度太少，拟合函数无法满足训练集，误差较大。

过拟合，训练考虑的维度太多，使得拟合的函数很完美的接近训练数据集，但泛化能力差，对新数据预测能力不足。

欠拟合和过拟合



神经网络存在的问题及解决办法

神经网络规模不断加大，可能会导致一个致命问题：

过拟合，泛化能力差

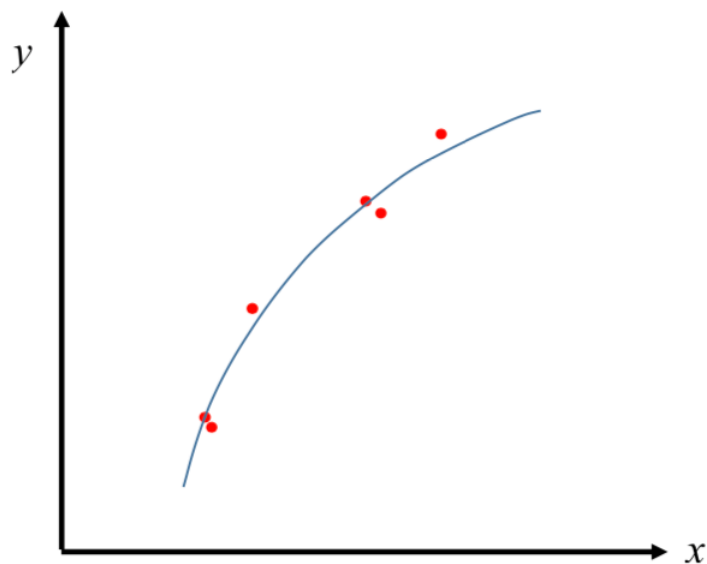


正则化

定义

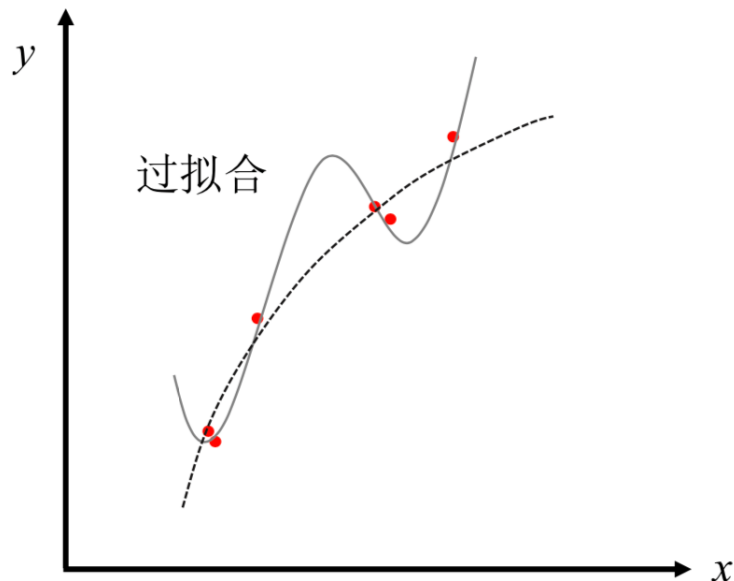
- 机器学习不仅要求数据在训练集上求得一个较小的误差，在测试集上也要表现好。因为模型最终是要部署到没有见过训练数据的真实场景。提升模型在测试集上的预测效果叫做**泛化**。
- **过拟合 (overfitting)** 指模型过度接近训练的数据，模型的泛化能力不足。具体表现为在训练数据集上测试的误差很低，但在验证数据集上的误差很大。
- 神经网络的层数增加，参数也跟着增加，表示能力大幅度增强，容易出现过拟合现象。
- 参数范数惩罚、稀疏化、Bagging集成、Dropout、提前终止、数据集扩增等正则化方法可以有效抑制过拟合

正则化思路



拟合函数

$$w_0 + w_1x + w_2x^2$$



过拟合

$$w_0 + w_1x + w_2x^2 + w_3x^3 + w_4x^4$$



$$L(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m \|y_i - \hat{y}_i\|^2$$

目标函数

加上惩罚项使 w_3 、 w_4 足够小

正则化思路

$$L(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m \|\mathbf{y}_i - \hat{\mathbf{y}}_i\|^2$$

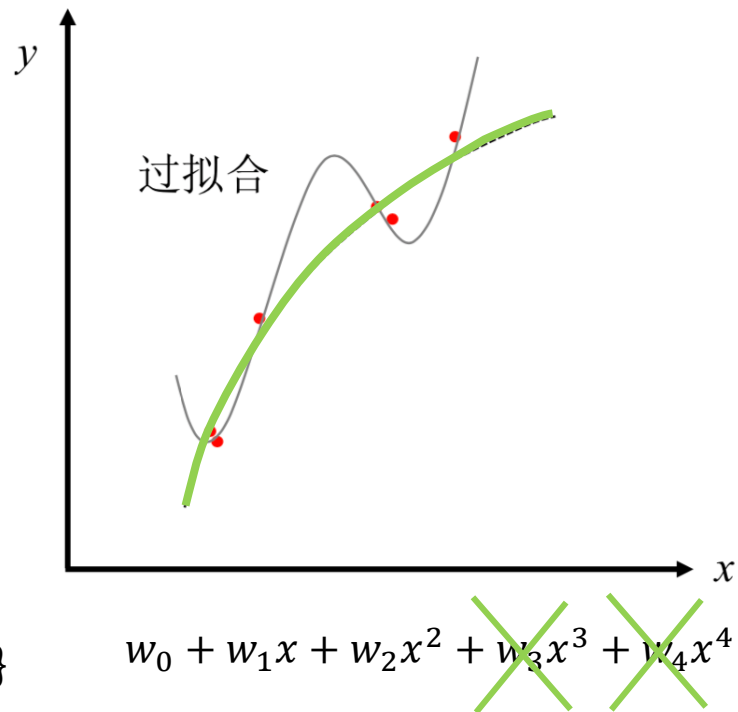


$$\min_{\mathbf{w}} \left\{ \frac{1}{2m} \sum_{i=1}^m \|\mathbf{y}_i - \hat{\mathbf{y}}_i\|^2 + C_1 * w_3^2 + C_2 * w_4^2 \right\}$$

C_1 、 C_2 可取常数，如取1000



$$\min_{\mathbf{w}} \left\{ \frac{1}{2m} \sum_{i=1}^m \|\mathbf{y}_i - \hat{\mathbf{y}}_i\|^2 + 1000 w_3^2 + 1000 w_4^2 \right\}$$



$$w_0 + w_1x + w_2x^2 + w_3x^3 + w_4x^4$$

要使目标函数最小，则应有 $w_3 \approx 0$ 、 $w_4 \approx 0$ ，除非 w_3 跟 w_4 起了非常大的作用。

我们就可以训练出一条绿色的曲线，比较好地将数据拟合出来，同时又有一定的泛化能力。

正则化思路

$$\min_{\mathbf{w}} \left\{ \frac{1}{2m} \sum_{i=1}^m \|\mathbf{y}_i - \hat{\mathbf{y}}_i\|^2 + C_1 * w_3^2 + C_2 * w_4^2 \right\}$$



在损失函数中增加一个惩罚项，惩罚高阶参数，使其趋近于0

$$\min_{\mathbf{w}} \left\{ \frac{1}{2m} \sum_{i=1}^m \|\mathbf{y}_i - \hat{\mathbf{y}}_i\|^2 + \theta \sum_{j=1}^k w_j^2 \right\}$$

核心思想：
在损失函数中对
不想要的部分加
入惩罚项

正则化项/惩罚项

θ 为正则化参数，神经网络中的参数包括权重 $\mathbf{w}$ 和偏置 $\mathbf{b}$ ，正则化过程仅对权重 $\mathbf{w}$ 进行惩罚，正则化项记为

$$\Omega(\mathbf{w})$$

正则化后的损失函数记为

$$\tilde{L}(\mathbf{w}; \mathbf{X}, \mathbf{y}) = L(\mathbf{w}; \mathbf{X}, \mathbf{y}) + \theta \Omega(\mathbf{w})$$

L^2 正则化

L^2 正则化项 $\Omega(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2$

目标函数 $\tilde{L}(\mathbf{w}; X, y) = L(\mathbf{w}; X, y) + \frac{\theta}{2} \|\mathbf{w}\|^2$

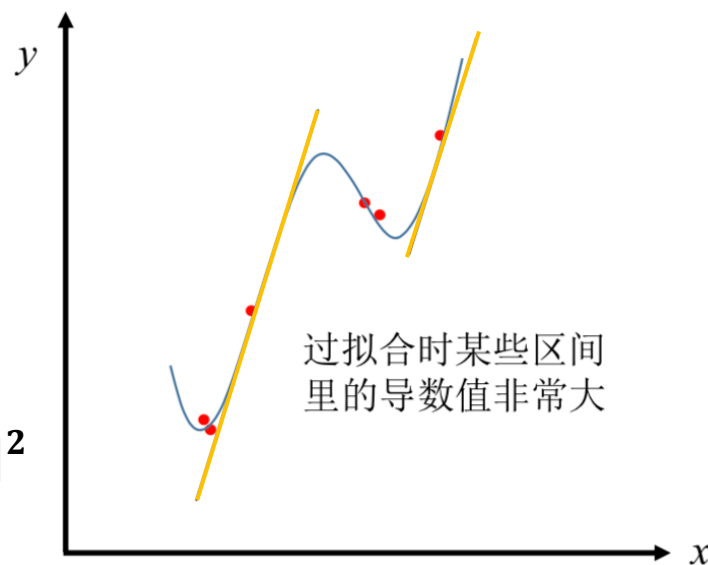
L^2 正则化如何避免过拟合?

$$\nabla_{\mathbf{w}} \tilde{L}(\mathbf{w}; X, y) = \nabla_{\mathbf{w}} L(\mathbf{w}; X, y) + \theta \mathbf{w}$$

单步梯度更新权重 $\mathbf{w} \leftarrow \mathbf{w} - \eta (\nabla_{\mathbf{w}} L(\mathbf{w}; X, y) + \theta \mathbf{w})$

$$\mathbf{w} \leftarrow (1 - \eta\theta) \mathbf{w} - \eta \nabla_{\mathbf{w}} L(\mathbf{w}; X, y)$$

在不使用 L^2 正则化时, 求导结果中 $\mathbf{w}$ 前系数为 1, 经变化后 $\mathbf{w}$ 前面系数为 $1 - \eta\theta$, 小于 1, 它的效果是减小 $\mathbf{w}$, 所以 L^2 正则化也叫权重衰减 (weight decay)。

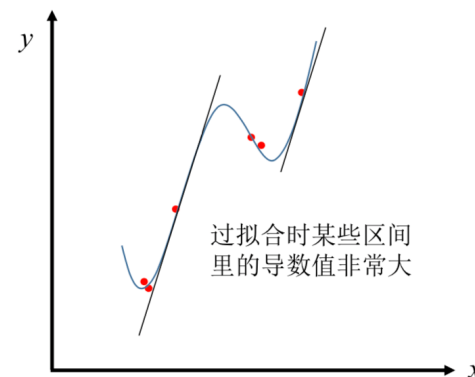


通过 L^2 正则化后, $\mathbf{w}$ 权重值变小

L^2 正则化

单步梯度更新权重 $\mathbf{w} \leftarrow \mathbf{w} - \eta(\nabla_{\mathbf{w}}L(\mathbf{w}; X, \mathbf{y}) + \theta\mathbf{w})$

$$\mathbf{w} \leftarrow (1 - \eta\theta)\mathbf{w} - \eta\nabla_{\mathbf{w}}L(\mathbf{w}; X, \mathbf{y})$$



为什么w小就能抑制过拟合呢？

- ▶ 直观来说，模型对于微小扰动的反馈差异大实际就是一个过拟合的表现。也就是模型中，高频分量不能多。
- ▶ 越复杂的模型，越是会尝试对所有的样本进行拟合，甚至包括一些异常样本点，这就容易造成在较小的区间里预测值产生较大的波动，这种较大的波动也反映了在这个区间里的导数很大，而只有较大的参数值才能产生较大的导数。因此复杂的模型，其参数值会比较大。
- ▶ L^2 正则就是限制了神经网络学到高频参数分量，更倾向于一个低频的平滑的函数，从而缓解过拟合。

L^1 正则化

L^1 正则化项是各个参数的绝对值之和

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_i |w_i|$$

L^1 正则化添加了一项
符号函数 $\text{sign}(\mathbf{w})$ 来
影响梯度

目标函数

$$\tilde{L}(\mathbf{w}; \mathbf{X}, \mathbf{y}) = L(\mathbf{w}; \mathbf{X}, \mathbf{y}) + \theta \|\mathbf{w}\|_1$$

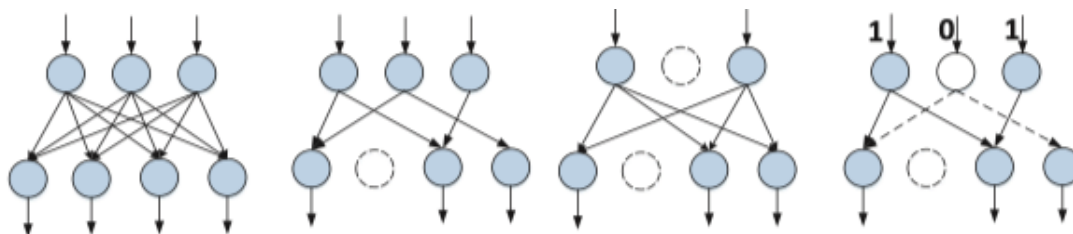
$$\nabla_{\mathbf{w}} \tilde{L}(\mathbf{w}; \mathbf{X}, \mathbf{y}) = \nabla_{\mathbf{w}} L(\mathbf{w}; \mathbf{X}, \mathbf{y}) + \theta \text{sign}(\mathbf{w})$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta (\nabla_{\mathbf{w}} L(\mathbf{w}; \mathbf{x}, \mathbf{y}) + \theta \text{sign}(\mathbf{w})) = \mathbf{w} - \eta \theta \text{sign}(\mathbf{w}) - \eta \nabla_{\mathbf{w}} L(\mathbf{w}; \mathbf{x}, \mathbf{y})$$

L^1 正则化通过加入一个符号函数，使得：当 w_i 为正时，更新后的 w_i 变小，
当 w_i 为负时，更新后的 w_i 变大，因此正则化后的效果就是让 w_i 接近 0，这样网络中的权重就会接近 0，从而也就减小了网络复杂度，防止了过拟合。

L1正则化的结果：稀疏化

- 训练时让网络中的很多权重或神经元为0（也就是模型结构的稀疏化）
- 90%的权重或神经元为0
- 降低正向传播时的计算量



稠密MLP

突触稀疏

神经元稀疏

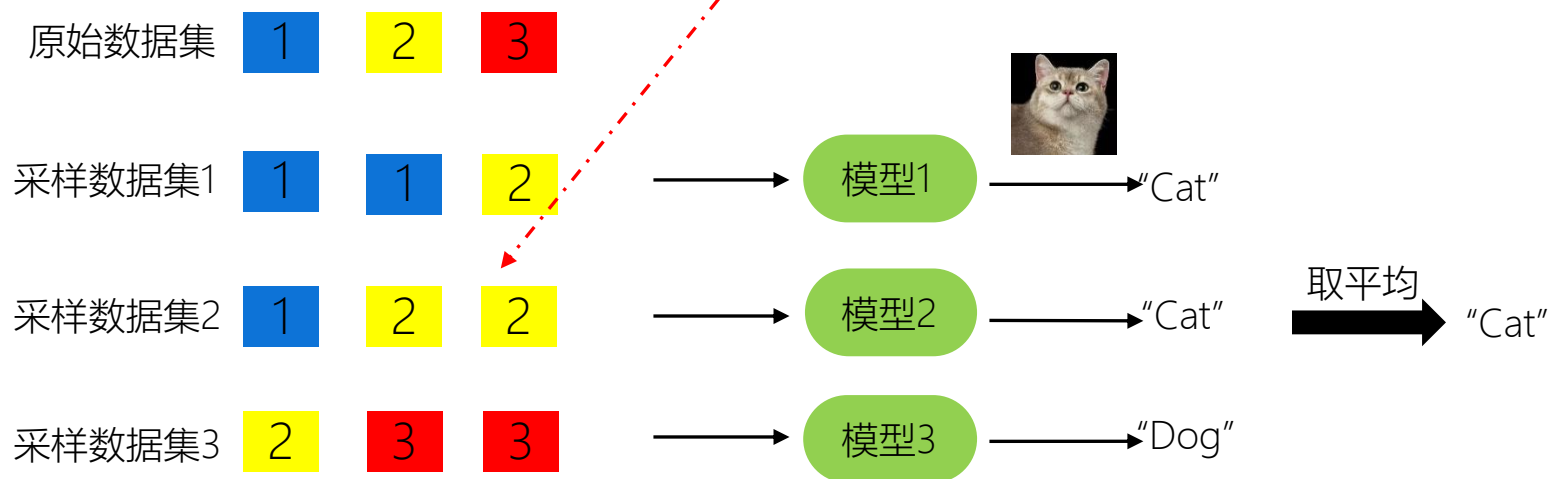
动态稀疏

Bagging集成方法

- Bagging 训练不同的模型来共同决策测试样列的输出，不同的模型即使在同一个训练数据集上也会产生不同的误差。
- Bagging可以多次重复使用同一个模型、训练算法和目标函数进行训练。
- Bagging的数据集从原始数据集中重复采样获取，数据集大小与原始数据集保持一致。

这种简单重复是否
有意义?

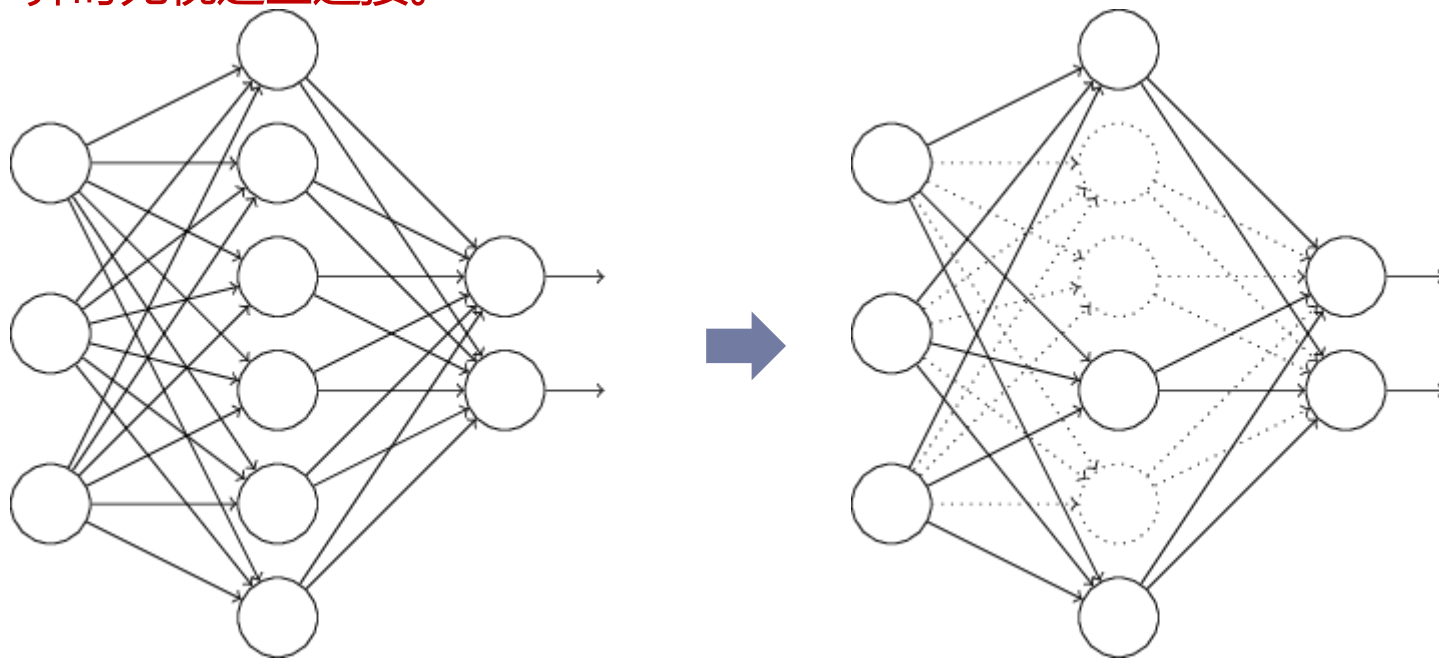
假设集成 $k=3$ 个网络模型



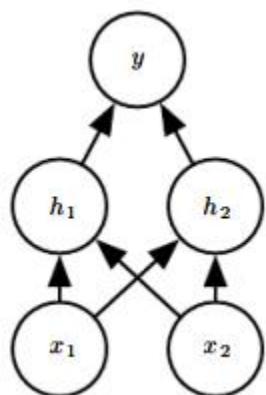
➤ 模型平均是减小泛化误差的一种可靠方法

Dropout 正则化

- L^2 和 L^1 正则化是通过在目标函数中增加一项惩罚项，Dropout正则化是通过在训练时暂时修改神经网络来实现的。
- Dropout正则化思路：在训练过程中随机地“删除”一些隐层单元，在计算时无视这些连接。

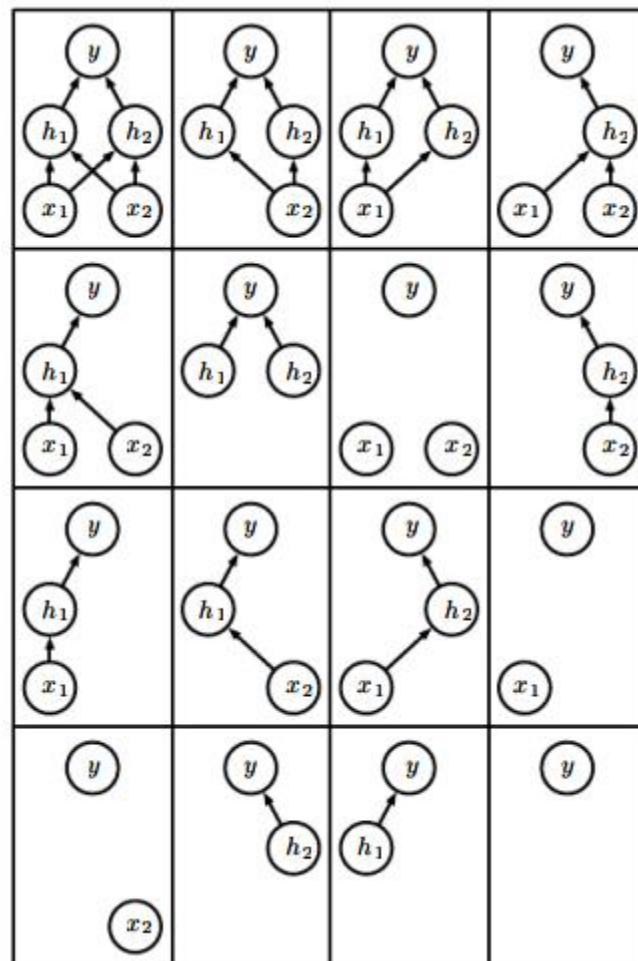


乘零的Dropout 算法

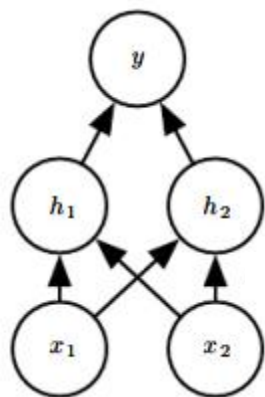


基础网络

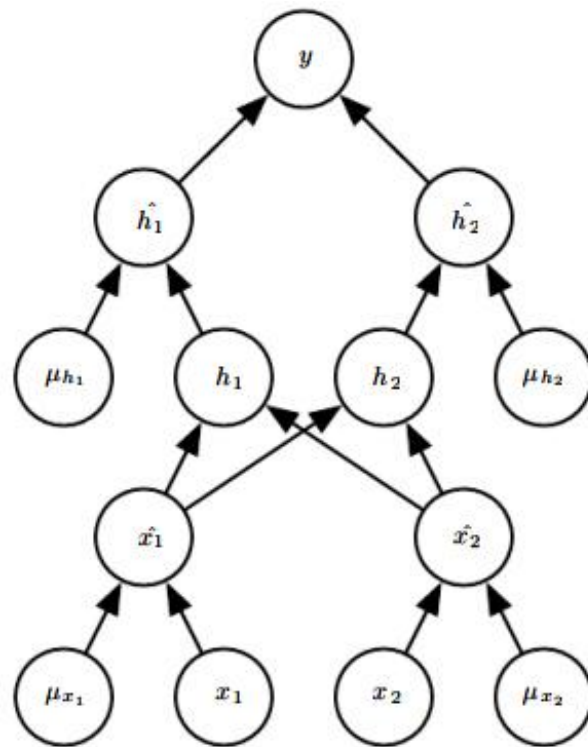
从基础网络中丢
弃不同的单元子
集形成子网络



子网络集成



基础网络



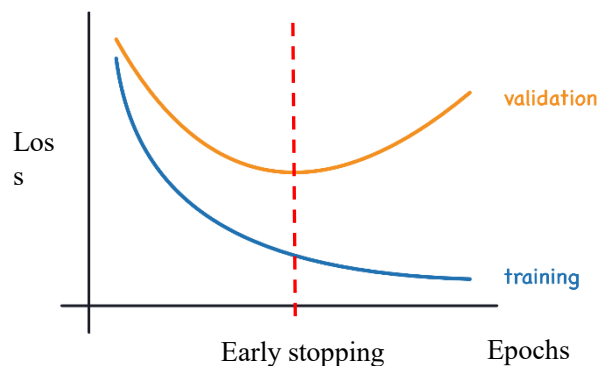
- 随机对掩码 μ 进行采样，输入单元的采样概率为0.8，隐藏单元的采样概率为0.5。网络中的每个单元乘以相应的掩码后沿着网络的其余部分继续向前传播
- 这里**Dropout**的意思不是 w 参数置为0，只是这些参数不更新。我们每一次迭代都是“随机”地去删掉一部分。第三次、第四次……都是这样，直至训练结束。

Dropout为何有助于防止过拟合

- ▶ **Dropout为什么有助于防止过拟合呢？**
- ▶ 在训练过程中会产生不同的训练模型，不同的训练模型也会产生不同的的计算结果。随着训练的不断进行，计算结果会在一个范围内波动，但是均值却不会有很大变化，因此可以把最终的训练结果看作是不同模型的平均输出。
- ▶ 消除或者减弱了神经元节点间的联合，降低了网络对单个神经元的依赖，从而增强了泛化能力。

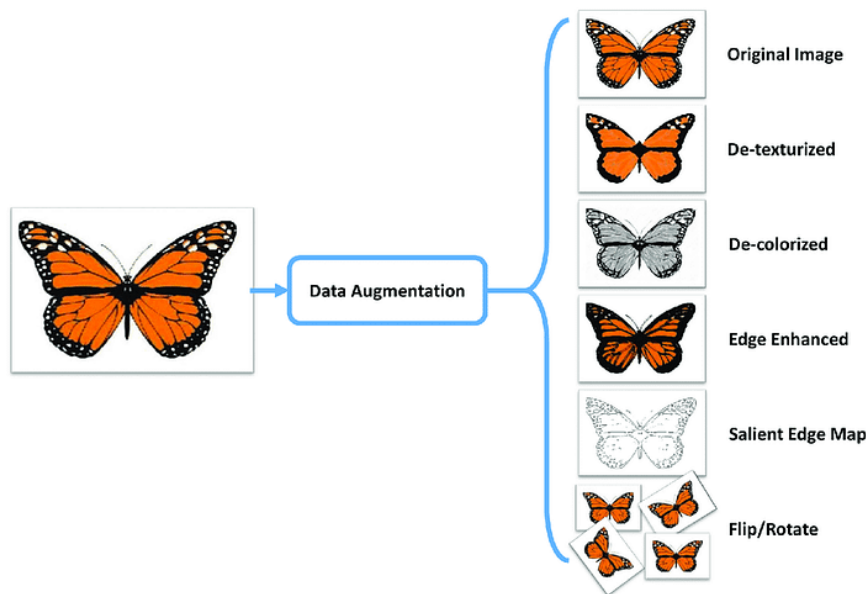
提前终止 (Early Stopping)

- 当训练较大的网络模型时，能够观察到训练误差会随着时间的推移降低但测试集的误差会再次上升。因此，在训练过程中返回预定迭代次数内测试误差达最低的参数设置，这种策略称之为提前终止。



数据增强 (Data Augmentation)

- 使用更多的数据进行训练，可对原数据集进行变换形成新数据集添加到训练数据中



Sources: <https://pub.towardsai.net/maximizing-the-impact-of-data-augmentation-effective-techniques-and-best-practices-c4cad9cd16e4>

批归一化 (Batch Norm)

- ▶ 在训练过程中每一层的输入分布随着前一层的参数变化而变化，即内部协变量偏移问题 (internal covariate shift) ， 会导致梯度消失或者梯度爆炸
- ▶ 对一个batch内的数据做归一化，将激活层的输入调整为标准正态分布（均值为0，方差为1）

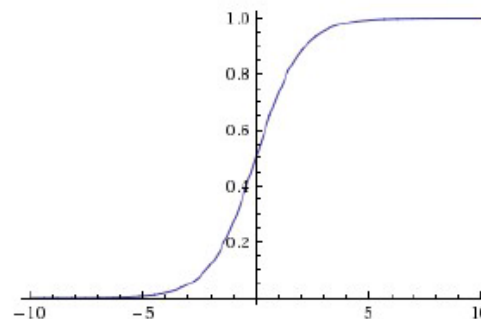
$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

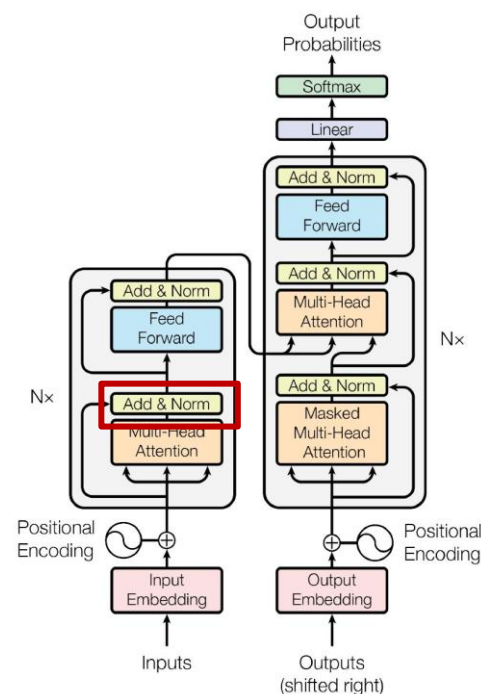
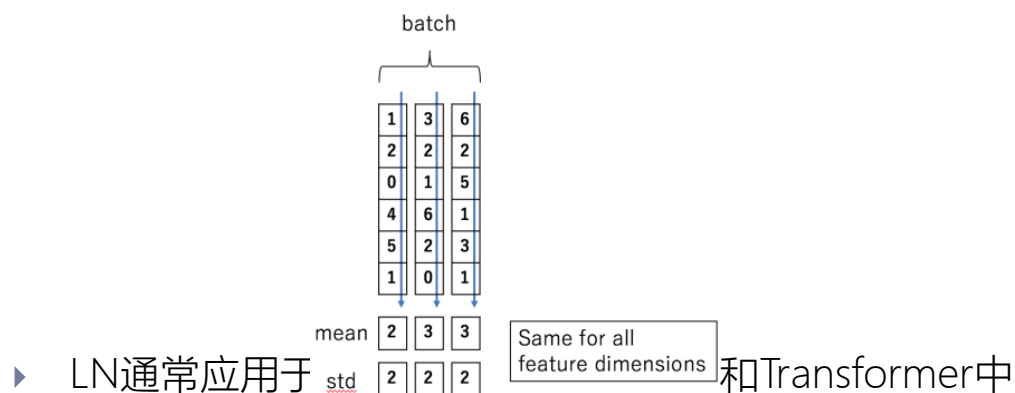
- ▶ BN通常应用于CV领域，应用于CNN中



Ioffe et al. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. ICML. 2015.

层归一化 (Layer Norm)

- ▶ 批归一化不适用于序列处理问题中（如RNN和Transformer），因为序列长度不固定，最长的序列部分样本数变少，不能反映全局统计规律
- ▶ 分样本归一化，同一个样本内计算均值方差



Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton. "Layer Normalization." arXiv:1607.06450, 2016

其他正则化方法

- ▶ 多任务学习

- ▶ 多任务学习通过多个相关任务的同时学习来减少神经网络的泛化误差。

- ▶ 参数共享

- ▶ 强迫两个模型（监督模式下的训练模型和无监督模式下的训练模型）的某些参数相等，使其共享唯一的一组参数。

提纲

- ▶ 从机器学习到神经网络
- ▶ 神经网络训练
- ▶ 神经网络设计原则
 - ▶ 网络的拓扑结构
 - ▶ 激活函数
 - ▶ 损失函数
- ▶ 过拟合与正则化
- ▶ 交叉验证
- ▶ 小结

交叉验证

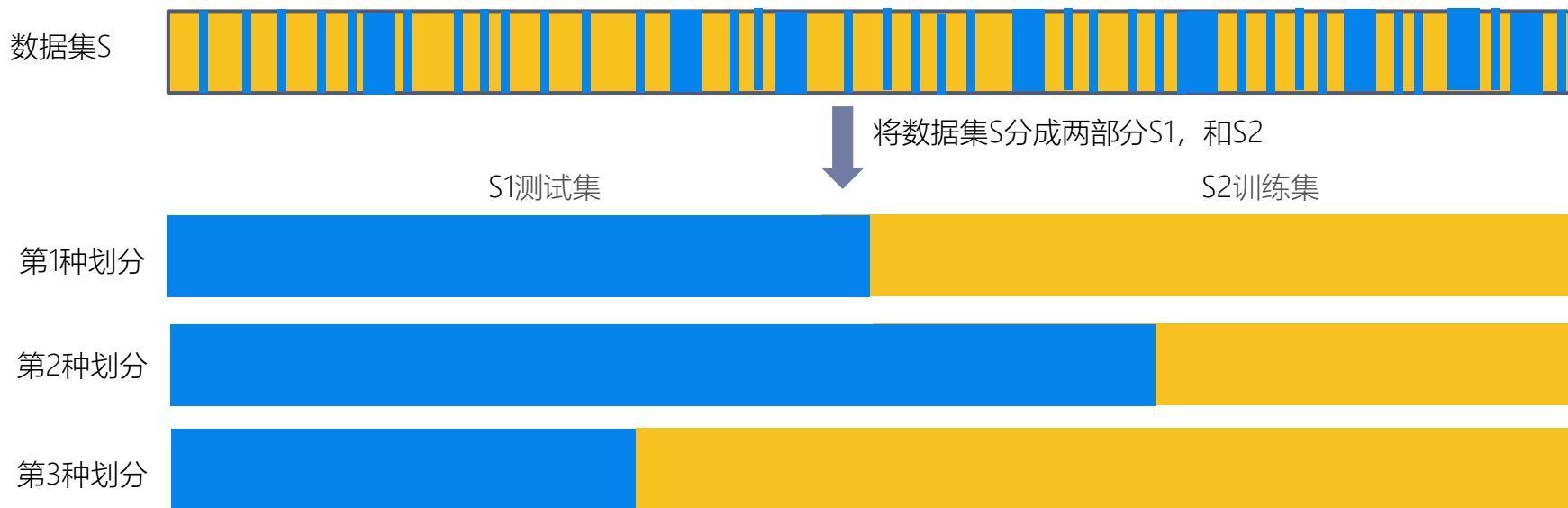
- 传统机器学习中，将数据集划分为训练集和测试集
 - 利用训练集进行训练
 - 利用测试集评估算法效果
- 划分测试集的目的
 - 避免过拟合
 - 评判各模型的鲁棒性

交叉验证

- 深度学习的数据集划分：训练集、验证集、测试集
 - 利用训练集训练模型参数（权重和偏置）
 - 利用验证集确定神经网络超参数（网络结构、学习率、训练迭代次数等）
 - 利用测试集评估模型效果

最简单的验证方式-随机分

测试集和训练集

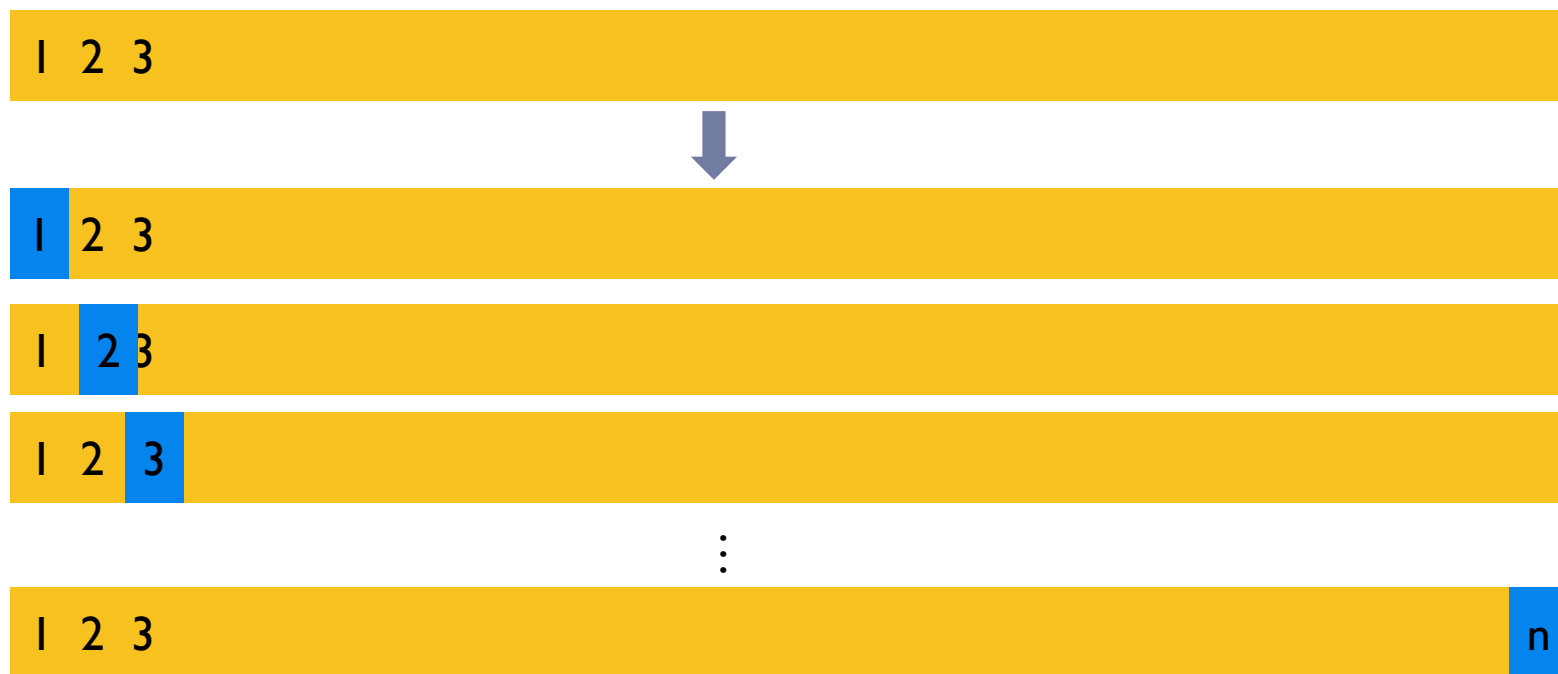


➤ 不同划分方式下，得到的MSE(Mean Squared Error)变动较大

缺点：最终模型与参数的选取将极大程度依赖于你对训练集和测试集的划分方法
只有部分数据参与了模型的训练

Leave-one-out 交叉验证方法

数据集S包
含n个数据



每次取出一个数据作为测试集的唯一元素，而其他 $n-1$ 个数据都作为训练集用于训练模型和调参。最终训练出 n 个模型，得到 n 个MSE。将这 n 个MSE取平均得到最终的test MSE。

缺点：计算量过大，耗费时间长

K-折交叉验证 (k-fold cross validation)

数据集S包
含n个数据
分成K=5份



不重复地每次取其中一份做测试集，用其他K-1份做训练集训练模型，之后计算该模型在测试集上的 MSE_i ，最后再将K次的 MSE_i 取平均得到最后的MSE

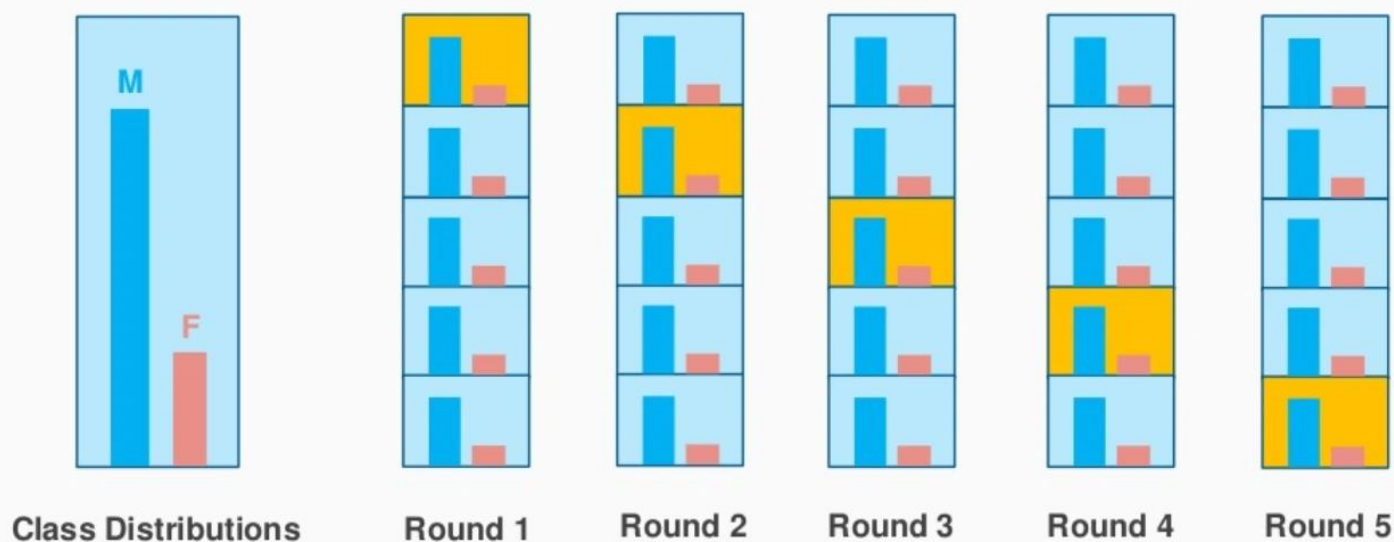
$$MSE = \frac{1}{K} \sum_{i=1}^K MSE_i$$

Leave-one-out cross-validation是一种特殊的K-fold Cross Validation (K=n)

优点：所有的样本都被作为了训练集和测试集，每个样本都被验证一次，相比Leave-one-out cross-validation，计算成本降低，耗时减少

Stratified K-Fold Cross-Validation

- ▶ Stratified K-Fold（分层 K 折）是普通 **K-Fold** 交叉验证的改进版本，核心目标是解决**类别不平衡数据集**中，普通 K 折可能导致某一折的类别分布与原始数据差异过大、进而导致模型评估偏差的问题。它通过“分层抽样”确保每一份折叠（Fold）中，各类别的样本比例与原始数据集完全一致，从而让模型评估结果更可靠。



From: <https://www.analyticsvidhya.com/blog/2021/05/4-ways-to-evaluate-your-machine-learning-model-cross-validation-techniques-with-python-code/>

Stratified K-Fold Cross-Validation

► 核心原理：为什么需要“分层”？

- 在分类任务中（尤其是二分类 / 多分类），若数据存在类别不平衡（例如：医疗数据中“患病样本”仅占 5%， “健康样本”占 95%），普通 K-Fold 的随机划分可能出现极端情况：
 - 某一折中几乎全是“健康样本”，模型训练后在该折上的验证 accuracy 会极高（但无实际泛化能力）；
 - 另一折中“患病样本”比例远超原始数据，导致验证误差被放大。
- 而**分层 K 折**通过“按类别比例抽样”避免这一问题：
假设原始数据中“正样本：负样本 = 1:9”，那么分层 K 折的每一份折叠中，“正样本：负样本”也会保持 1:9 的比例，确保训练集和验证集的类别分布始终与原始数据一致，评估结果更贴近模型真实泛化能力。

深度学习与交叉验证

- 在图像分类、目标检测、机器翻译等大部分应用领域，通常可以收集到较大规模的数据集，因此无需使用K-折交叉验证
- 在某些领域，如医学图像处理、遥感图像处理等，数据收集较为困难，依然需要使用K-折交叉验证来评估模型的泛化能力，避免神经网络过拟合

小结

- 从机器学习到神经网络

掌握单变量线性回归、多变量线性回归、感知机、神经元、激活函数、偏置等的概念和相关用法

- 正向传播、反向传播

了解数据正向传播和反向传播的过程。

- 激活函数

了解常用激活函数的优缺点

- 损失函数

了解损失函数的作用和种类、掌握至少一种验证方式。

- 过拟合与正则化

了解过拟合出现的原因并掌握防止过拟合的正则化方法

- 交叉验证

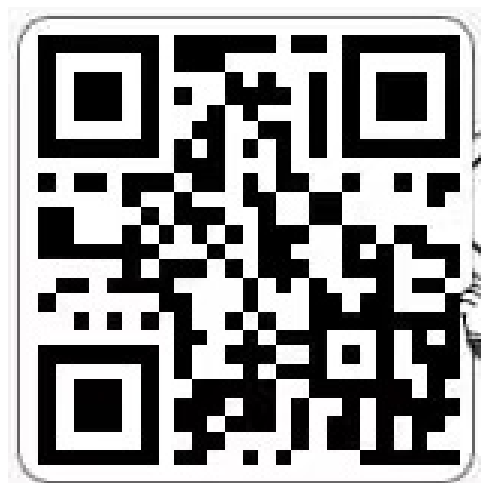
掌握至少一种交叉验证的原理和方法



谢谢大家！



课程公众号



B站账号